



NEXUS

AI 기반 물류 자동화 및 재고관리 플랫폼

Vision 판정 · LLM 에이전트 의사결정을 입출고 및 재고 관리와 연계

KT AIVLE SCHOOL 9기 2반 5조 | NEXUS Project



팀 구성원 소개



장문경

PM/TL

Nexus 전 계층 구축 및 개발
(AI·BE·FE·인프라·배포)
팀 설계·PR 리뷰·기술 멘토링 총괄
Multi Agent 검수 파이프라인
설계·구현(WBF, RAG, UBCI)
투 트랙 개발 관리 - 선행 개발 후 매
주차 칸반 보드로 팀원 과제 부여



박민우

BE

PostgreSQL 기반
DB Schema 설계
Core API 연동 계약 관리
DB Lock 기반 동시성 제어 구현
Report Agent 구축 및 튜닝



박준희

FE

Jotai 기반 관리자·계정 UI 개발
HITL·Data Grid·알림 UI 구축
자동발주·동적 가격 Agent 연동



서다은

BE

Redis/Celery 기반
검수 작업 처리
Restock Agent 구축
JWT·SSE 실시간 알림
피킹·출고 API 및 재검수 연동



소한민

BE/FE

물류 정책 데이터 정리
ChromaDB·
FDS 배치 구축
모바일 PWA·바코드·
통계 화면 개발
S3 storage 구축

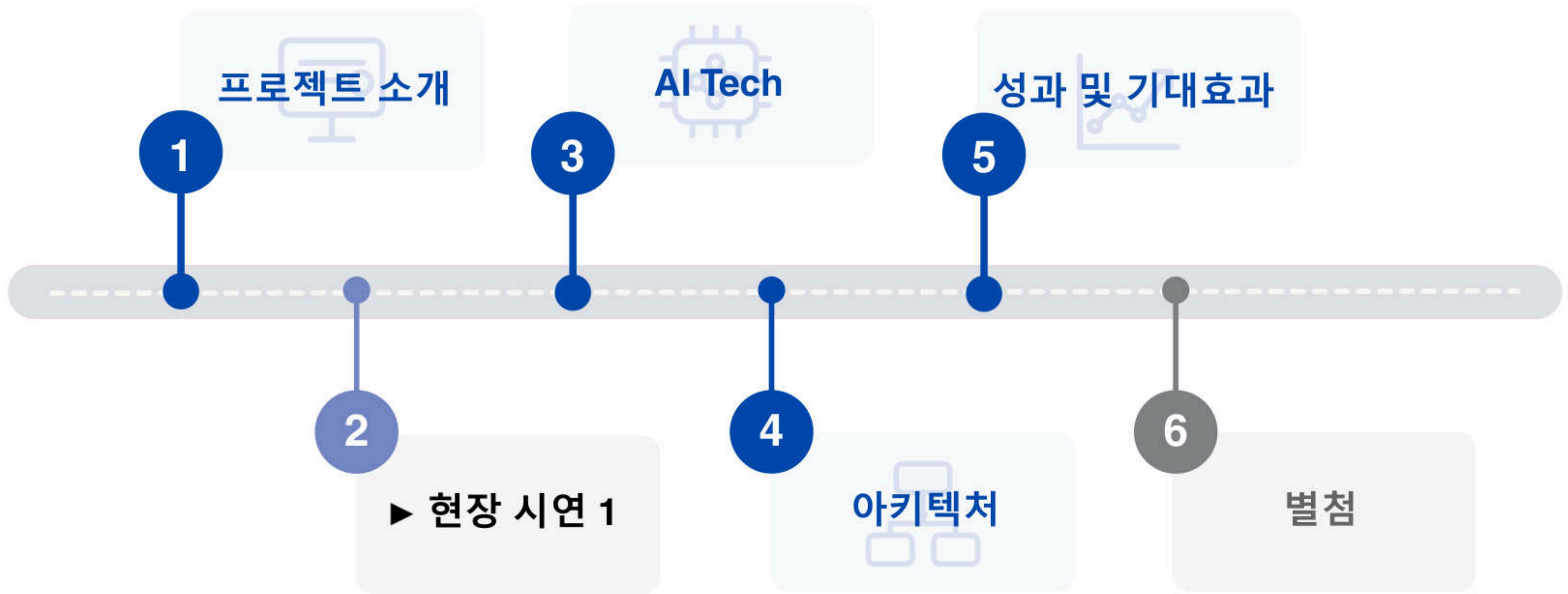


홍경표

AI

LangGraph Multi
Agent·RAG 설계 및
LLMOps 고도화
LangChain 구축
사내 정책서 확립

목차



01. 프로젝트 소개

문제 정의

연간 2.5조 원 규모의 온라인 서적 시장, 반품 상태 판정은 여전히 사람에게 의존합니다.



국가데이터처, 「온라인쇼핑동향조사」
상품군별 온라인쇼핑 거래액 '서적'

출처 ① 국가데이터처, 「온라인쇼핑동향조사」 2021~2025년(2025년 잠정치) ② 머니투데이, 2024.09.11.
③ Belin·Hedman, Chalmers University of Technology, p.30 ④ 국민일보, 2025.12.22.

머니투데이

약 8,000원

택배·검수·양품화 비용

해외 유통 물류센터 실측 사례

약 121초/건

수작업 반품 검사·재포장 실측

국민일보

'최상' 등급도 파손·하자·오배송

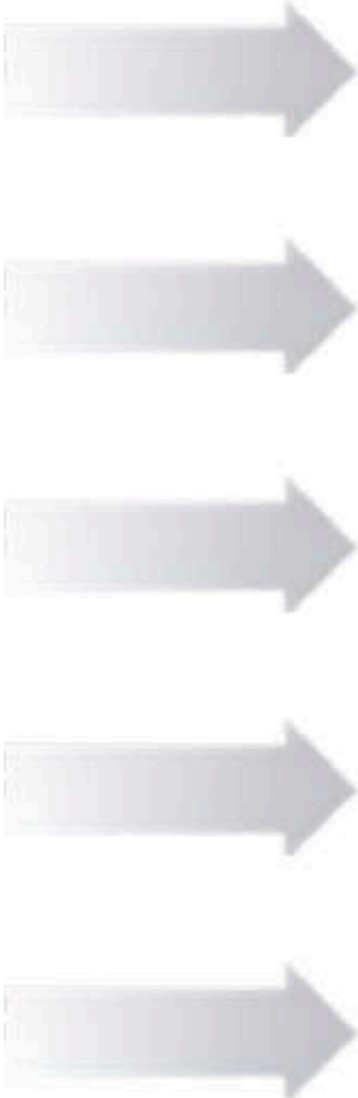
잘못된 상태 판정은 재반품과 불신으로 연결

서비스 제안

“주관적 판단에서, AI 기반 표준 프로세스로”

AS-IS : 기존 방식의 한계

- 1 육안 판정
사람마다 등급 기준 상이
- 2 주관적 책정
매입가·판매가를 경험에 의존
- 3 사후 발주 검토
재고 소진 확인 후 발주
- 4 사후 발견
이상거래·사기·반품 발생 후 확인
- 5 감사 불가
등급 산정 근거 재현 어려움



TO-BE : Nexus가 바꾸는 것

- 1 AI 멀티에이전트 판독
결정론 산식으로 등급 일관성 확보
- 2 UBCI 기반 가격 산정
점수 기반 동적 가격 자동 산정
- 3 AI 선제 발주 제안
저재고·반려 이벤트 기반 수량 제안
- 4 상시 AI 관제
FDS 룰엔진 + Analyst Agent 상시 관제
- 5 검수 원장 기록
판정 전 과정 기록으로 100% 재현 가능

핵심 기능

물품 검수부터 운영 판단까지, 5가지 기능으로 연결

검수 결과를 판매 · 포장 · 발주 · 이상거래 관리까지 이어갑니다.



01

AI 멀티에이전트 검수

물품 상태를 분석해 등급을 판정



02

등급 · 가격 · 품질보증서

등급별 판매가와 보증서 생성



03

3D Bin Packing

도서 크기에 맞는 포장 상자 추천



04

자동 발주 에이전트

부족 재고를 찾아 발주안을 제안



05

이상거래 탐지 · 주간 인사이트

의심 거래를 알리고 운영 현황을 요약

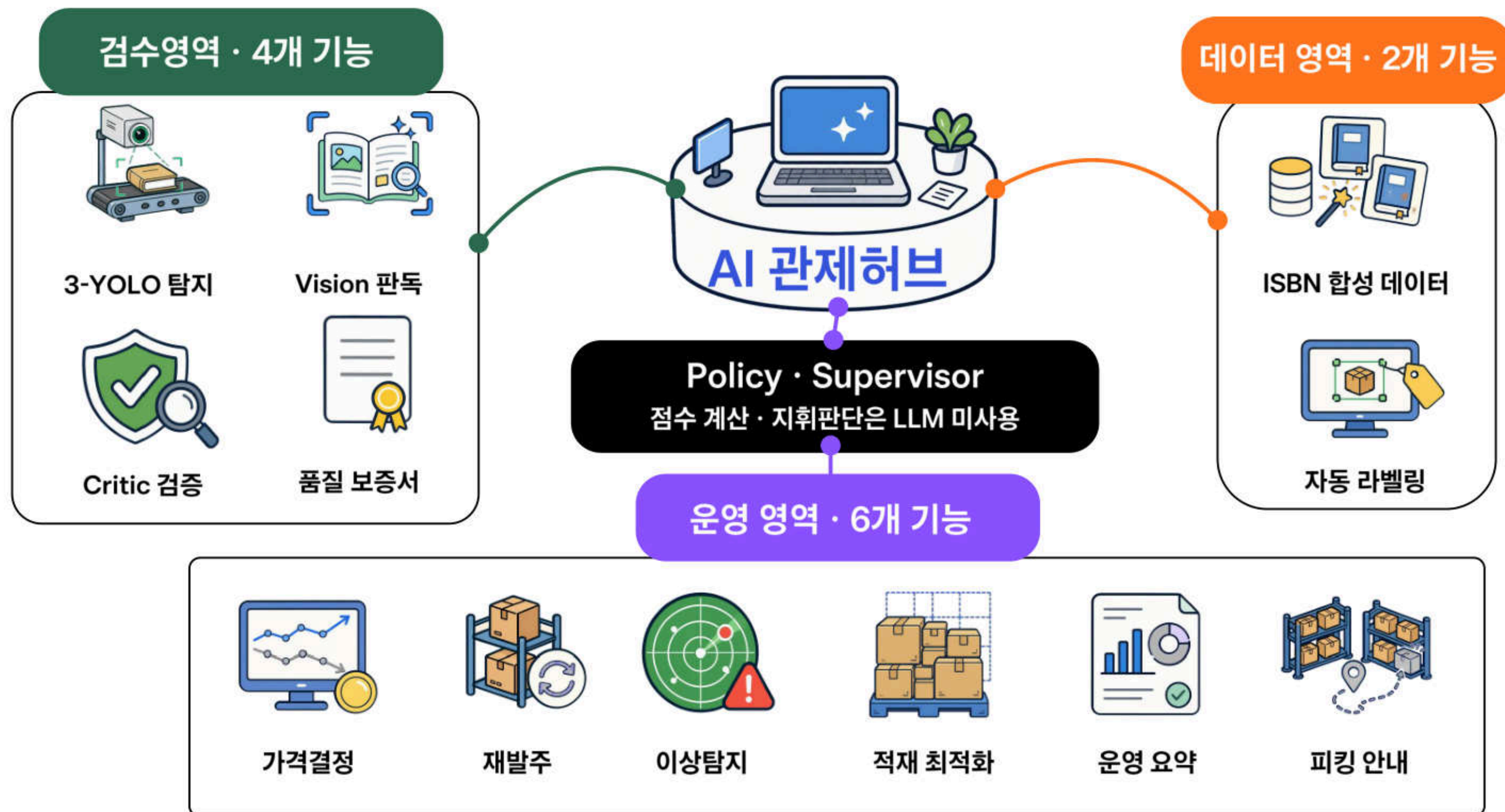
02. 현장 시연



입고는 신품과 중고로 나뉩니다

03. AI Tech

AI MAP



앙상블(Ensemble), WBF 정의

앙상블(Ensemble)

여러 요소가 함께 어우러져 하나의 조화로운 결과를 만드는 것



AI에서는 서로 다른 모델의 판단을 함께 사용하는 방식

WBF (Weighted BBox Fusion)



정밀탐지 모델의 박스 쪽으로 위치를 더 반영해 하나의 박스로 다시 그립니다.

※ 같은 결합 종류이며 겹친 박스만 합침 (모델 가중치 precision 1.5 · recall 1.0 · doodle 1.0)

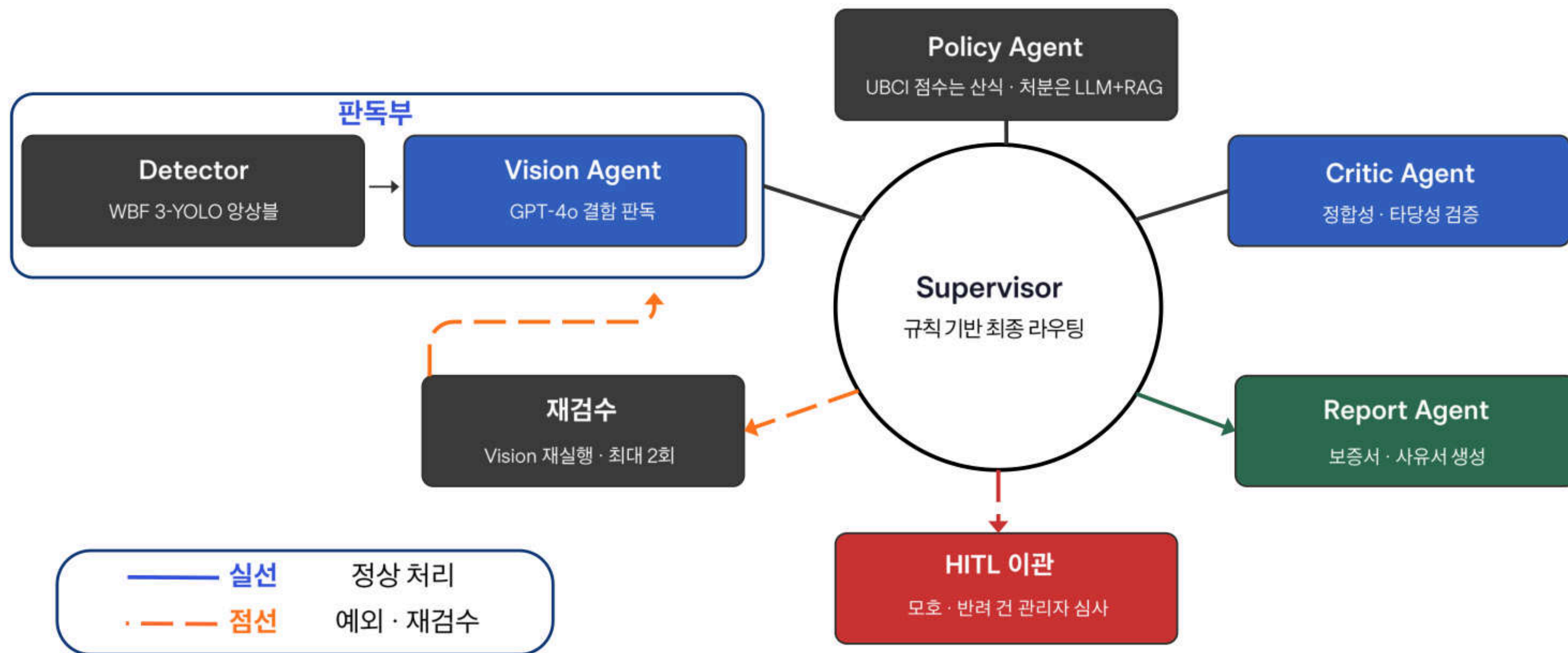
① 5면 전수 판독

겉과 속을 모두 보는 다시점 앙상블 검출

탐지 백본 YOLOv8 커스텀 3종 · 기여는 융합 전략과 판정 권한 분리에 있다



② LangGraph 멀티에이전트 오케스트레이션



AI 기반 판독과 결정론적 의사결정의 분리

LLM 호출 최소화 · **UBCI 산정 재현성** · 감사 추적성 · HITL 예외 통제

③ 비즈니스 AI - 동적 가격 · 3D Bin Packing

PICK -260813-0012

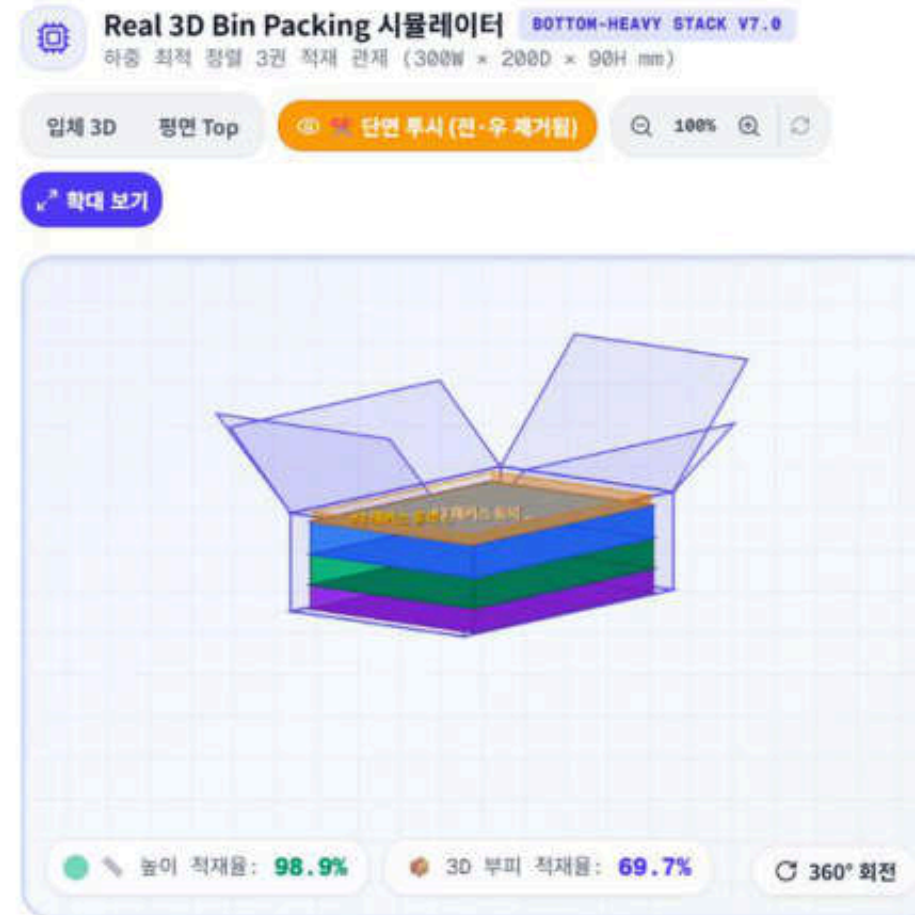
고객사 교보문고 B2B 지점
주문일 2026-08-13
배송유형 택배

No.	도서 정보	상태
1	도서 1	신품
2	도서 2	중고
3	도서 3	중고

판매가 계산

도서 정가 합계 66,900원
신품 10% 할인
중고 상태 · 보관일 반영

최종 출고가
37,440원



16종 박스 비교

도서 전용 슬림 박스 (8종)		일반 택배 표준 박스 (8종)	
도서슬림 소형 1호 무게조리 높이조리 크기조리 최대 250x150x50mm 2kg 수용률 (2.2kg 과적)	도서슬림 소형 2호 높이조리 크기조리 최대 250x150x50mm 3kg 수용률 (높이 89mm 대형필요)	도서슬림 중형 1호 높이조리 최대 300x200x70mm 4kg 수용률 (높이 89mm 대형필요)	도서슬림 중형 2호 수용 최대 300x200x90mm 5kg 적재율 63%
도서슬림 대형 1호 최대 350x250x100mm 7kg 적재율 78.5%	도서슬림 대형 2호 최대 350x250x140mm 8.5kg 적재율 72%	도서슬림 특대형 1호 최대 400x300x100mm 10kg 적재율 68.4%	도서슬림 특대형 2호 최대 400x300x200mm 12kg 적재율 61.2%

완충재 비교 및 하중 최적 정렬

스타트 도서 완충재 추천 (유격 & 슬림 정지)		완충재 비교 및 하중 최적 정렬	
3D 물 블록 코너 입	책이 들어 3D 입체	과다 에어 필러 거드	PE폼/책꽂이 4면 측면 채움
【적재용기】 (30.0%)	【적재용기】 (30.0%)	【적재용기】 (30.0%)	【적재용기】 (30.0%)
완충재 필링 용이	크리프트 종이 4면 채움	책꽂이 입장 25mm 채움	에어필로우 슬림채도
【적재용기】 (22.0%)	【적재용기】 (22.0%)	【적재용기】 (22.0%)	【적재용기】 (22.0%)
하중 최적 정렬 (Bottom-Heavy Stacking) 레이아웃 가이드 (3권) 적재용기/방재 30mm 배서			
완충 채움: 에어 필로우 슬림채도	#1 (책) 책꽂이	#2 (책) De-IT	#3 (책) 책꽂이
【적재용기】 (30.0%)	【적재용기】 (30.0%)	【적재용기】 (30.0%)	【적재용기】 (30.0%)
완충 채움: 에어 필로우 슬림채도	완충 채움: 에어 필로우 슬림채도	완충 채움: 에어 필로우 슬림채도	완충 채움: 에어 필로우 슬림채도
【적재용기】 (30.0%)	【적재용기】 (30.0%)	【적재용기】 (30.0%)	【적재용기】 (30.0%)



작업자는 무엇을 얼마에, 어떤 박스에, 어떤 순서로 담을지 한 화면에서 확정

④ AI가 재고 부족은 미리 제안, 수상한 거래 사전 탐지



AI는 먼저 찾고 제안하며, 최종 판단은 관리자가 맡습니다.

AI 검수 오류를 막는 3단 안전장치

검수 실패는 결함 없음이 아니다 - 감지·재검증·관리자 확인으로 오승인 차단



04. 아키텍처

시스템 전체 구성

책을 한 번 촬영하면 판독·등급·가격·적재·관제까지 하나의 시스템이 완성

한 번의 촬영

요청이 끊기지 않고
AI 판단까지 이동

모바일 접수 · API
Redis/Celery · Worker



결과가 바로 업무로

등급·가격·적재 위치와
운영 근거를 함께 반환

UBCI 산정 · 3D Bin Packing
PostgreSQL · S3 · 관제

1 촬영·접수

PWA 카메라 · 바코드 스캔
이미지 압축 · 업로드

2 비동기처리

FastAPI · Redis/Celery
검수 Worker

3 AI 판독·정책

YOLO/WBF · GPT-4o
UBCI Policy

4 저장·전달

PostgreSQL · S3
CDN/Presigned URL

5 재고·관제

3D Bin Packing
CloudWatch · Sentry

기술 선택 근거

01 서비스·확장

Kubernetes	여러 서버에 프로그램을 나눠 띄우고 관리하는 도구 검수가몰릴 때 AI 워커만골라서늘릴 수 있다
KEDA	밀린 작업 수를 보고 서버 대수를 조절하는 장치 CPU가 아니라 대기열 길이로 판단해야 실제 부하와 맞는다
Celery + Redis	오래 걸리는 일을 줄 세워 뒤에서 처리하는 구조 검수에 27초가 걸려도사용자는기다리지않는다
FastAPI	파이썬으로 만드는 업무 서버 AI 라이브러리와같은언어라모델을그대로붙일 수 있다
Next.js	화면을 만들고 서버에서 미리 그려주는 도구 한 주소로역할별 화면을 나누고, 통신이 끊겨도 작업이 저장된다

02 데이터·AI

PostgreSQL	표 형태로 자료를 저장하는 데이터베이스 등급·가격·재고가 서로 얽혀있어 일관성이 깨지면 안 된다
S3 + CloudFront	파일 보관소와 배달망 사진이 커서 서버가 직접 나르면 느려진다. 접근은서명으로 통제
LangGraph	AI 처리 단계를 그래프로 이어 붙이는 도구 판독 → 검증 → 보고로 나눠야 어디서 틀렸는지 추적된다
GPT-4o	사진을 보고 뜻을 읽어내는 AI 결함의 종류와 심각도는 규칙으로 정의할 수 없다
YOLO 앙상블	이미지에서 물체 위치를 찾는 모델 3개를 합침 손상 위치를 좌표로 정확히 잡아야 근거 사진을 만들 수 있다

이 시스템이 보장하는 4가지

멈추지 않고 · 밀리지 않고 · 새지 않고 · 되돌릴 수 있다

구성 상세 별첨 F-1 · 보안 상세 별첨 G부

① 작업이 사라지지 않습니다

처리 도중 서버가 꺼져도 검수 요청은 다시 처리됩니다

강제 종료 2회 → 4/4 완료 · 유실 0건

작업 재처리 · 실패 보관함

② 몰려도 밀리지 않습니다

요청이 쌓이면 처리 서버가 자동으로 늘어납니다

0 → 2대 자동 증감

대기열 기반 자동 확장

③ 외부에서 들어올 길이 없습니다

데이터베이스는 인터넷에 열려 있지 않습니다
검수 사진은 서명된 임시 주소로만 열립니다

공개 접근 차단 설정

내부 전용 · 서명 URL

④ 모든 판정에 근거가 남습니다

누가 언제 무엇을 보고 정했는지 전부 기록됩니다

판정 과정 100% 원장 보존

검수 원장 · 감사 로그

05. 성과 및 기대효과

사람의 손이 줄어드는 구조

사람이 고치는 부분 ➡ 다음 모델을 가르치는 교재

운영 DB 전수 129건 실측

74.4%

AI가 사람 없이 끝냄

14.0%

AI 이관

11.6%

관리자 재검수

※ 25.6%는 AI 판별실패가 아닙니다. 절반 가까이는 관리자가 결과를 보고 직접 재검수한 것입니다.

교정 데이터가 다시 모델로 돌아가는 학습 환류

1

재검수

- 관리자가 재고 화면에서 검수를 다시 부른다
- 결함 위치(BBox)를 손으로 직접 그린다

2

데이터 축적

- 관리자가 그린 좌표가 감사 기록에 남는다
- 사람이 확정한 값이므로 신뢰도가 높다

3

모델 재학습

- 쌓인 정답지로 탐지 모델을 재학습한다
- 외부 AI 판독 결과도 학습 라벨로 옮겨 담는다

4

관리자 업무 감소

- 잘못 짚는 경우가 줄면 Recall도 줄어든다
- 재검수 비율이 내려가고 자동 처리가 많아진다

이 구조가 성립하는 근거

- 되불러오기(RECALL)는 예외 처리가 아니라 설계된 경로다 — 감사 기록에 좌표까지 남도록 만들었다
- 외부 AI가 읽어낸 결과를 자체 모델 학습 라벨로 옮기는 오토라벨링 경로가 이미 동작한다
- 탐지 모델을 우리 데이터로 다시 학습시키는 것이 선행 과제임을 확인했다 (현행 모델은 오탐이 많다)

- 단, 재학습 후 성능은 아직 측정하지 않았다 — 개선 폭을 숫자로 약속하지 않는다
- AI 이관 14.0%는 자동 이관과 기록이 유실된 초반이 섞여 있을 수 있어 원인을 단정하지 않는다

도메인 확장성

불변요소

01	촬영 → 검수 → 등급 → 가격 → 적재	동일한 업무 흐름 적용
02	작업대기열 · 자동증설	처리량이 바뀌어도 동일 구조
03	장애복구 · 재처리	품목과 무관한 안전 장치
04	감사 기록 · 권한 분리	누가 무엇을 정했는지 남기는 방식은 동일
05	보안 5계층	네트워크부터 데이터까지 그대로

가변요소

- 01

감점 기준표

품목별로 결함·상태 판단 기준 상이
- 02

탐지 모델 학습 데이터

품목별 이미지 데이터로 모델 재학습
- 03

등급 규정

도메인별 등급·가격 산정

01 전자기기

외관 흠집 · 액정 상태
→ 중고 매입가 산정 구조가 동일

02 의류

오염 · 보풀 · 변색
→ 등급별 가격 정책이 이미 있는 시장

03 명품 · 잡화

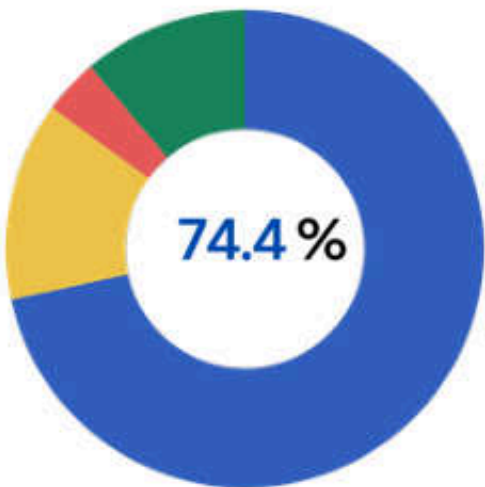
사용감이 가격을 좌우
→ 감사 추적 요구가 가장 강한 영역

성과 및 기대효과

기대효과

시연과 실제 운영 환경에서 확인한 성과입니다.

자동 검수 성과



- 검수 승인 74.4%
- HITL 14%
- 관리자 수동 검수 11.6%
- 반려 3.9%

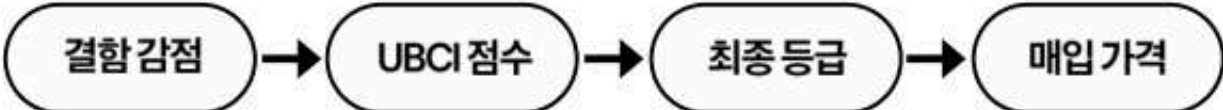
건당 검수 시간 27초

등급·가격 결정론 검증

동일 입력을 270회 반복해 산식의 재현성을 검증

270 / 270

등급·가격 결과 전부 동일



같은 입력 X 270회 = 동일한 등급·가격 오차 0건

LLM이 금액을 직접 계산하지 않고, 코드로 고정된 UBCI·등급·가격 산식이 최종값을 확정

처리량 자동 확장

0 → 2대

작업량이 늘면 처리 서버 자동 증설 요청 감소 시 자동 축소

상품 확장 가능성

657 종

657종의 도서 정보를 실제 서비스에 적용
가전·의류 등 다른 품목으로 확장 가능

안전한 데이터 관리

유출 0건

두 차례 장애 테스트에서도 정보 유출 없음
모든 판정 과정과 근거 기록

KT AIVLE SCHOOL 9기 2반 5조

감사합니다

경청해 주셔서 감사합니다.

NEXUS Project

THANK YOU



06. 별첨

별첨 구성

발표에서 다루지 못한 상세 내용을 부문별로 정리

01

설계

서비스 · 비즈니스 AI

A 서비스 상세

- 전체 서비스 플로우
- 역할별 주요 업무 · 주요 화면
- 검수 운영 방식 비교

B AI 파이프라인

- LangGraph 그래프 · 에이전트 해부
- UBCI 매트릭스 · YOLO 학습 계보
- RAG 지식베이스 · 프롬프트 원칙

C 비즈니스 엔진

- 4대 핵심 알고리즘
- 3D 적재 · 물리 규격 추정 · 로케이션
- 피킹 · 발주 · 동적 가격

02

운영

신뢰성 · UX · DevOps

D 백엔드 신뢰성

- 모듈 구조 · 인증 · SSE
- Celery · DLQ · Redis Lock
- 테스트 분포 · DB · API

E 프론트엔드

- URL 물리통합 · 카메라 UX
- PWA 오프라인 · 상태 관리
- 필수 기능 4종

F DevOps

- CI/CD · k8s · KEDA · CronJob
- CloudWatch 수집 설계
- Sentry 오류 추적

03

증거

보안 · 실험 · 개발 기록

G 보안 · 개인정보

- 5계층 방어 · 네트워크 실측
- 보안 헤더 · CloudFront 서명
- 감사 추적 · 미적용 구분

H 실험 · 연구

- 결정론 270회 검증
- 카오스 테스트 · 성능 비용 실측
- 연구 확장 가설 · 역량평가

I · J 보류 · 개발과정

- 만들었다 되돌린 것
- 검토했지만 쓰지 않은 것
- 오류 정정 기록 · 문서 체계

서비스 상세

서비스 상세

전체 서비스 플로우

입고부터 검수·등급·출고까지 하나의 흐름으로 관리합니다.



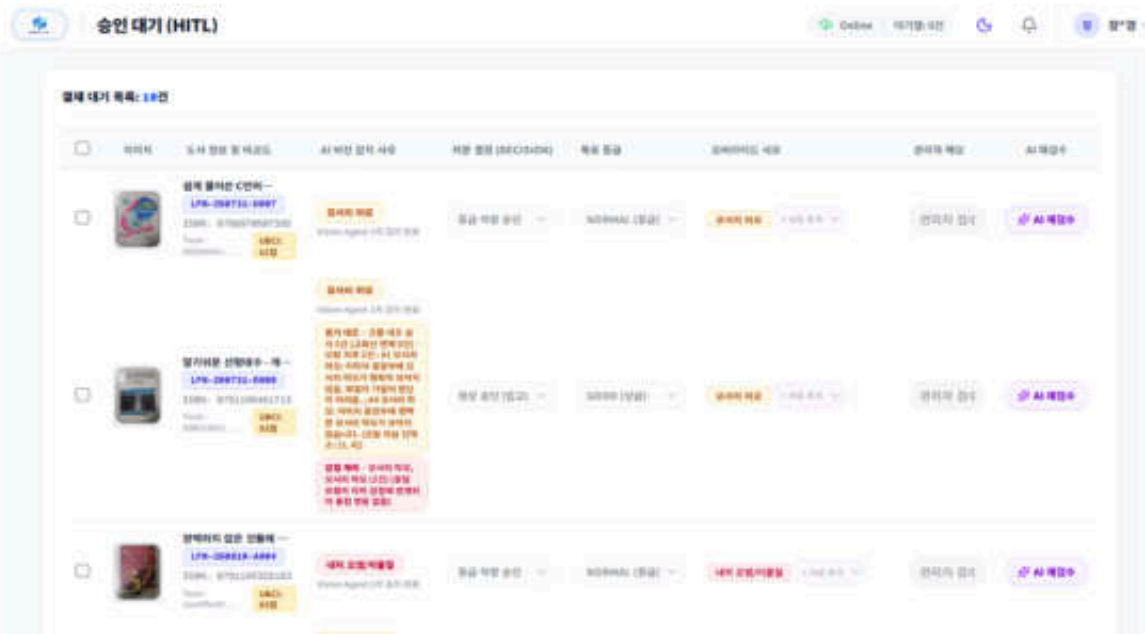
검수 결과는 등급·가격·적재 위치·출고까지 이어집니다.

주요 서비스 화면 (1/2) - 관리자

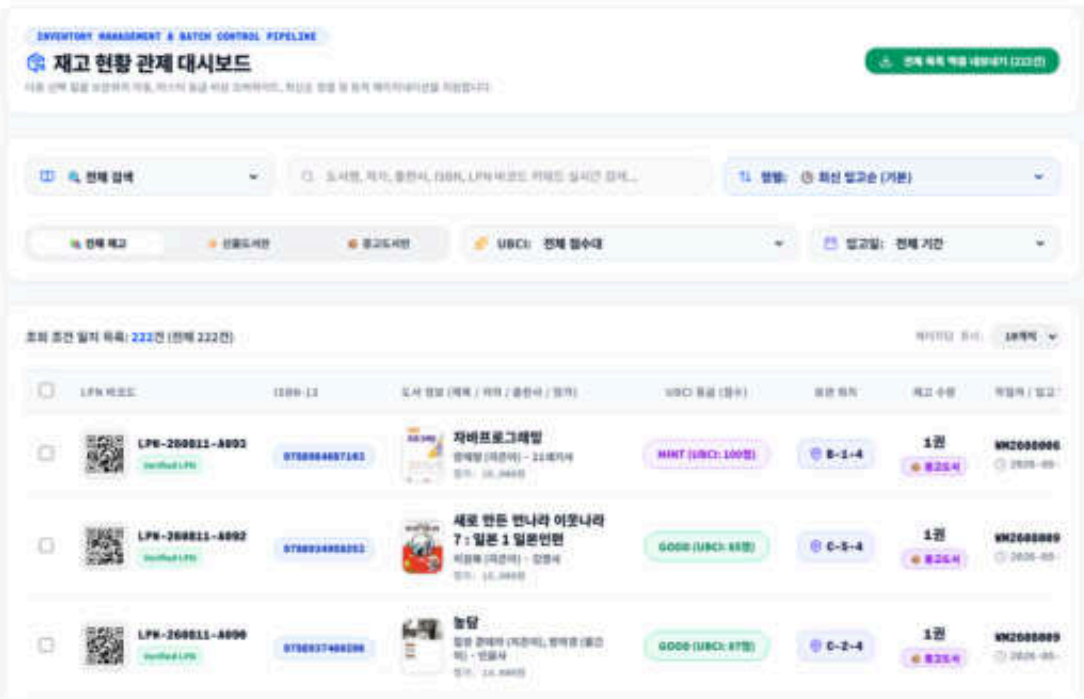
관제 대시보드



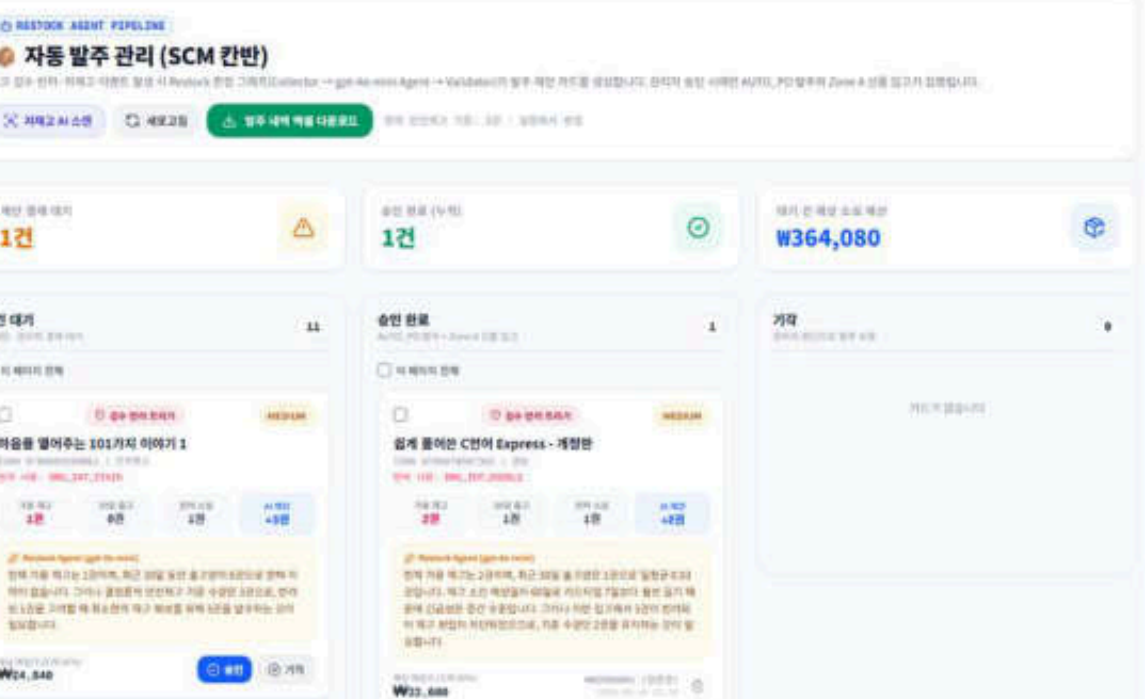
HITL 결재 화면



재고 현황



발주 제안 관리

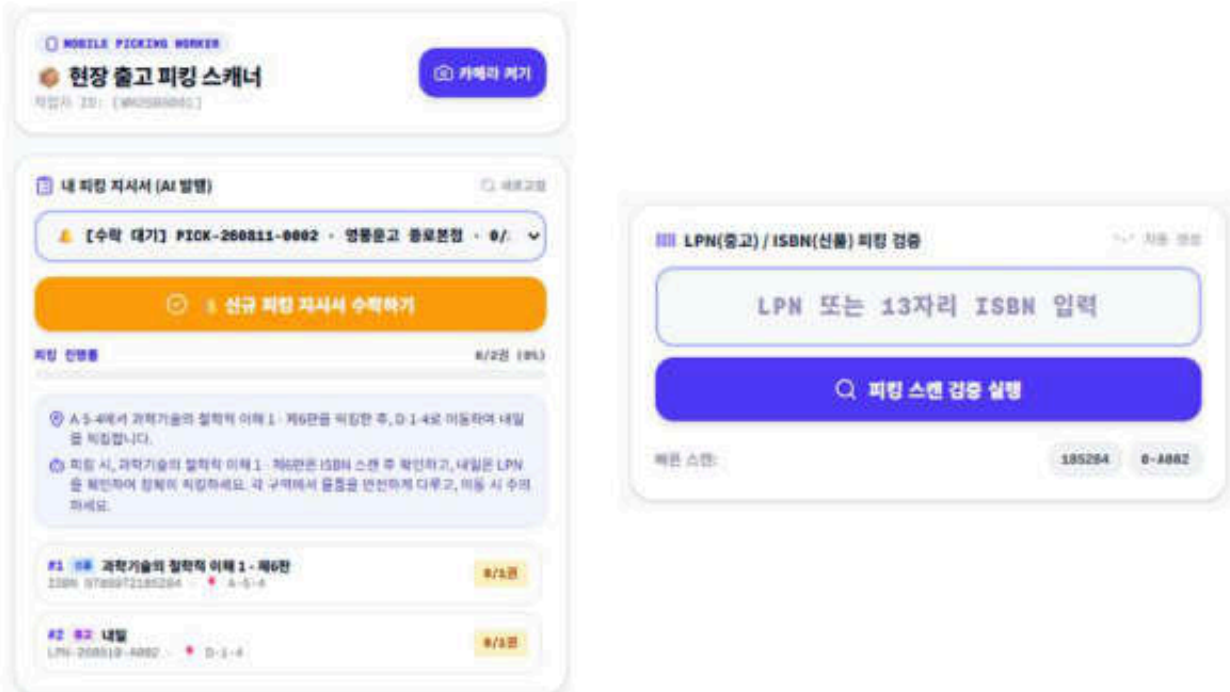


주요 서비스 화면 (2/2) - 작업자/소비자

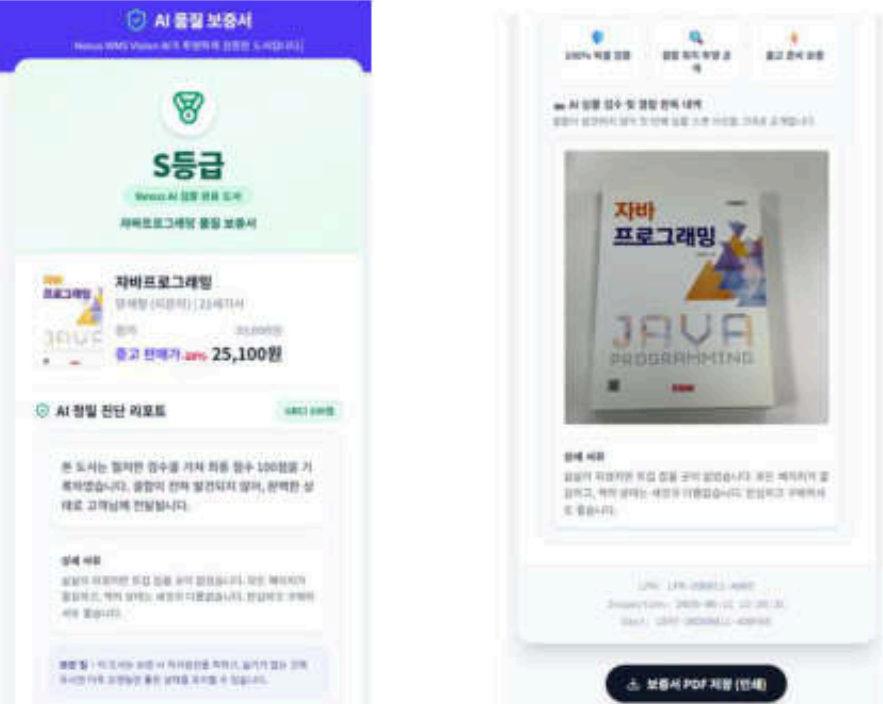
작업자 촬영 UI



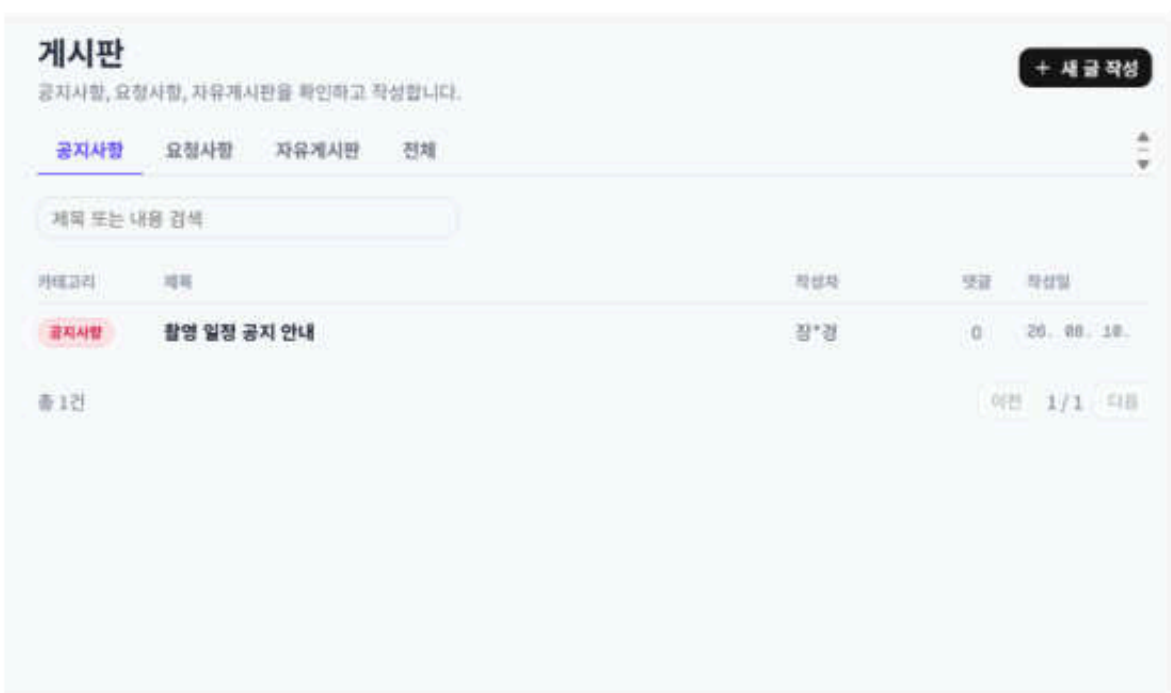
피킹 스캔 UI



QR 공개 보증서



게시판



검수 운영 방식 비교

수작업 검수

사람 경험에 의존



기준이 사람마다 다름



검수 후 수동 처리



기록 활용 어려움

규칙 기반 시스템

정해진 규칙으로 판단



예외 상황에 약함



부분 자동화



규칙 재작성 필요

Nexus

AI + 관리자 확인



동일 기준 적용



등급·가격 자동 연결



판단 이력 추적·재활용

검수에서 운영까지 연결

역할별 주요 업무

현장 작업자 

도서의 입고와 출고 업무를 처리합니다.

입고부터 출고까지 현장 작업 수행

촬영 · 라벨 부착 · 피킹

관리자 

AI 판정과 운영 현황을 관리합니다.

AI 판정과 운영 현황 관리

판정 확인 · 발주 승인 · 이상거래 확인

최고 관리자 

직원 계정과 접근 권한을 관리합니다.

조직과 권한 관리

계정 생성 · 역할 설정 · 권한 관리

고객 

구매한 도서의 품질 정보를 확인합니다.

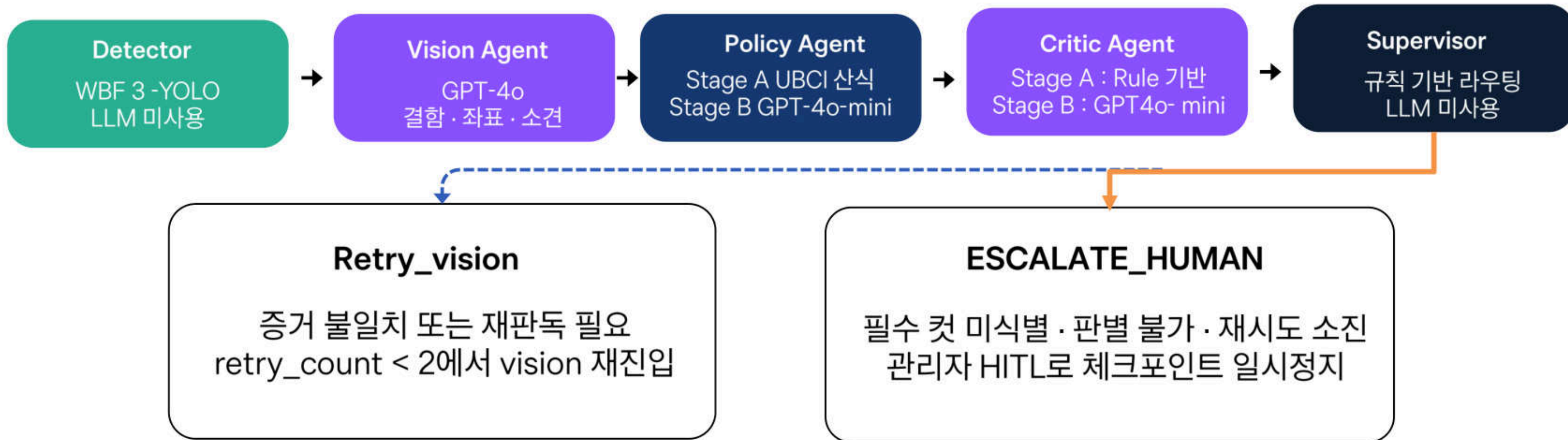
QR로 도서 품질 정보 확인

도서 정보 · 상태 등급 · 품질 보증서

AI 파이프라인 딥다이브

LangGraph 전체 그래프

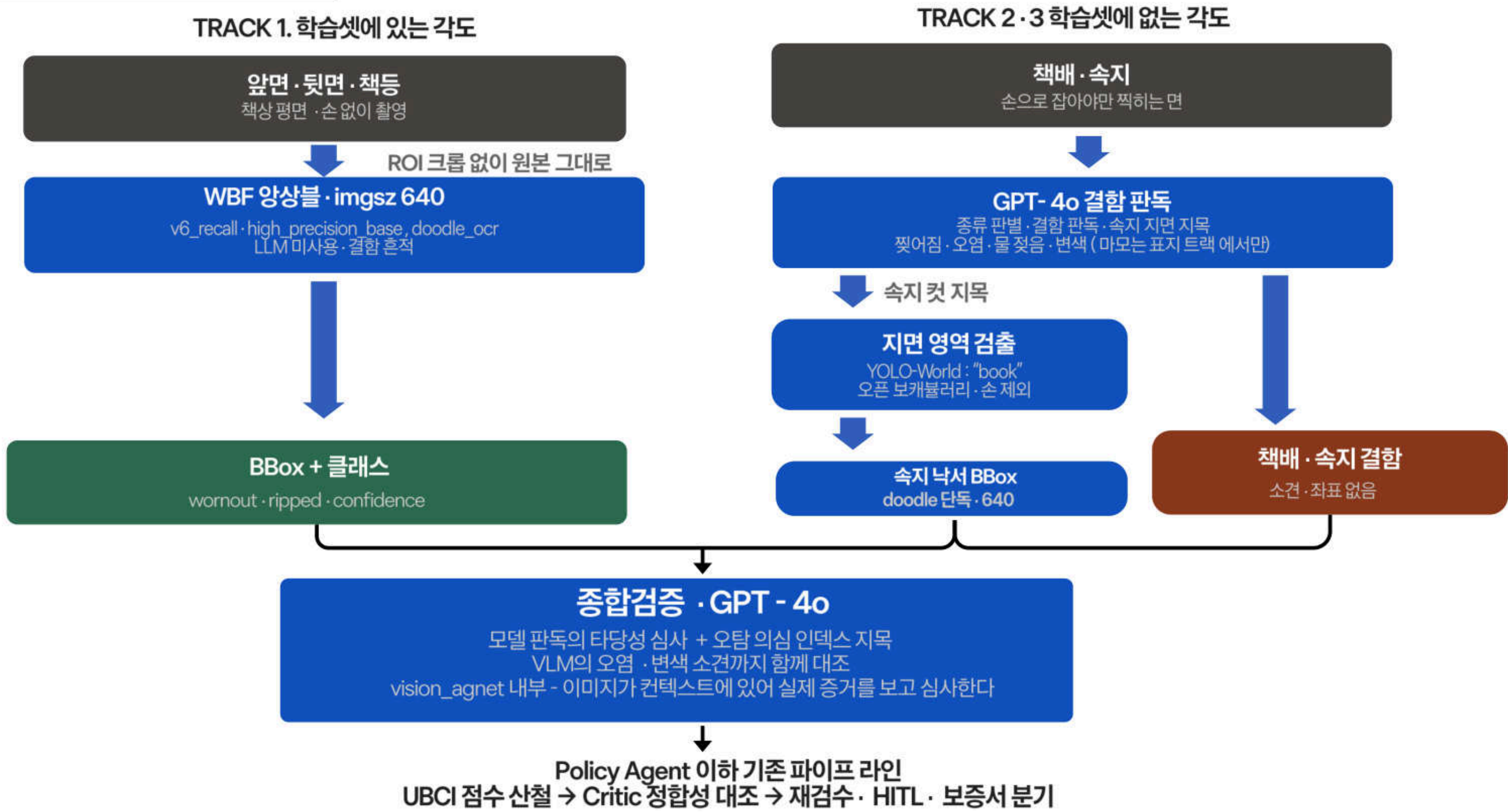
판정 체인 - 탐지 → 의미 판독 → 점수 → 검증 → 보증서



상태 · 감시 추적

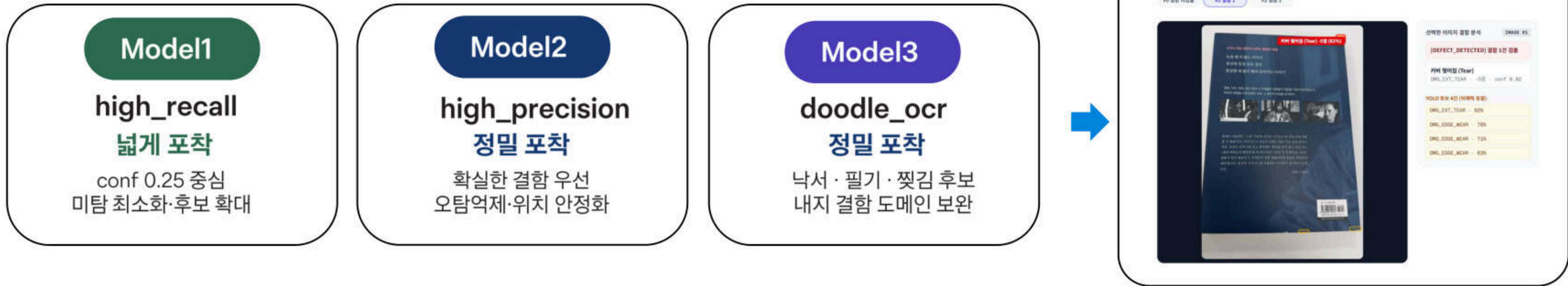
- MemorySaver 체크포인트: 재판독·HITL 재개 시, 이전 상태를 유지해 중복 API 호출 감소
- return_jobs.agent_logs(JSONB): Detector·Vision·Policy·Critic·Report 실행 요약과 근거를 원장에 보존
- RAG는 확정감점의정책근거인용과 HITL 브리핑에만 사용하며, 점수계산에는 개입 X

결함 판독 라우팅 - Track 별 분기



Detector 해부

WBF 3-YOLO 앙상블 (YOLOv8 커스텀 모델)



- WBF(Weighted Boxes Fusion)로 겹치는 예측박스를 좌표는 모델 가중평균, 신뢰도는 최댓값으로 융합
- 단순 NMS 대비 세 모델의 판단을 하나로 정확히 통합

□ 확정 결함,
□ YOLO 결함 탐지 후보

결정론적 처리
LLM 호출 없음

적용 범위
앞, 뒤, 책등

역할 경계
책 전체 ROI 탐지 아닌 결함 후보 생성

Vision Agent 해부

GPT-4o 결함 판독

탐지 박스만 믿지 않고 원본 이미지와 함께 비교 → 결함 종류 · 심각도 · 확신도를 구조화

입력



+



YOLO 후보 3건 (미채택 포함)

DMG_EDGE_WEAR · 81%

DMG_INT_DOODLE · 77%

DMG_EDGE_WEAR · 49%



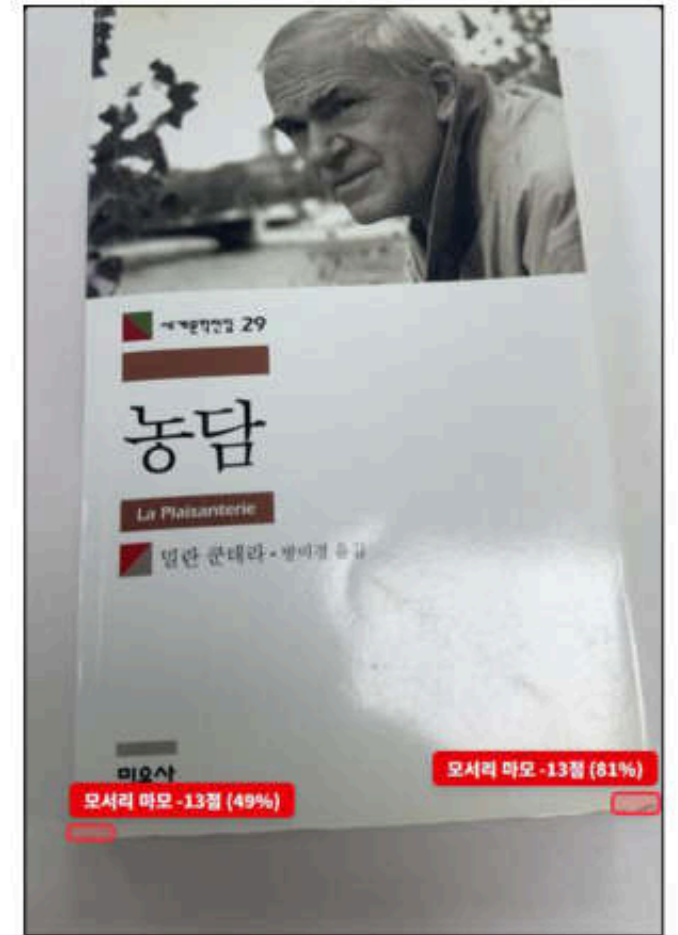
GPT-4o

Vision Agent

- 증거 대조 검증: BBox 영역 건별 재심사, 확신도 무관 전건 심사
- UNCLEAR(애매) 판정은 감점을 유지한 채 다음 단계로 전달



구조화 판독 결과



[DEFECT_DETECTED] 결함 2건 검출

1. 모서리 마모 DMG_EDGE_WEAR · -13점 · conf 0.81
2. 모서리 마모 DMG_EDGE_WEAR · -13점 · conf 0.49

원본이미지

책 전체 문맥 · 촬영면
· 주변 상태

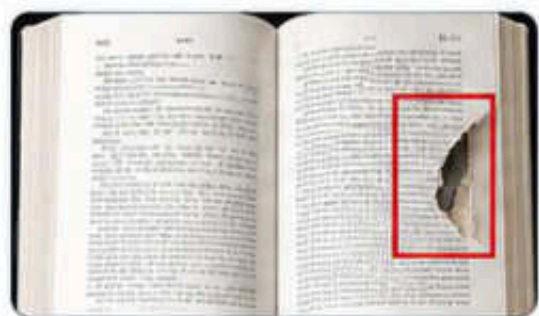
후보 영역

BBOX · 결함 후보 · 신
뢰도

Policy Agent 해부

UBCI 결정론 산식

구조화 결함 근거(예시)



Defect Code, 면적 · 심각도
, image_index, **bbox**, ...

감점 매트릭스

DOODLE	-10 / -15 · Cap -15
TEAR	-5 / -10 / -15 · Cap -25
CRUSH	-3 / -5 / -10
STAIN	-5 / -10 / -15
EDGE_WEAR	-3 / -5 / -10 · Cap -15
WET	즉시 REJECT · UBCI 0

가중치 · 중복 방지

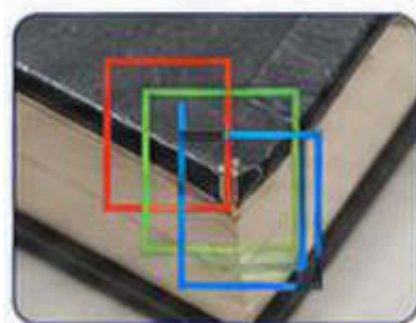
$$UBCI = \text{clamp}(100 - \sum \text{조정 감점}, 0, 100)$$

본문 침범 : 감점 x 1.5

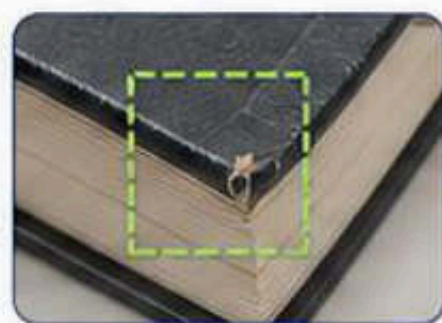
찢어짐 : 이미지 · 페이지별 누적, 총 Cap -25

모서리 마모 : 기본감점 + (서로 다른 모서리 수 -1) x 2, Cap -15

중복 결함 (동일 물리 위치) 병합 예시



여러 카메라 박스
(동일 물리 위치)

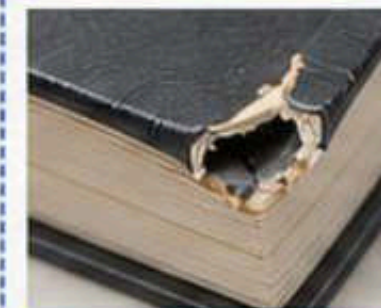


정규화된 책 모서리 위치 1건
(표준 좌표로 병합)

등급 · 기본 처분

MINT (S)	95 ~ 100	자동 입고 · 최고가
GOOD (A)	85 ~ 94	입고 승인
NORMAL (B)	65 ~ 84	감가 입고
REJECT (C)	0 ~ 64	반송 · 폐기

검증 예시



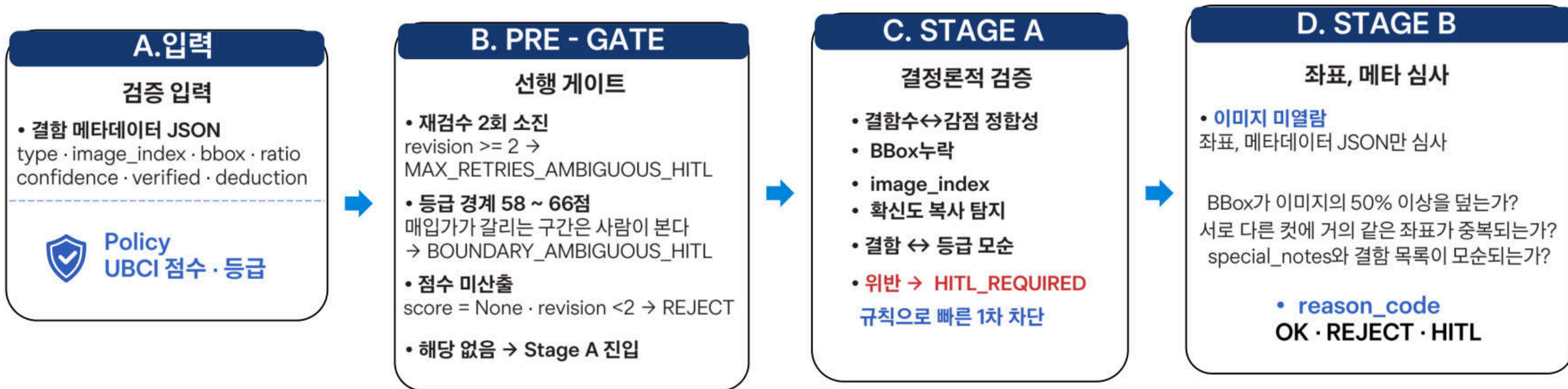
DMG_EXT_TEAR

면적 15% → Severe -15

UBCI 85점 · GOOD(A)

Critic Agent 해부

2단 교차검증 (Stage A 결정론 + Stage B LLM)



- ▶ Stage A (결정론, LLM 미사용): 결합수 ↔ 감점정합성 · BBox누락 · image_index 범위 · 확신도 복사 탐지 — 위반 시 즉시 HITL
 - ▶ Stage B (GPT-4o-mini): 결합 1건 이상일 때만 판독 타당성 심사 — MINT 물량추가비용 0
 - ▶ 부가검증은 fail-open: LLM 장애 시 Stage A 결과만으로 진행 (판정마비방지)

Supervisor 해부

지휘 판단 + 판정 커버리지 게이트

0. 검증 결과 입력

- Vision : 판독 결과
- Policy : UBCI 점수 · 등급
- Critic : reason_code

검증 결과 다시 판독 X,
경로 판단에만 사용

1. 판정 커버리지 게이트

 유효하게 판독된
촬영 컷이 0장인가?

YES → NO_VALID_IMAGE_HITL (사람 판단)

>>>> NO → 다음 판단

 안 본 것 ≠ 봐서 결함 없음
점수 null · MINT 자동 확정 차단

2. 판단 규칙 (LLM 호출 0회)

- ✓ OK → 정상발행
 - ! REJECT → Vision 재검수
 - 👤 HITL → 관리자 확인
 - ✓ Vision 실패
 - ✓ 정합성 위반
 - ✓ 등급 경계 · 판독불가
- Critic의 문제 보고와
시스템 상태를 결정론적으로 해석

3. 최종 경로 선택

검증 통과 → Report 발행
증거 불충분 → AI 재검수
불확실 · 위반 → HITL 관리 판정

지휘 책임을 Supervisor 한 곳에 집중

역할 경계



점수 · 등급 재계산 안 함
결함을 다시 판독하지 않음
검증 결과에 따라 경로만 결정

안정성과 감사추적



- ✓ 판독 실패를 정상 결과로 해석하지 않음
- ✓ reason_code, 선택 경로, 재시도 횟수 저장
- ✓ HITL은 판독불가, 경계 사례만 이관

Report Agent 해부

GPT-4o-mini 서술 생성, 확정된 판정을 사람이 읽는 보증서로 변환

Policy·Critic·Superviosr 검증 통과 결과



UBCI 점수·최종 등급

확정 결함 목록·증거 이미지



입고·감가·반송 처분

1 사실 잠금



- ✓ 점수 변경 금지
- ✓ 등급 변경 금지
- ✓ 결함 추가·삭제 금지

READ ONLY

2 문장으로 변환



- ✓ 점수 변경 금지
- ✓ 등급 변경 금지
- ✓ 결함 추가·삭제 금지

READ ONLY

3 AI 검수 보증서



A등급

Nexus AI 검증 완료 도서

본 도서 품질 보증서



동생을 바꾸고 싶어

실벤느 자우이 (지은이), 홍자희 (그림), 이선미 (옮긴이) | 크레용하우스

정가

중고 판매가 -48% 4,200원

AI 정밀 진단 리포트

UBCI 90점

본 도서는 UBCI 최종 점수 90점으로 A급(중음)으로 평가되었습니다. 다만, 내부에 손글씨가 남아 있는 점은 참고하시기 바랍니다.

생제 사유

책물 앞표지 뒷표지 뒷표지, 결함 없이 있다는 사실을 찾기 어려웠습니다. 다만, 내부에 손글씨 이력이 남아 있는 점은 양해 부탁드립니다.

100% 픽셀 검증

결함 위치 투명 공개

출고 준비 보증

AI 식물 검수 및 결함 판독 내역

감정 사유가 가결된 식물 스펙 사진과 판독 위치를 그대로 공개합니다.



내부 손글씨/낙서 내부 표지

책의 내부에 손글씨 이력이 손글씨로 남아 있습니다.

UPN: 179-000011-0000

Registration: 2020-08-13 09:30:30

City: 12107-20200813-751012

보증서 PDF 저장 (인쇄)

Policy 2단 상세

STAGE A

UBCI 점수 등급 산정

결정론적 매트릭스 (LLM 없음) -입력 결함 유형, 심각도, 영역

100점 - 결함별 고정 감점 = UBCI

점수 구간에 따라 등급을 고정 매핑

MINT	GOOD	NORMAL	REJECT
≥ 95	≥ 85	≥ 65	< 65

출력 ubci_score • suggested_grade

같은 입력이면 같은 점수 • 정책 조항 라벨을 함께 기록



점수 변경 불가

STAGE B

거래 처분 규정

GPT-4o-mini + RAG

-입력 반품 • 배송 • 취소 조건

근거 조항을 바탕으로 거래 처분만 해석

반품 수용 ACCEPT / REJECT	배송비 부담 부담 주체	귀책 후보 책임 판단
--------------------------	-----------------	----------------

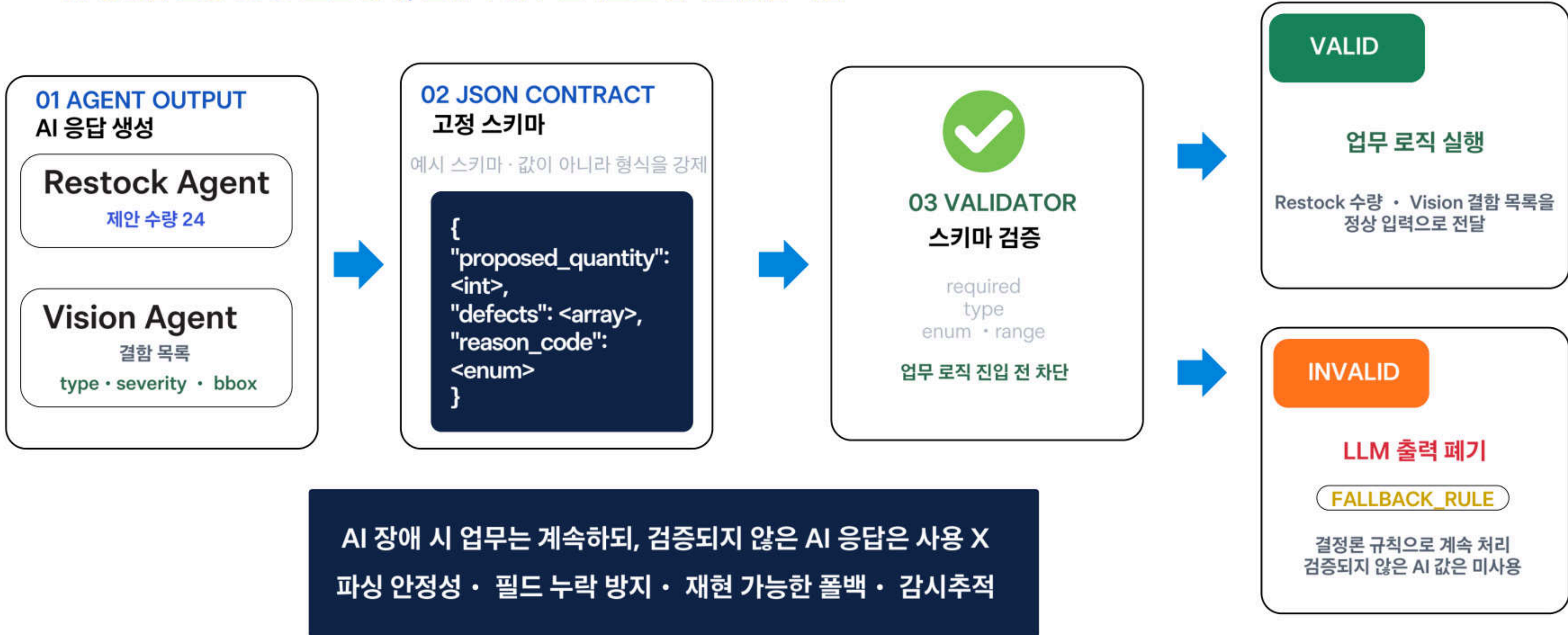
판정 근거 Internal Statute | 외부 플랫폼
약관은 참고만

출력 return_policy

근거 없는 결론은 무효화 한 후 HITL

구조화 출력 전략

AI 응답을 고정 JSON으로 통제, 오류가 업무 로직으로 번지는 것을 차단



모델 배정 근거표

정확도가 중요한 판단과 반복 업무를 분리해 모델 배정



YOLO v8 커스텀 3종 + WBF

저지연 로컬 추론 · 도서 결함 데이터 특화
세 모델의 검출 후보를 신뢰도 가중 평균으로 통합



GPT - 4o

복잡한 시각 추론 · 결함 종류와 심각도 서술
원본 이미지와 결함 영역을 함께 대조



GPT - 4o-mini

Critic Stage B · Report · Restock
정형 업무는 저비용 모델, 수량은 Validator 최종 클램프

YOLO 학습 - 데이터 수집



증강 · 합성 데이터



결함 패턴 합성

정상 ISBN 표지 + 결함 마스크
알파블렌딩(alpha-blending)으로 위치 · 강도 조절

$$I_{\text{syn}} = \alpha D + (1 - \alpha)I$$



촬영 조건 증강

회전 조명 편차 블러
각도와 밝기가 달라도 같은 결함을 학습

현장 촬영 편차 대응



클래스 불균형 보완

희귀 결함 표본을 선택적으로 확장
부족한 클래스의 학습 사례 보완

희귀 결함 데이터 공백 취소

합성데이터는 실데이터를 대체하지 않고, 희귀 결함의 빈칸을 보완

프롬프트 엔지니어링 전략



01 판정 기준 고정

few-shot 예시로 명시 + RAG 근거

결함 유형과 심각도 기준을 판독 컨텍스트에 명시

02 반증 질문

Critic Stage B의 독립 재질의

'처음 판단이 틀렸다면?' 을 묻고 확증 편향을 억제

03 근거와 계산 분리

MD는 문맥 보존 · YAML/Parser는 규칙 연산

AI는 문서를 읽고 점수 · 등급은 결정론 코드가 계산

결정론적(Deterministic) 감점 매트릭스 기반 판정 일관성 확보

"동일결합 = 동일감점"을 UBCI 산식으로코드레벨에서강제 — 확률적출력의점수변동성차단

판정 원리

100

점에서 시작

100 - 감점 합계

유형별 상한(Cap) 적용

예외

젖음·습기·힘
→ 즉시반려 · 0점

결합 코드	결합 내용	감점	상한·특칙
DMG_INT_DOODLE	내지 낙서·필기	Minor -10 / Severe -15	내지 낙서·필기
DMG_EXT_TEAR	커버·내지 찢어짐	Minor -5 / Mod -10 / Sev -15	커버·내지 찢어짐
DMG_EXT_CRUSH	표지모서리찍힘	Minor -3 / Mod -5 / Sev -10	표지모서리찍힘
DMG_INT_STAIN	내지오염·이물질	Minor -5 / Mod -10 / Sev -15	내지오염·이물질
DMG_EDGE_WEAR	모서리 마모	Minor -3 / Mod -5 / Sev -10	모서리 마모
DMG_EXT_WET	액체오염·습기·힘	즉시반려 · UBCI 0점	치명적 결합 — 다른 감점과 합산하지 않음

판정편차를 막는 세 가지 보정규칙

01 왜 상한(Cap)을 두었는가

- 같은 유형이 여러 장에 걸치면
- 감점이 무한히 쌓여 전부 반려가 됨
- 유형별 최대 감점을 잘라 과잉 반려 방지

02 본문 침범 가중치 ×1.5

- 같은 유형이 여러 장에 걸치면
- 감점이 무한히 쌓여 전부 반려가 됨
- 유형별 최대 감점을 잘라 과잉 반려 방지

03 모서리 마모의 특수 규칙

- 한 모서리를 여러 각도로 찍으면
- 건수로 세면 중복 감점됨
- 기본감점 + (모서리 수 - 1) × 2점

YOLOv8 커스텀 모델 파인튜닝(Fine-Tuning) 및 최적화 이력

실패 런과 하이퍼 파라미터 튜닝을 포함한 반복학습으로도 메인 특화 Precision 개선

01 성능을 높이는 방법을 검증

모델 크기·해상도·전이학습 방식을 바꾸며 실패 원인을 분리

01	Medium (Base)	채택
	YOLOv8m 기본 학습	채택 · mAP50 0.599, 이후모든비교의기준선
02	Large	폐기
	더 큰 모델로 성능 확보 시도	폐기 · 데이터량 대비과대적합으로 성능저하
03	Finetune v3	폐기
	해상도 800으로 업스케일	폐기 · 학습 해상도와 추론 해상도 불일치
04	Finetune v4	폐기
	전이 학습 재시도	폐기 · 파국적 망각 발생
05	Finetune v5	성공
	freeze + 저학습률로 망각 방어	성공 · 이후 파인튜닝의 표준 레시피가 됨

02 역할별 모델을 확정

재현율·낙서·현장 배경에 맞춰 학습 목적을 분리

06	train_unified_v6	폐기
	cls 가중치 0.5	폐기 · 33epoch 중단, 서버에 쓰인 적 없음
07	train_unified_v6_recall	채택
	cls 가중치 1.5로 재현율 강화	채택 · 300epoch 완주, 현행 Model 1
08	doodle_ocr	채택
	AIHub 손글씨 1만 장 전담 학습	채택 · 내지낙서도메인전용 Model 3
09	Finetune v4	폐기
	실촬영 150장으로 도메인 적응	1차 적용완료 · 배경오탐대응

실험로그가 남긴 결론
기준선확립 · Model 1 재현율강화 · Model 3 내지낙서전담 · v8 배경오탐대응

앙상블 모델 평가 지표(Metrics) 및 기준선(Baseline) 확립

검증 결과

0.599

mAP50

0.885Precision
오탐 비율 최소화**0.560**

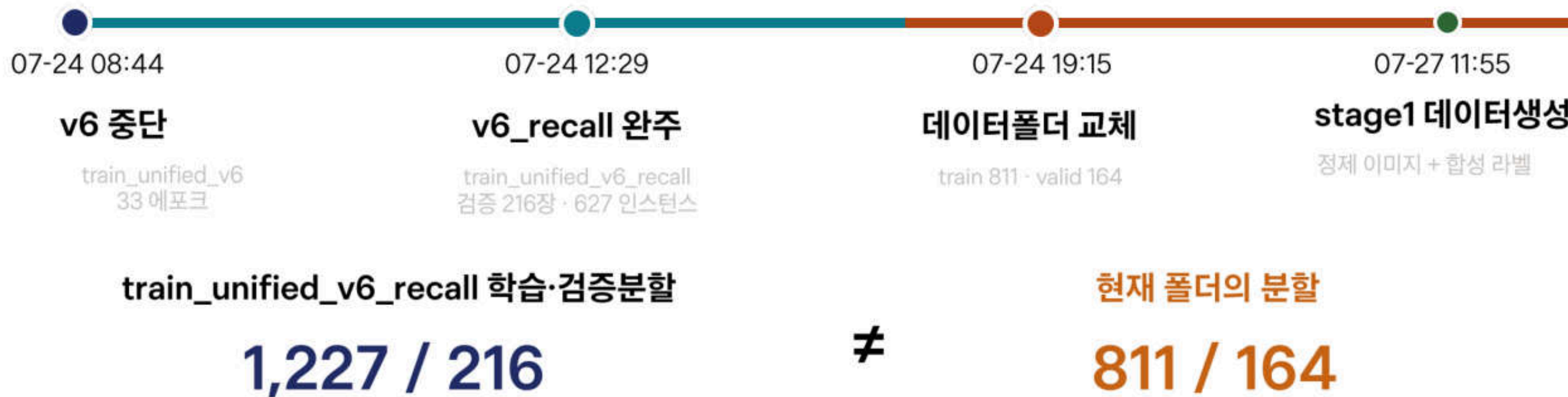
Recall @conf 0.25

0.448

Recall @conf 0.25

검증셋이 바뀐 시점

같은 지표라도 시험지가 달라지면 성능 변화로 해석할 수 없습니다



미탐 최소화를 위한 재현율 우선 5대 전략

오탐보다 비싼 미탐을 줄이기 위한 데이터·학습·추론·운영의 5중 방어

두 오류의 비용은 같지 않습니다

고객에게 넘어간 미탐은 되돌리기 어렵고,
오탐은 검수 단계에서 바로 지울 수 있습니다

미탐 · 놓친결함

사업비용 큼

결함
놓침MINT
입고상태
불일치반품·신뢰
손실

미탐 · 놓친결함

관리비용작음

후보 1건
증가관리자
확인즉시
삭제

설계방향

정밀도일부양보 · 재현율우선

1

데이터융합

v9i 939장 + v3i 620장 병합 = 통합 1,559장 데이터셋

절대 데이터량 확보를 먼저 진행

2

손실 가중치 조정

cls가중치 0.5 → 1.5 · 분류손실에 더 큰 벌점

경계가 애매한 결함까지 잡도록 유도

3

증강 극대화

mosaic 1.0 · mixup 0.15 · scale 0.5 · 촬영각도·조명편차대응

AdamW · lr0 0.001 · 300 에포크

4

가변 추론 임계값

학습평가는 conf 0.25 · 서빙은 conf 0.12~0.15로 낮춤

후보를 넓게 잡고 뒤에서 다시 걸러냄

5

HITL 이중 방어선

모델이 못 잡은것은사람이보완 · 수정좌표는재학습재료

모델 성능을 운영으로 보완

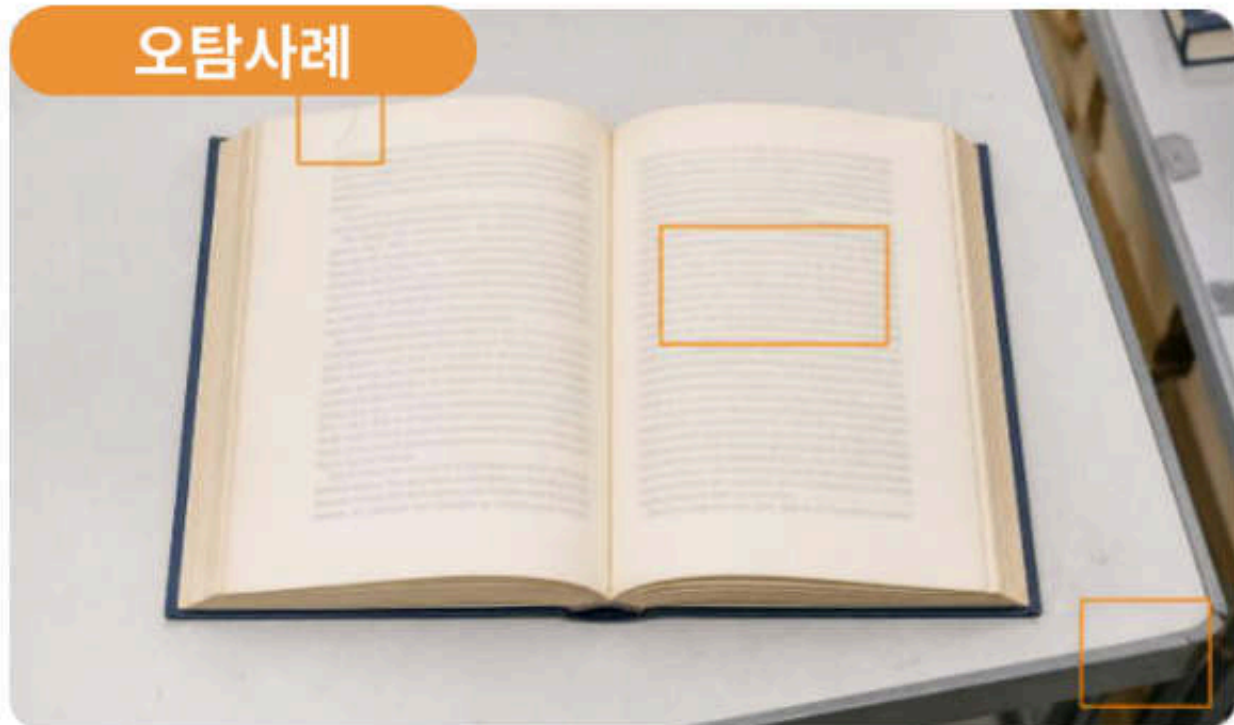
낮춘임계값이만든오탐

후보확대 → WBF 융합 + Critic 검증 → 최종오탐제거

실도메인이미지(In-Domain Data) 파인튜닝 및 오탐제어

오탐보다 비싼 미탐을 줄이기 위한 데이터·학습·추론·운영의 5중 방어

오탐사례



Negative 89장 넣은 이유

손·책상·머리카락·인쇄 본문은 결함이 아닙니다.
빈 라벨 사진으로 '여기엔 아무것도 없다'를 학습시켜
모든 장면에서 무언가를 찾으려는 오탐을 억제했습니다.

평가원칙

전체mAP만보지않고
동일검증셋에서 v6_recall과 직접 A/B 대조

현장 데이터로 학습 환경 세팅

실촬영 150장 육안 분류 · 포지티브 71 / 네거티브 89

Positive - 71장

Negative - 89장

결함 위치 학습

결함이 아닌 장면 학습

방어적 재학습

lr0 0.0001 · freeze 10 · v5에서 얻은 설정계승

학습결과

29 에포크EarlyStopping · Best Epoch 9 · 0.293시간

동일 검증 셋 성능

0.803

Precision - 오탐 비율 최소화

0.455

Recall @conf 0.25

0.490

mAP50

0.333

mAP50-95

클래스	Images	Instances	P	R	mAP50	mAP50-95
Wornout	137	434	0.713	0.383	0.410	0.263
ripped	58	93	0.893	0.527	0.569	0.403
ALL	188	527	0.803	0.455	0.490	0.333

HITL 기반 오류 정정 및 능동 학습(Active Learning) 파이프라인

관리자 BBox수정분을 감사 원장에 축적해 오토라벨링·재학습 정답지로 환류

검수 결과가 학습셋이 되는 경로

초안 생성부터 정답 좌표 추출까지, 판단 근거가 단계별로 남습니다

- 1 YOLOv8 사전라벨링**
현행 모델이 먼저 결함 후보 박스를 생성
- 2 labellmg검수**
사람은 박스를 삭제·이동해 오탐과 좌표를 교정
- 3 VLM 지식증류 + 좌표저장**
GPT-4o 판단을 YOLO 라벨로 변환 · `admin_audit_logs.defect_coordinates`
- 4 검증 라벨 추출**
`hitl_excluded` 오탐 제거 · `GET /research/export-dataset` · YOLO 포맷 출력



사람은 처음부터 박스를 그리지 않습니다
모델 초안에서 잘못된 박스를 지우거나 옮기기만 합니다

관리자 1회 수정

감사로그 + 검증좌표 + 재학습라벨

오탐은 제외하고, 확정·수정된 박스만 학습 데이터가 됩니다

자동화 경계 수정좌표저장 → 오탐제외 → 학습셋추출

`admin_audit_logs.defect_coordinates`에 좌표를 저장하고
`hitl_excluded` 오탐을 제외합니다.
API 한 번 호출로 YOLO 학습셋을 추출합니다.

재학습 · 배포

추출 데이터로 YOLO를 다시 학습하고
새 가중치를 배포합니다.

클래스 불균형(Class Imbalance) 극복 — 합성 데이터(Synthetic Data) 증강

알파 블렌딩으로 희귀 결함을 합성하고 Augmentation을 병행해 엣지 케이스 커버

실제 표지 → 합성 결함 샘플



희귀 클래스의 빈칸을 합성으로 채우고
현장 편차까지 학습사례로 확장

01

ISBN으로 원본 확보

실제 도서 표지 이미지를 ISBN 기준으로 수집

02

결함 패턴 합성

알파 블렌딩으로 찢어짐·오염을 정상 표지 위에 합성

03

증강 적용

회전·조명·블러로 실제 촬영 환경의 편차 반영

04

스토리지 이원화

S3 연동 모드 / 로컬 테스트 모드를 환경 변수로 전환

왜 합성이 필요했나

- 물에 젖거나 심하게 찢어진 책은 중고 유통에서 수집이 어려움
- 희귀 클래스 불균형이 학습을 방해

완전 생성이 아닌 이유

- 실제 표지를 참조로 사용
- Reference-based Synthetic
- 배경·조판의 현실성 유지

한계와 사용 원칙

합성 질감은 실제 결함과 다를 수 있고 합성만으로 학습하면 현장에서 밀릴 수 있으므로, 실촬영 데이터와 반드시 섞어 사용

RAG 기반 정책 Grounding — 판정 근거의 조항 단위 인용

알파 블렌딩으로 희귀 결함을 합성하고 Augmentation을 병행해 엣지 케이스 커버

A 감점 근거 인용

- 확정 감점에 맞는 조항 검색
- 점수 적용과 보류를 구분

`cite_deduction_basis`



정책문서 · 검색조항 · 인용근거

B 거래 처분 판단

- Policy Stage B 이후 실행
- 반품·수용·배상 후보

`evaluate_return_policy`

C HITL 브리핑

- 관리자 화면에 근거 요약
- AI 판단과 관리자 결정 분리

`build_hitl_briefing`

- 유사도** 코사인(hnsw:space=cosine)
문서 길이가 달라도 방향만 비교해 긴 조항의 편향을 방지
- 임베딩** text-embedding-3-small
한국어 정책 검색 품질과 호출 비용의 균형
- 벡터 DB** ChromaDB클라이언트-서버
워커마다 인덱스가 복제되는 문제를 서버 모드로 일원화
- 배포** k8s Deployment + EBS 2Gi
기존 kubectl 체계에 편입하고 노드 재부팅 시 자동 복구
- LangChain** 필요한모듈만사용
전체 체인의 불투명성을 피하고 내부 동작을 설명 가능하게 유지
- 규정집** ConfigMap주입
이미지 재빌드 없이 규정 개정 내용을 반영

평균 정책 조항 검색

44.3 ms

110개 조각 · 질의 10종 × 20회 = 200회
2회 반복: 44.34ms / 44.30ms

조항→질문임베딩증강안 250-280ms · 검색과분리유지

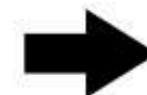
RAG는 점수 계산에 개입 X

검색된 조항은 확정된 감점의 인용 근거로만 사용 — 계산과 인용의 분리

UBCI 결정론
점수·감점확정



확정된판단
감점·처분유지



RAG 조항인용
근거만연결

벡터 DB 장애 → 근거인용생략 → 점수와판정은그대로진행 (fail-open)

LLM Hallucination 억제 — 프롬프트 설계 5원칙과 코드 강제

스키마 강제 · 판단 주체 단일화 등 5원칙 — 프롬프트로 지시한 것은 코드로도 차단

형식은 스키마로 강제

- 'JSON으로 답해줘'라고 쓰지 않음
- 구조화 출력으로 형식 자체를 고정
- 파싱 오류·필드 누락 차단

금지는 코드로도 막는다

- '점수를 바꾸지 마'라고 지시
- 동시에 코드에서 점수 필드를 읽기 전용 처리
- 프롬프트만으로 막지 않음



대상을 좁히고 확대해서

- 이미지 전체가 아닌 결함 후보 영역만 전달
- ROI를 잘라 크게 보여줌
- 판단 범위를 좁혀 정확도 확보

모호함을 '결함없음'으로 읽지 않음

- UNCLEAR는 '결함 없음'이 아닌 별도 상태
- 감점을 유지한 채 다음 단계로 전달
- 안 본 것과 봐서 없는 것을 구분

판단 주체는 유형당 하나

같은 판단을 두 에이전트에 맡기지 않습니다. 결함 판독은 Vision Agent만, 점수와 처분은 Policy Agent만 담당합니다.

프롬프트튜닝에서 기각한 접근



CSV로 지식베이스를 잘게 분해
문맥이 잘려 환각이 급증해 폐기

대체 | MD 문맥보존 + YAML 규칙연산



'확신이 없으면 답하지마'
모호한 건을 전부 침묵 처리해 미탐 증가

대체 | UNCLEAR를 명시해 후속 단계로 전달

E2E 통합검증 — 컨테이너실환경 전 구간가동확인

단위 테스트가 못 잡은 WBF 양상블 비활성(의존성 누락)을 통합 검증이 검출 — SSE · DLQ · 원장 기록 확인

한 번의 요청을 끝까지 추적

실환경에서 확인한 검증결과

실행증거 · 장애경로 · 저장원장



WBF 양상블 실가동

컨테이너 내 비활성 상태를 발견하고 의존성 추가·재빌드 후 실제 가동을 확인



SSE 스트림 순서확인

요청 접수부터 판정 완료까지 상태 이벤트가 순서대로 도달



DLQ · Redlock안전장치

중복 처리 차단 동작 확인 · DLQ 적재 0건



agent_logs원장저장

Detector · Vision · Policy · Critic · Report 실행 요약이 원장에 기록



후속버그 2건 즉시 수정

재검증 중 새 문제를 발견해 조치 · 알라딘 API 조회 오류 포함

단위테스트가 놓친 문제

컨테이너 안에서 WBF가 꺼진 채 단일 모델로 조용히 풀백

원인 컨테이너 이미지에서 torch 계열 의존성 누락

위험 로그를 읽지 않았다면 '양상블 동작'으로 발표할 뻔함

개선 기능활성 여부를 로그 한 줄로 확인 하도록 표시

비즈니스 엔진

UBCI 점수 기반 품질 등급 및 처리 방식

등급	점수 기준	HITL 기본 처분
MINT	≥ 95	APPROVE_NORMAL
GOOD	≥ 85	APPROVE_NORMAL
NORMAL	≥ 65	APPROVE_DOWNGRADE
REJECT	< 65	REJECT_RETURN



동적 가격 엔진

가격 산정은 룰 기반 결정론 함수, LLM 미개입

신품 가격 정책

01 입력

정가

02 핵심 산식

판매가 = 정가 × (1 - 기본 할인율)

03 출력

할인율 · 판매가



중고 동적 가격 정책

01 입력

정가 · 도서 카테고리 · UBCI 점수 ·
시즌 지수 · 희소성 정보 · 재고 체류 기간

02 핵심 산식

판매가 = 정가 × 카테고리 판매가율 × 상태 보정
× 시즌 가중치 × 희소성 보정
× 재고 체류 보정

03 출력

할인율 · 판매가 · 가격 선정 사유

중고 가격 보정 기준

카테고리	USED_RETAIL_BASE_RATE
상태 보정	$0.60 + 0.40 \times (\text{UBCI} / 100)$
시즌 가중치	$1 + (\text{seasonality_index} - 1) \times 0.5$
희소성	절판·한정판·초판 등 프리미엄, 최대 +15%
체류 보정	유행 민감 카테고리 감가, 최대 -10%

Restock Agent 상세

저재고 · 반려 이벤트 자동 발주 제안



LLM 호출 실패 시 FALLBACK_RULE로 결정론 대체 (fail-open) - 발주 프로세스 중단 방지

FDS 룰셋

룰 코드	탐지 대상
R1	블라인드 승인 (무검토 즉시 승인 비율)
R2	등급 오버라이드 (AI 등급 반복 상향)
R3	야간 대량 고액 (평소 대비 3배 이상)
R4	반품 어뷰징 (향후 적용)

- 적발은 룰(결정론), Analyst Agent는 권고 조치문만 서술

Weekly Insights Agent

정기 생성 구조



🔔 주간 인사이트 (2026-W33)

Insight Agent (gpt-4o-mini)

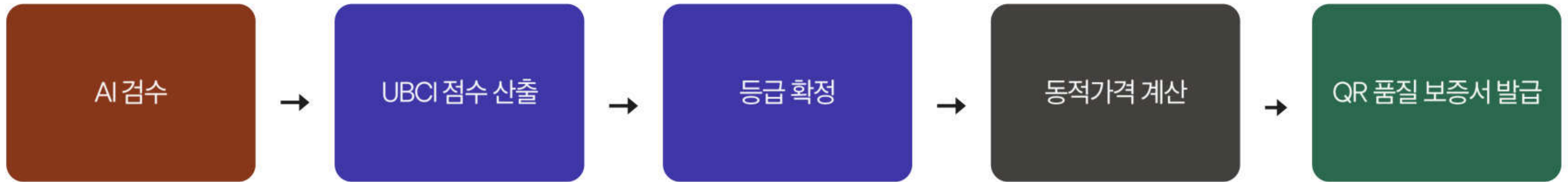
2026-W33 주간 집계에 따르면, 이번 주에는 총 144건의 검사가 이루어졌으며, 이를 통해 158,400원의 인건비 절감 효과를 달성했습니다. 특히, B 구역에서 93건의 검사가 집중적으로 이루어져, 해당 구역의 물류 효율성이 높아진 것으로 보입니다. 다음 주에는 반품이 0건으로 예상되어, 재고 관리와 출고 효율성을 더욱 주의 깊게 살펴볼 필요가 있습니다.

절감 인건비 (추정)	주간 검수	주간 주문	차주 반품 예측
158,400원	144건	11건	0건

▼ 지난 주간 인사이트

SQL 집계	LLM(GPT-4o-mini)
핵심 운영 수치 집계 매출·검수량·이관율·비용절감액	자연어 설명 생성

가격 결정 흐름 예시



ex) 결함 경미 → UBCI 88점 → GOOD 등급 → `calculate_used_retail_price(GOOD)` 호출 → 가격 확정 → QR 보증서에 등급 · 가격 동시 표시

외부 의존성 없는 4대 핵심 비즈니스 알고리즘 자체 구현

동적가격 · 3D 적재 · 상태점수 · 자동발주 — LLM 비의존결정론엔진



도서물류 전 과정을계산하는 네 개의엔진

동일입력 → 동일결과 · 외부상용솔루션호출없음

- 1 UBCI 상태점수**
100점 기준 결함별 감점 · 유형·심각도·감점 매트릭스
동일한사진과결함입력에대한동일점수
- 2 3D 적재배치**
Extreme Point + Best-Fit-Decreasing 자체구현
16종 규격 중 최적 박스선택 · 하단중량·상단 경량배치
- 3 카테고리 동적가격**
 $\text{정가} \times \text{카테고리율} \times \text{상태} \cdot \text{시즌} \cdot \text{회소성}$
재고기간별감가 · LLM 비개입규칙산식
- 4 수요예측 자동발주**
안전재고 이하발주제안 · 관리자승인이후집행
검증단계의제안수량상·하한 고정

범용 라이브러리 미사용 근거

도서 특화 적재 제약

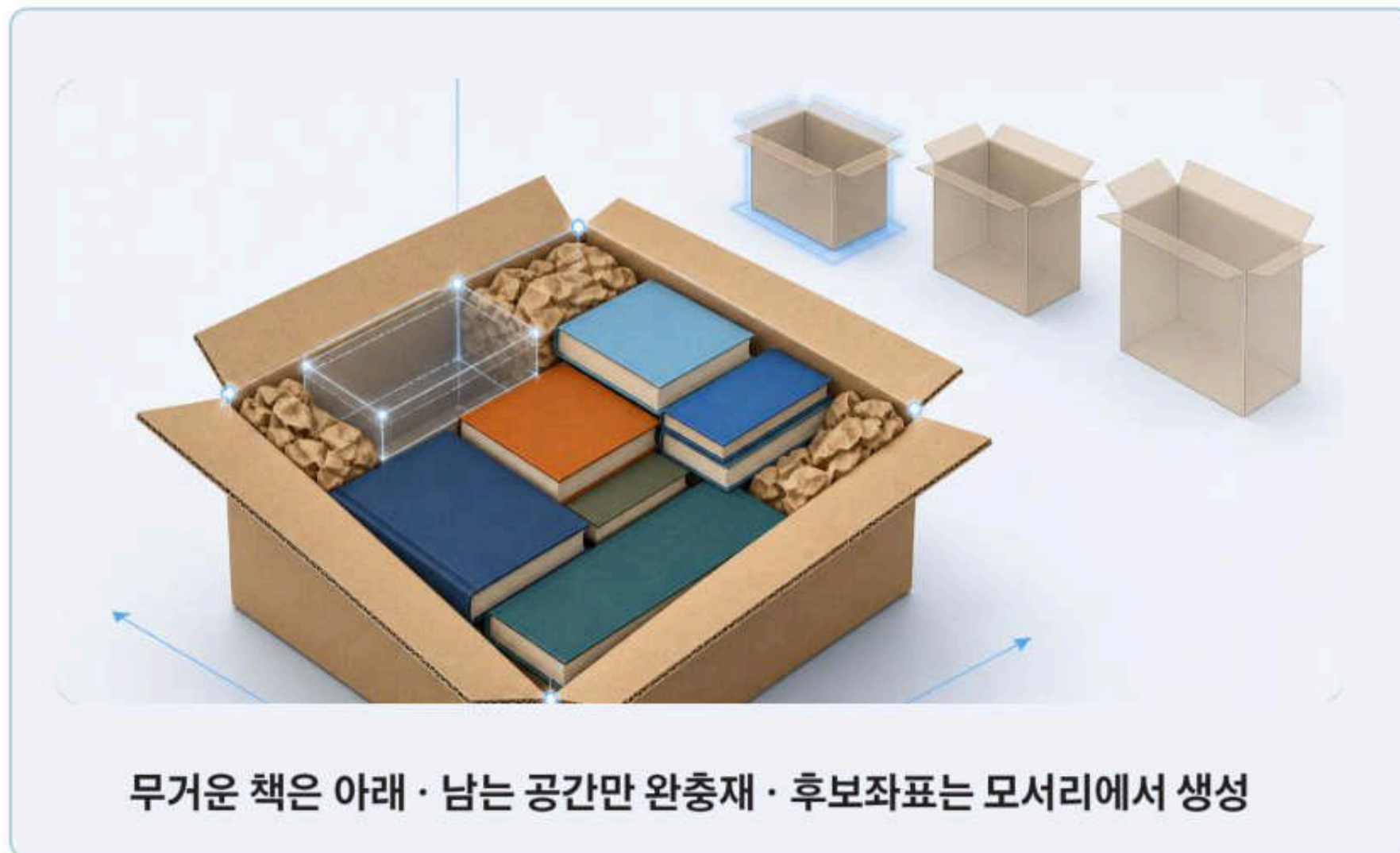
얇은두께와수평적재 · 16종 박스 · 좌표와하중의동시검증

결정론 일관성

UBCI 상태 점수의 직접입력 · 동일입력동일결과 · 가격·발주 규칙의 코드고정

3D Bin Packing(EP-BFD) 기반 최적 박스·완충재·하중배치

가상 3차원 좌표(x,y,z)에 실제 적재 시뮬레이션 — 16종 박스 중 최소 공극 선택



EP-BFD 적재 판단 흐름

입력된 도서가 실제로 들어가는지 좌표·회전·하중으로 검증

- 1 물성추정**
판형·페이지·양장 여부 → 가로·세로·두께·무게
- 2 BFD 정렬**
무겁고 큰 책부터 먼저 배치
- 3 Extreme Point**
이미 놓인 상자의 모서리를 다음 후보 좌표로 등록
- 4 배치판정**
겹침·박스 경계·총하중을 후보점마다 검사
- 5 박스선택**
16종 전수 탐색 → 공극이 가장 작은 규격 채택

검증근거

하중배치

양장·중량 도서 z=0
Bottom-Heavy 확인

치수·완충재

450p 컬러 양장 42.0mm
완충재 12mm · x=12.0 반영

분할·용량

150권 → 4박스 · 34.9kg ≤ 35kg
적재율 47.2% · 공극 52.8%

3D 좌표검증

좌표오류 0 · FIT-CHECK PASS
높이 98.6% · 너비 78.1% · 겹침 0

v1 → v2 | 부피비교에서 좌표 기반 충돌검증으로

v1은 부피만 맞으면 '들어간다'고 판단해 형태 충돌을 놓쳤습니다. v2는 좌표·회전·겹침을 직접 검사하고 16종을 전수 탐색해 판정과 실제의 불일치를 줄였습니다.

불완전 메타데이터 극복 — 도서 물성(두께/무게) 추정 알고리즘

외부 API가 제공하지 않는 물리 규격을 페이지수·재질 가중치 산식으로 추정



한 권의 책을 판형 · 페이지 · 재질 · 커버 정보로 재구성

EP-BFD 적재판단흐름

입력된도서가실제로들어가는지 좌표·회전·하중으로 검증

- 1 실측값 우선** 알라딘 TTB 실측규격
가로·세로·두께·무게·페이지 | 있으면무조건사용
- 2 추정식 풀백** 판형·페이지 수 기반계산
실측규격이없을때만사용

판형 Base Area

신국판 152×225 | 46판 128×188 | 4×6배판 188×257 mm

소설·에세이 기본 | 만화 | IT·교재 — 누락 시 카테고리 역추정

두께 $t = \max(3.0, p \times \tau + c)$

페이지당 흑백 0.05 / 컬러 0.08mm · 양장 +6.0mm · 일반 +0mm

하한 3.0mm — 페이지정보가작거나없어도두께가 0이 되는경로차단

중량 $w = (p/2) \times A \times g + c_v$

평량흑백 80 / 컬러 120g/m² · 커버양장 150g / 일반 50g

장(leaf) 수 × 판형 면적 × 평량 + 커버중량

계산예시

450p 컬러양장

$$450 \times 0.08 + 6.0 = 42.0\text{mm}$$

신국판 300p 흑백

$$150 \times 0.0342 \times 80 + 50 = 460.4\text{g}$$

추정값의소비처

EP-BFD 좌표배치

치수 3축 · 겹침판정

박스 허용 하중

중량검증

분할출고

마스터카톤 35kg 제약

3D 뷰어

치수 + 배치좌표

3차원 물류 로케이션(Zone-Rack-Shelf) 주소 체계 및 자동배정

3계층 물리주소체계 + 동선최적화 빈 공간자동추천



등급 · 카테고리 · 판형을결합해가장가까운 빈 공간탐색

판정결과가 물리 주소로변환

B · 03 · 02

ZONE 품질등급
A 신품 / B MINT / C GOOD
D NORMAL / E 반품·폐기 격리

RACK 카테고리
소설·IT·만화 등 분야구분
분야별 랙 1~5 배정

SHELF 판형크기
선반 1~4를 판형기준으로 구분
같은 규격을 모아 적재 효율 향상

ZONE A NEW · Fast-Track

신품 전용 고속 입출고

ZONE B MINT · UBCI ≥ 95

중고 최상급 전용 존

ZONE C GOOD · 85-94

중고 상급 표준 출고

ZONE D NORMAL · 65-84

중고 중급·할인 보관

ZONE E REJECT · ≤ 64

반품·폐기 격리

왜 등급을 위치로 분리했는가

검수에서 확정된 등급이 곧 물리 위치가 되어, 패키징 단계의 재확인을 없애고 현장 동선까지 이어집니다.
등급 혼재 보관 차단 → 가격·처리 경로 분리 → AI 판정의 현장 실행

동시성 오류 방지 — 스냅샷(Snapshot) 기반 피킹 지시서



순환논리를 끊는 3계층 분리

- 1 재고할당**
주문에 붙일 단품을 결정론 규칙으로 확정
AI가 개입하지 않는 구간
- 2 지시서 발행**
할당 결과를 스냅샷으로 고정
위치·수량·바코드 저장 후 재고 변동과 분리
- 3 동선안내**
GPT-4o-mini가 피킹 순서만 서술
위치 변경 불가 · 안내문 생성 전용

왜 3계층으로 분리했는가

위치는 결정론이 정하고, AI는 이미 정해진 위치를 어떤 순서로 돌지만 서술합니다.
발행된 지시서를 스냅샷으로 보존해 현재 작업자가 보는 값과 시스템 값의 불일치를 차단
AI가 동선과 재고 위치를 동시에 정할 때 생기는 원인·결과의 순환구조 제거

01 재고할당

결정론 규칙 · LLM 없음
order_items에 단품 배정

02 지시서 발행

결정론 스냅샷 · LLM 없음
picking_instruction_items 고정

03 동선 안내문

GPT-4o-mini · LLM 사용
피킹 순서 서술 · 위치 변경 불가

04 재고 차감

결정론 규칙 · LLM 없음
confirm-packing 시점 확정 차감

이벤트 기반(Event-Driven) 저재고 감지 및 자동 발주 루프

출고·폐기 이벤트마다 안전재고 임계 확인 → 발주 제안 → 관리자 승인 대기열



제안은 자동, 발주는 승인 후

- 1 트리거 3경로**
안전재고 미달 · REJECT · 예약점검
- 2 Collector**
재고·반품 이벤트집계 · LLM 미사용
- 3 Restock Agent**
GPT-4o-mini 1회 호출 · 제안 수량산출
- 4 Validator**
상하한 클램프 · 비정상 수량 보정
- 5 승인 칸반**
관리자가 최종 승인 또는 기각

왜 단일호출 + 결정론가드인가

발주 수량은 에이전트 토론보다 범위안의 안정성이 중요합니다. LLM은 한 번만 호출하고 Collector와 Validator가 앞뒤를 감쌉니다.

호출 실패 시 FALLBACK_RULE이 규칙산식으로 대체해 발주절차가 멈추지 않습니다.

01 환경별 정책 분리

자동제안 · HTIL

실제 발주는 승인 게이트 통과

02 신품 입고 경로 재사용

Fast-Track 그대로 활용

별도 입고 경로신설없음

03 가용재고 통합

신품 + 중고합산

부족 오인 · 과잉 발주 방지

04 Write-time 제안생성

이벤트 시점 1회 생성

조회마다 LLM 호출없음

XGBoost 기반 동적 가격(Dynamic Pricing) 추론 모델 학습

시즌·희소성·재고체류기간을 피쳐로 R^2 0.992 — 감가상각률 최적화

R^2 0.992

검증데이터분산의 99.2% 설명

입력피쳐 4종

할인율
상태점수
체류일
카테고리

0.017

MAE

평균 1.7%p 오차

0.022

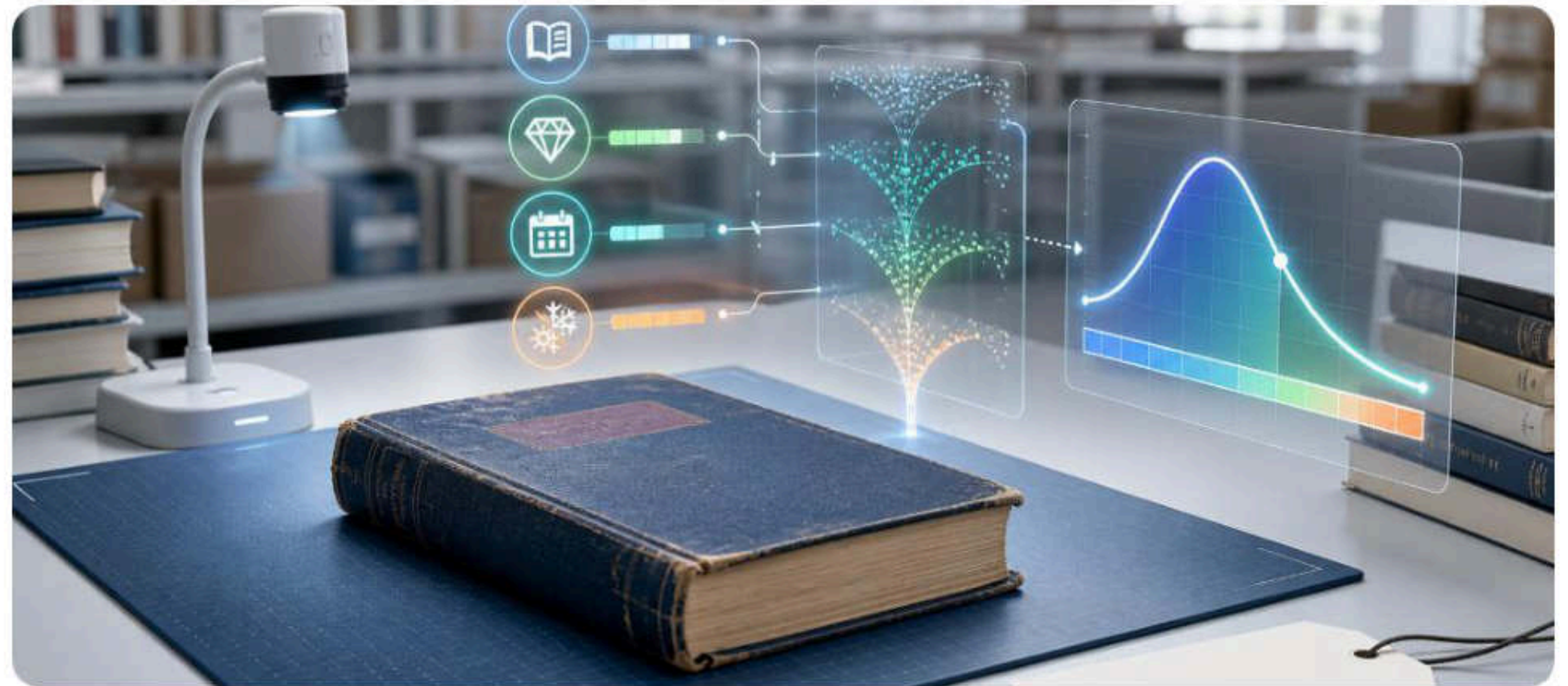
RMSE

큰 오차에도 안정

77.2%

오차감소

선형 대조군 대비



모델이 하는 일

할인율에 따른 구매 확률 예측
입력피쳐 4종 — 할인율·상태 점수·체류일·카테고리

모델이 하지 않는 일

가격상·하최종 판매가 직접 결정 없음
한은규칙고정 · 출력은 할인율 탐색에만 사용

학습이 실제로 일어난 증거

체류 180일 57.5% → 181일 53.2%
규칙에없는 -4.3%p 절벽 · 포화×할인 상호작용 학습

합성 데이터지만 규칙을 복제 X

규칙산식을 라벨로 복사하지 않고 비선형 반응을 데이터에 심어 모델이 스스로 발견하는지 검증했습니다. 선형대조군 MAE 0.0748 → 0.0171, 77.2% 개선은 규칙 밖 패턴을 실제로 학습했다는 증거입니다.

SPOF 방지 — AI 모델과 비즈니스 룰 엔진 디커플링 (Fail-Open)

XGBoost → 결정론산식 → 고정비율 3중 폴백 — 모델장애가 출고를 막지 않는다



왜 폴백을 3중으로 두었는가

AI 판독은 실패하면 HITL로 넘길 수 있지만, 가격은 빈 값일 수 없습니다. 모델 → 산식 → 고정률순으로 내려가며 모든 단계에서 가격을 산출하고, 사용된 폴백단계는 로그로 남겨 사후 구분합니다.

가격은 빈 값이 되지 않는다

정상 경로부터 최소보장 경로까지 이어지는 3중 안전망



상태보정 · 재현성 확인

평가 20,000원 · 체류 30일 · 소설

MINT
0%

GOOD
0.94

NORMAL
0.88

동일입력 → 동일가격

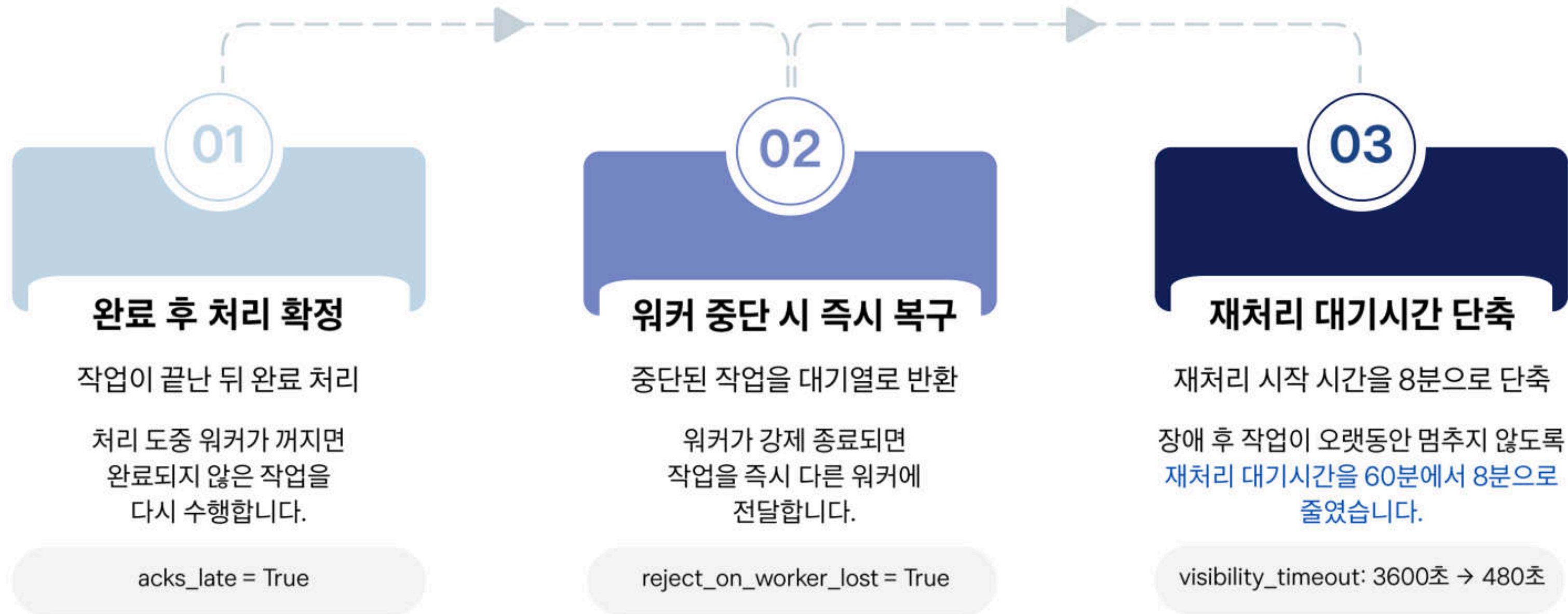
모델장애 ≠ 출고중단

SPOF 제거 · 결정론계산 · 고정률최소보장 · 폴백단계감사로그

백엔드 신뢰성

작업 유실을 막는 Celery 설정

작업 도중 워커가 중단돼도 검수 요청이 다시 처리되도록 구성했습니다.



DLQ 보관 및 재처리 정책

작업 도중 워커가 중단돼도 검수 요청이 다시 처리되도록 구성했습니다.

- 최대 재시도 횟수를 초과한 작업은 DLQ로 자동 이관
- DLQ는 최대 1,000건 또는 14일 기준으로 순환 정리
- 관리자가 실패 원인을 확인한 뒤 개별 작업 재처리 가능

DLQ 재처리 흐름

재시도에 실패한 작업을 별도 보관

재시도 횟수 초과
반복 처리 후에도 실패



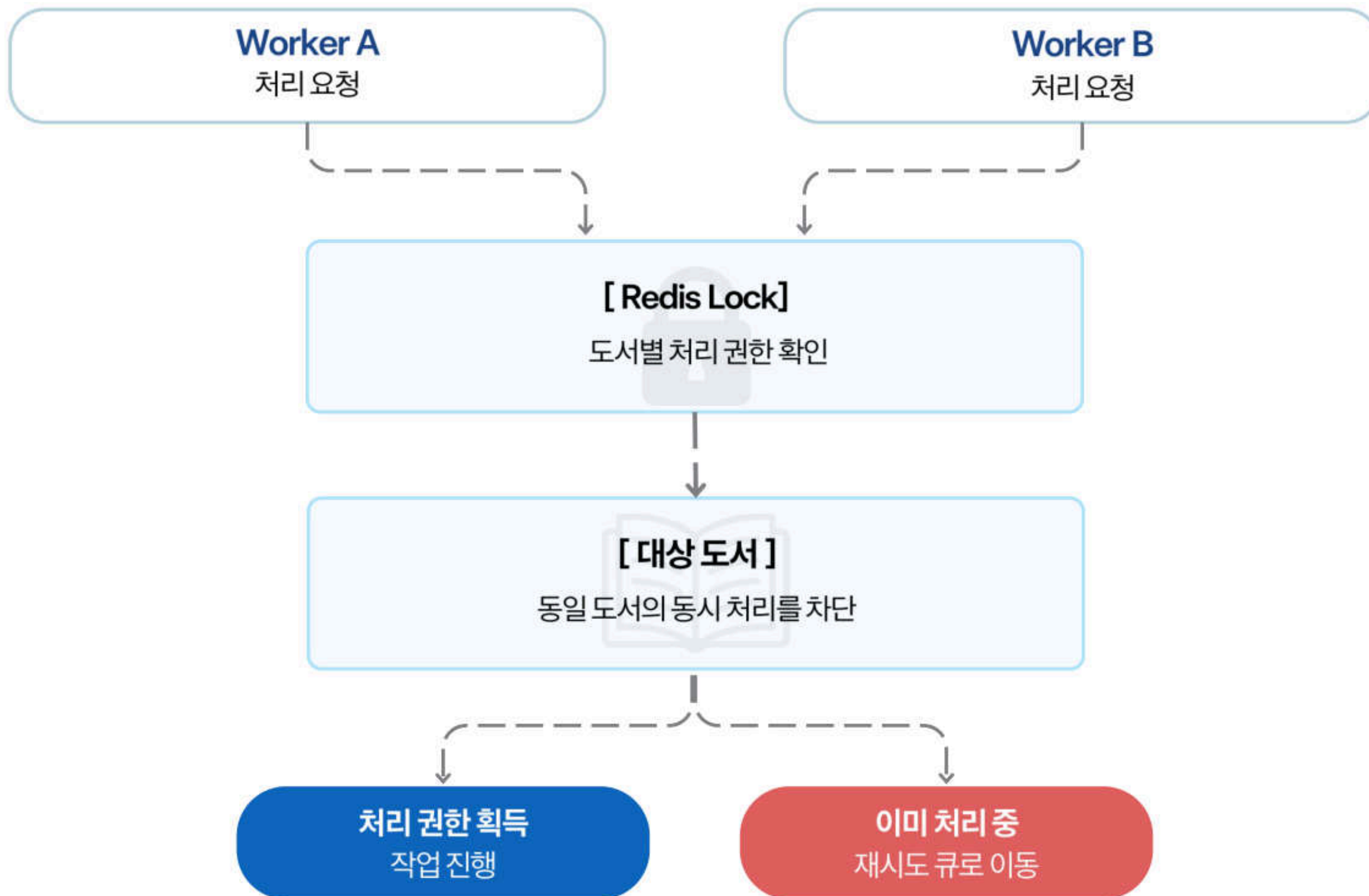
DLQ 자동 이관·보관
최대 1,000건 · 14일



원인 확인 후 재처리
관리자가 개별 작업 실행

Redis Lock 기반 동시성 제어

Redis Lock으로 같은 문서의 중복 처리를 막습니다.



SSE 기반 실시간 상태 전달

검수 요청 후 진행 상태와 결과를 화면에 실시간으로 전달합니다.

SSE 연결 준비



실시간 처리 흐름



카오스 테스트로 장애 복구 구조를 검증

검수 중 워커를 강제 종료하고, 작업 복구와 데이터 유실 여부를 확인했습니다.

실험 방법

01

검수 작업 실행

AI 검수 작업 4건 시작

02

워커 강제 종료

판정 진행 중 kill -9 실행

03

동일 조건 재실행

수정 전·후 각 2회 반복

동일 조건에서 반복 실행 가능한 테스트 스크립트 사용
scripts/chaos_test_runner.py

발견과 개선

발견

복구 시작까지 최대 1시간 지연

개선

복구 대기시간을 8분으로 단축

워커가 중단돼도 작업은 다시 처리됐으며,
실험에서 확인한 복구 지연을 설정에 반영했습니다.

성능·비용 실측 결과

검수 처리 시간과 장애 복구, 건당 운영 비용을 확인했습니다.



정밀 검수 시간

26~36초/건

이미지 업로드부터 최종 판정까지

결함 건 크롭 재검증 포함



장애 복구 결과

4/4건 완료

워커 강제 종료 후에도 작업 복구

데이터 유실 0건



검수 LLM 비용

약 20원/건

검수 1건 기준 상한 추정

모델 호출량과 환율에 따라 변동 가능

측정 기준 | 라이브 환경 실측 · 카오스 테스트 2회 · LLM 비용은 호출량 기준 추정

테스트 구성 및 재현성

백엔드·프론트엔드 테스트와 반복 실행으로 결과 일관성을 확인했습니다.

테스트 구성

Backend

pytest 기반 테스트
19개 테스트 파일

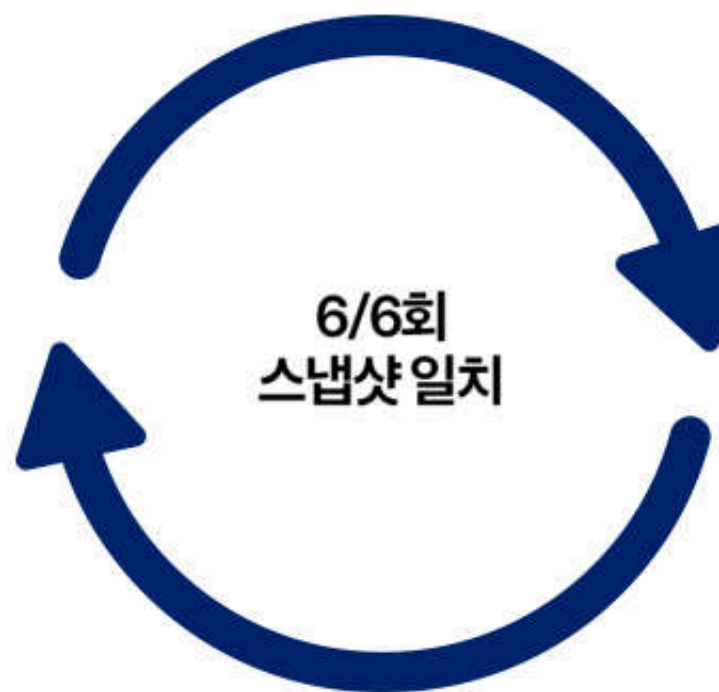
API·AI 파이프라인·운영 로직 검증

Frontend

Vitest 기반 테스트
8개 테스트 파일

화면 로직과 주요 기능 검증

결과 재현성



시드 2종을 각 3회 실행한 결과,
모든 스냅샷이 일치했습니다.

검증 조건 | 시드 2종 · 각 3회 반복

역할 기반 백엔드 모듈 분리(Decoupling) 및 의존성 역전

업무 기능은 도메인별로, AI 검수는 처리 단계별로 나누어 서로의 변경 영향도를 줄였습니다.



AI 검수는 특정 업무 기능에 속하지 않고, 검수 워커와 관리자 재검수에서 함께 사용됩니다.
따라서 AI는 도메인 밖에 두고 그래프 노드 단위로 분리했습니다.

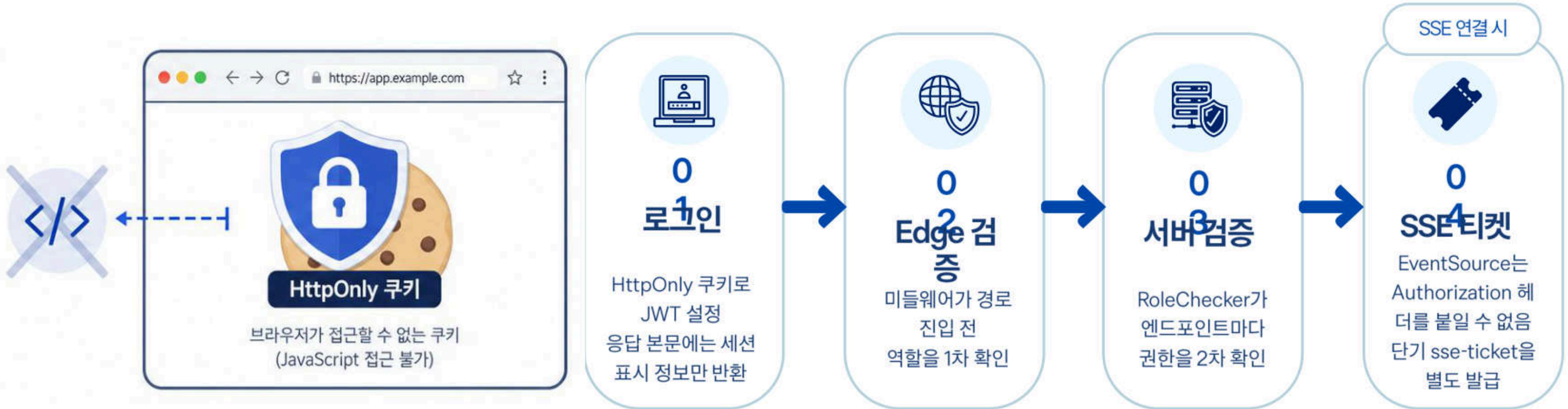
완료 에이전트 노드 분할

완료 orders 슬라이스 정리

진행 중 admin 슬라이스 계층 정리

HttpOnly 쿠키 기반 JWT 인증 및 XSS·CSRF 방어

JWT는 JavaScript가 접근할 수 없는 HttpOnly·Secure 쿠키로만 전달하고, 권한은 서버가 이중으로 검증합니다.



토큰을 localStorage·sessionStorage에 저장하지 않아 XSS 탈취를 차단하고, SameSite 쿠키 정책과 서버 권한 검증으로 CSRF 요청을 방어합니다



XSS로 인한 토큰 접근 차단

JWT는 HttpOnly 쿠키에만 있어
JS가 읽을 수 없음



클라이언트 번들에 인증 정보 없음

관리자 API도 일반 호출과 같은 경로로 요청



권한 판정은 서버에서 수행

프론트를 우회해도 서버 권한을 얻을 수 없음

단기 Ticket 기반 SSE 스트림 인증

EventSource의 커스텀 Authorization 헤더 제약을 1회용 sse-ticket으로 보완했습니다.



쿠키만으로 연결?
CORS·SameSite
조건이 까다로움



티켓을 짧게 만든 이유

- URL 쿼리로 전달되는 값
- 브라우저 기록에 남을 수 있음
- 짧은 수명·1회 사용으로 노출 범위 축소



ALB idle timeout 330초

- SSE는 장시간 유지되는 연결
- 기본 타임아웃이면 중간 연결 종료 가능
- 유휴 시간을 330초로 늘려 연결 유지



Authorization 헤더를 붙일 수 없는 SSE 연결은, 짧은 수명의 1회용 티켓으로 안전하게 인증합니다

단위·통합 테스트로 비즈니스 정합성 검증

기능별 검증과 반복 실행을 통해 결과의 일관성을 확인했습니다.

219건

전체 통과테스트

45개

테스트 함수

14.05초

전체 실행 시간

38%

라인 커버리지

검증 범위

01



결정 로직 테스트

UBCI감점 매트릭스
함수 8개 · 181개 케이스
같은 입력이면 같은 점수인지 확인

02



단위 테스트

개별 함수·산식 검증
함수 20개
가격 산식 · 규격 추정 · 등급 경계

03



API 통합 테스트

엔드포인트 요청·응답 검증
함수 17개
인증·권한처리 포함

“219건 통과”만으로 범위를 과장하지 않았습니다



219건 중 181건은 UBCI감점 매트릭스의 파라미터 조합입니다.

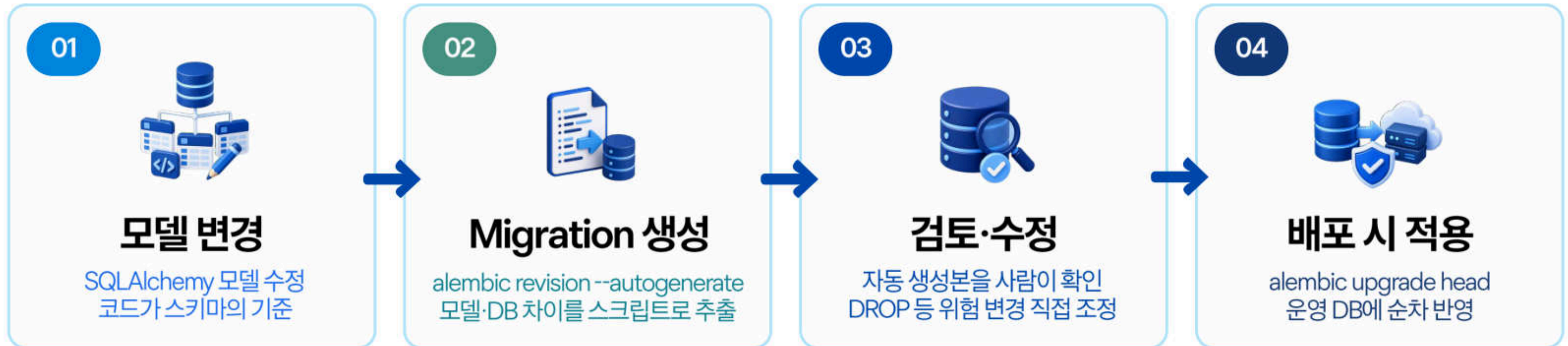
따라서 테스트 함수 기준으로는 45개이며, 어떤 결정 로직·개별 함수·API 흐름을 검증했는지 함께 제시했습니다.

라인 커버리지는 38%입니다. 총 6,767줄 중 4,216줄은 아직 테스트하지 않았으며, 수치를 부풀리기보다 미검증 영역을 명확히 관리했습니다.

Alembic 기반 데이터베이스 형상 관리

Migration as Code로 모든 스키마 변경을 기록하고, 개발·운영 환경에 같은 순서로 적용합니다.

스키마 변경 관리 흐름



베이스라인 리셋을 한 이유

- 초기 마이그레이션이 여러 갈래로 분기
- 적용 순서를 신뢰하기 어려움
- 현행 스키마를 기준으로 다시 시작

드리프트 점검

- 코드 모델과 실제 DB 차이 확인
- autogenerate 결과가 비어 있는지 확인
- 비어 있으면 모델과 DB가 일치

운영 DB 취급 규칙

- 라이브 시연 DB는 SELECT만 수행
- UPDATE·DELETE 직접 실행 금지
- 스키마 변경은 반드시 마이그레이션으로 적용

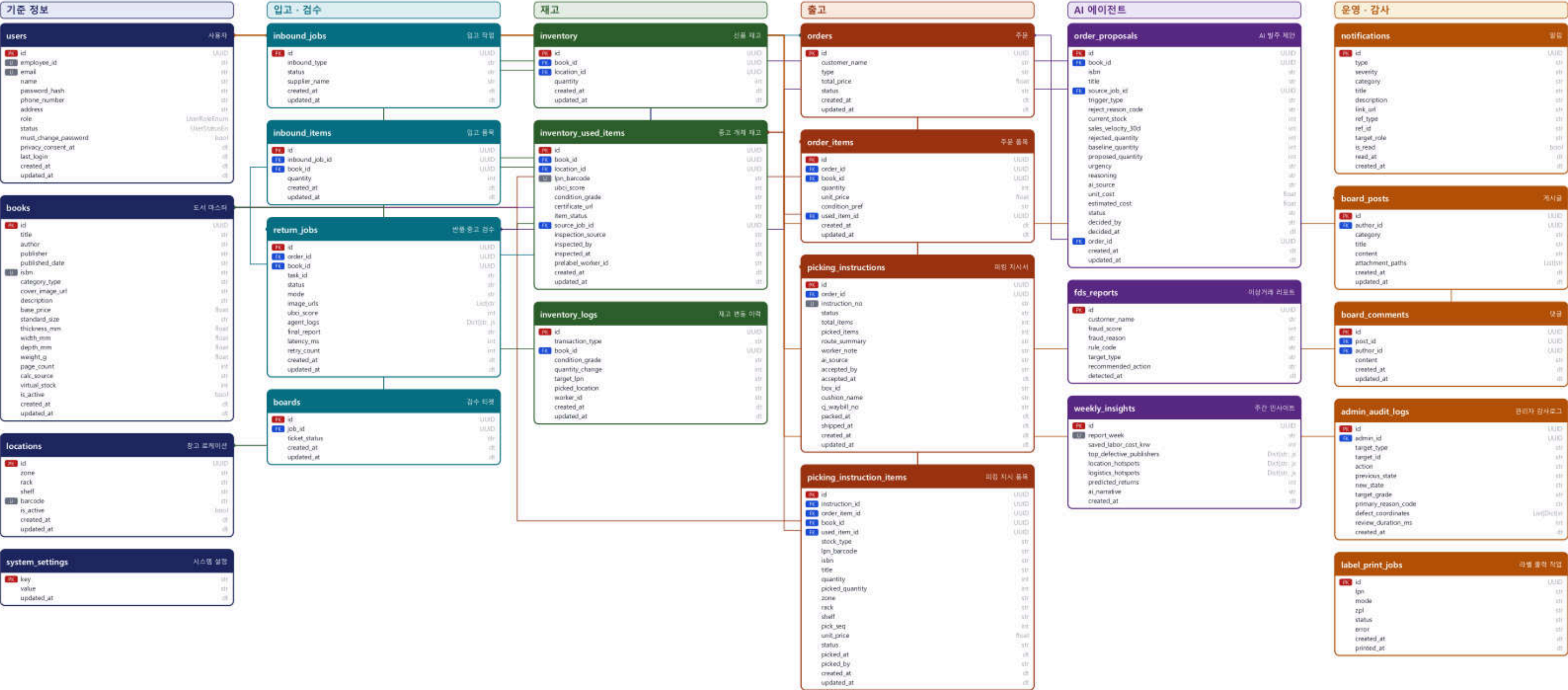
운영 DB를 직접 수정하지 않고 검토된 migration만 적용합니다. 변경 이력과 환경 간 정합성을 유지하며, 필요 시 이전 revision으로 되돌릴 수 있습니다.

백엔드 신뢰성

ERD

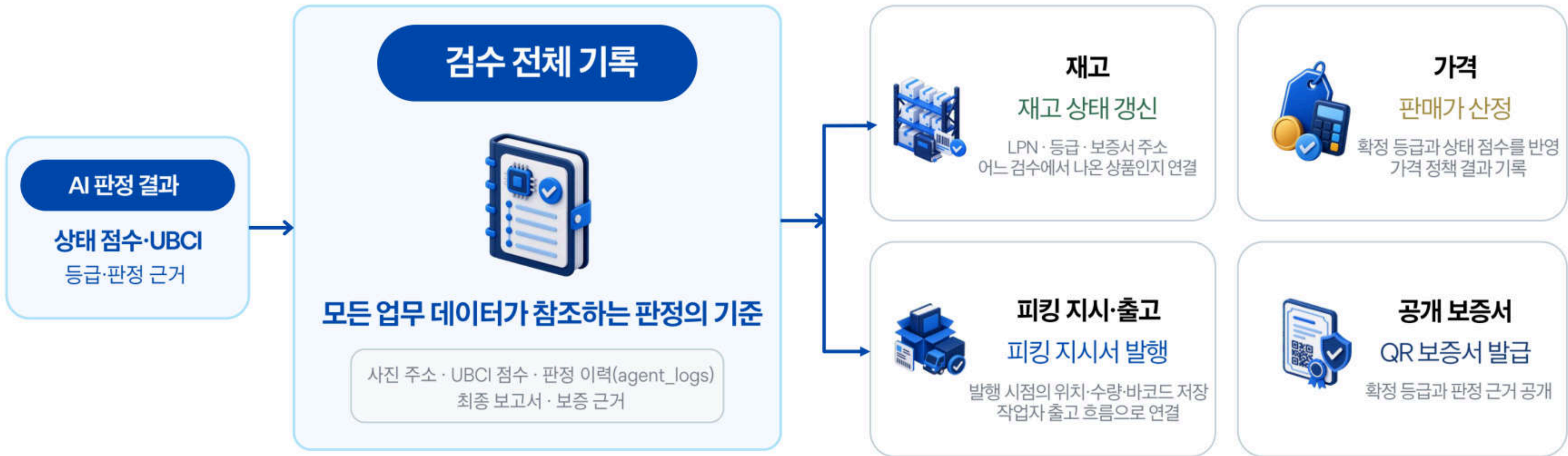
Nexus WMS — ERD (Entity Relationship Diagram)

테이블 23개 · 컬럼 250개 · 외래키 26개 | PostgreSQL 16 / SQLAlchemy / Alembic 관리



검수 기록 기반 데이터 통합 파이프라인

AI 판정 결과는 검수 기록을 기준으로 재고·가격·출고·공개 보증서 데이터에 일관되게 반영됩니다.



관리자 감사 기록

조치 종류 · 전후 상태
확정 등급 · 결함 좌표 기록

발주 제안

Restock Agent 산출
관리자 승인 대기 상태 관리

왜 판정 이력은 JSONB로 보관하나?

에이전트마다 판정 근거의 형태가 다릅니다.
Detector는 좌표 배열, Vision은 서술·확신도, Policy는 감점 내역, Critic은 위반 코드를 남깁니다.
이를 테이블로 나누면 여러 조인과 스키마 변경이 필요합니다.
대신 조회가 잦은 점수·등급·상태는 별도 컬럼으로 분리해 인덱싱했습니다.

도메인 주도 설계(DDD) 기반 REST API 라우팅 분리

같은 업무 데이터라도 도메인별 Endpoint와 역할별 DTO를 분리해, 권한에 맞는 데이터만 제공합니다.



비밀번호 변경을 프로필 수정과 분리한 이유

프로필 수정 요청에 비밀번호 필드를 함께 두면, 이름만 변경해도 비밀번호 값이 함께 전송될 수 있습니다. 빈 값이 들어가거나 로그에 남을 위험이 있어, 현재 비밀번호 확인이 필요한 별도 Endpoint로 분리했습니다.

LAN 내부망 하드웨어(라벨 프린터) 제어를 위한 로컬 브리지(Bridge) 설계

브라우저가 아닌 현장 브리지가 LAN 안의 프린터와 통신해, 클라우드 인쇄 명령을 현장까지 전달합니다.

웹·클라우드 환경

브라우저는 보안 정책상
LAN 프린터와 직접 통신 불가
클라우드 서버도 내부 LAN에 직접 도달 불가

포트 개방 시 내부망 노출 위험



02 현장 로컬 브리지

현장 PC의 브리지 에이전트가
큐의 인쇄 명령을 수신
내부 LAN으로 프린터에 전달



03 라벨 출력

현장 LAN 프린터로 전송
50×31mm 열전사 라벨 출력
LPN 바코드·QR 라벨 인쇄



01 인쇄 명령 적재

클라우드 서버가 Print Job 생성
큐에 요청을 안전하게 보관
에이전트가 꺼져 있어도
복구 후 오래된 요청부터 출력

✓ 큐 적재 → 에이전트 수신 → 출력 확인 | ✓ 클라우드-현장 E2E 경로 동작 확인 | ✓ 50 × 31mm 규격 · 바코드 인식 확인

브리지가 클라우드와 현장 LAN 사이를 중계해, 내부 장비를 안전하게 제어합니다.

글로벌 예외 핸들러(Global Exception Handler) 기반 에러 응답 규격화

서버·AI·DB에서 발생한 오류를 하나의 응답 규격으로 바꿔, 프론트엔드가 안정적으로 처리합니다.

01 예외 클래스 12종



BadRequest · Unauthorized
Forbidden · NotFound
PasswordPolicyViolation 등
서버 · AI 모델 · DB 오류 매핑

02 전역 핸들러 2개



HTTPException
UnexpectedException
발생 위치와 관계없이
예외를 한 곳에서 처리

03 표준 JSON 응답

```
{  
  "status": "...",  
  "code": "...",  
  "error_code": "AI_TIMEOUT",  
  "message": "처리 지연",  
  "path": "...",  
  "timestamp": "..."  
}
```

04 화면 분기



문구가 아닌 error_code 기준
프론트엔드 화면 분기

확장 필드 분리

로그인 실패 잔여 횟수·재시도 대기 시간은
message가 아닌 별도 필드로 전달

요청 로깅 미들웨어

요청 경로·상태 코드·지연(ms) 기록
X-Process-Time 헤더로 반환

트랜잭션 자동 롤백

처리 중 하나라도 실패하면
세션 전체를 되돌려 부분 저장 방지

문구가 아니라 error_code로 화면을 분기하는 이유

오류 문구는 수정되거나 다국어로 바뀔 수 있지만, error_code는 화면과 서버가 공유하는 고정된 계약입니다.
또한 출고 중 재고 차감은 성공했지만 지시서 저장이 실패하는 경우처럼, 일부만 저장된 상태가 남지 않도록 트랜잭션 전체를 자동 롤백합니다.

프론트엔드

프론트엔드 기술 스택과 화면 구조

역할별 화면을 분리하고, 공통 기능은 재사용할 수 있도록 구성했습니다.

Core Stack



Next.js App Router

페이지와 기능별 라우팅 구성



TypeScript

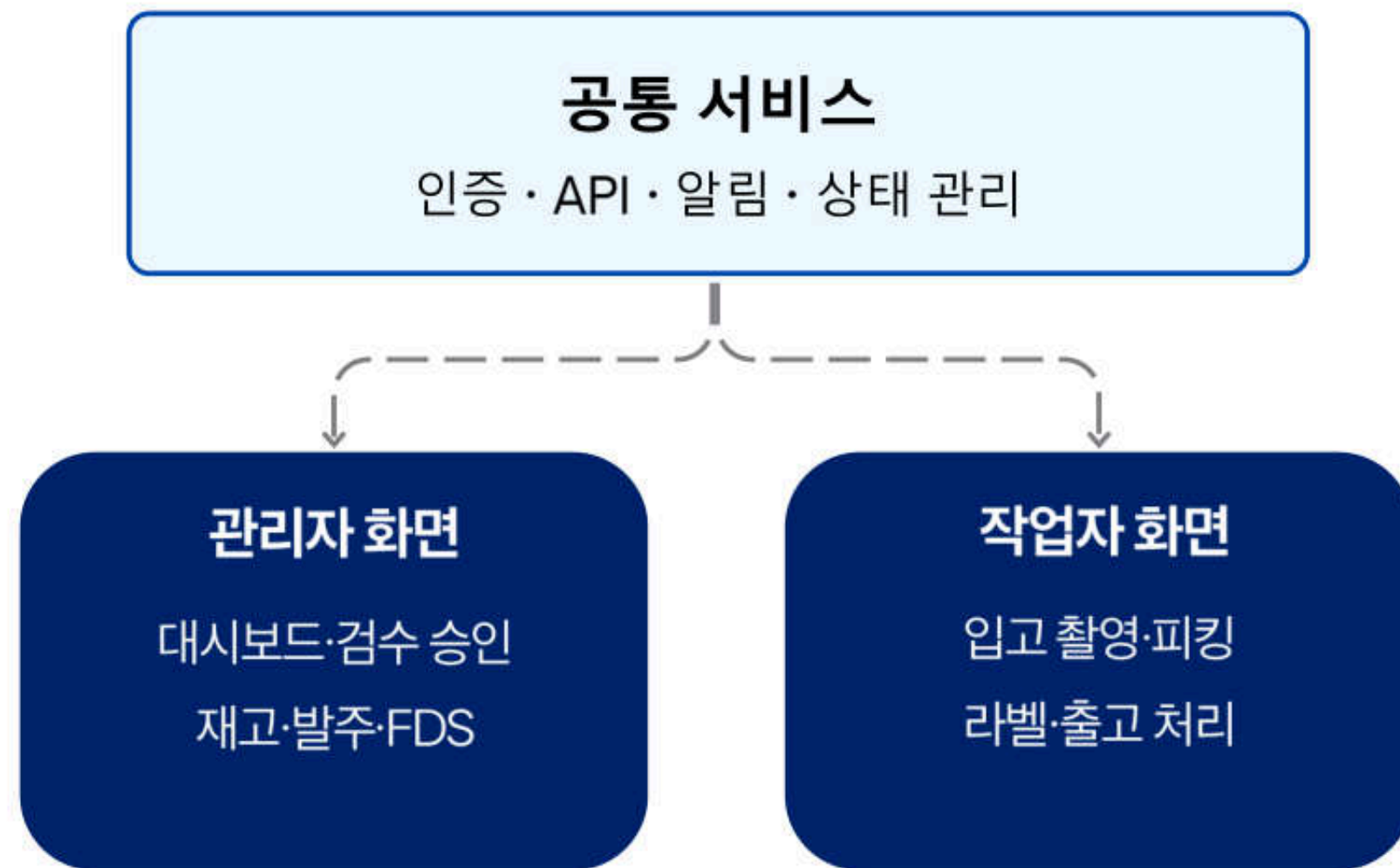
데이터 형식과 오류를 개발 단계에서 검증



Jotai

화면 간 상태를 가볍게 공유

역할별 화면 구조



PWA 오프라인 작업 처리

네트워크가 끊겨도 작업을 저장하고, 연결이 복구되면 자동으로 전송합니다.

01

네트워크 단절

통신 불안정 발생

Offline



02

작업 로컬 저장

작업을 기기에 임시 저장

IndexedDB



03

네트워크 복구

연결 상태 자동 감지

Online Event



04

자동 재전송

대기 중인 작업 전송

Background Sync

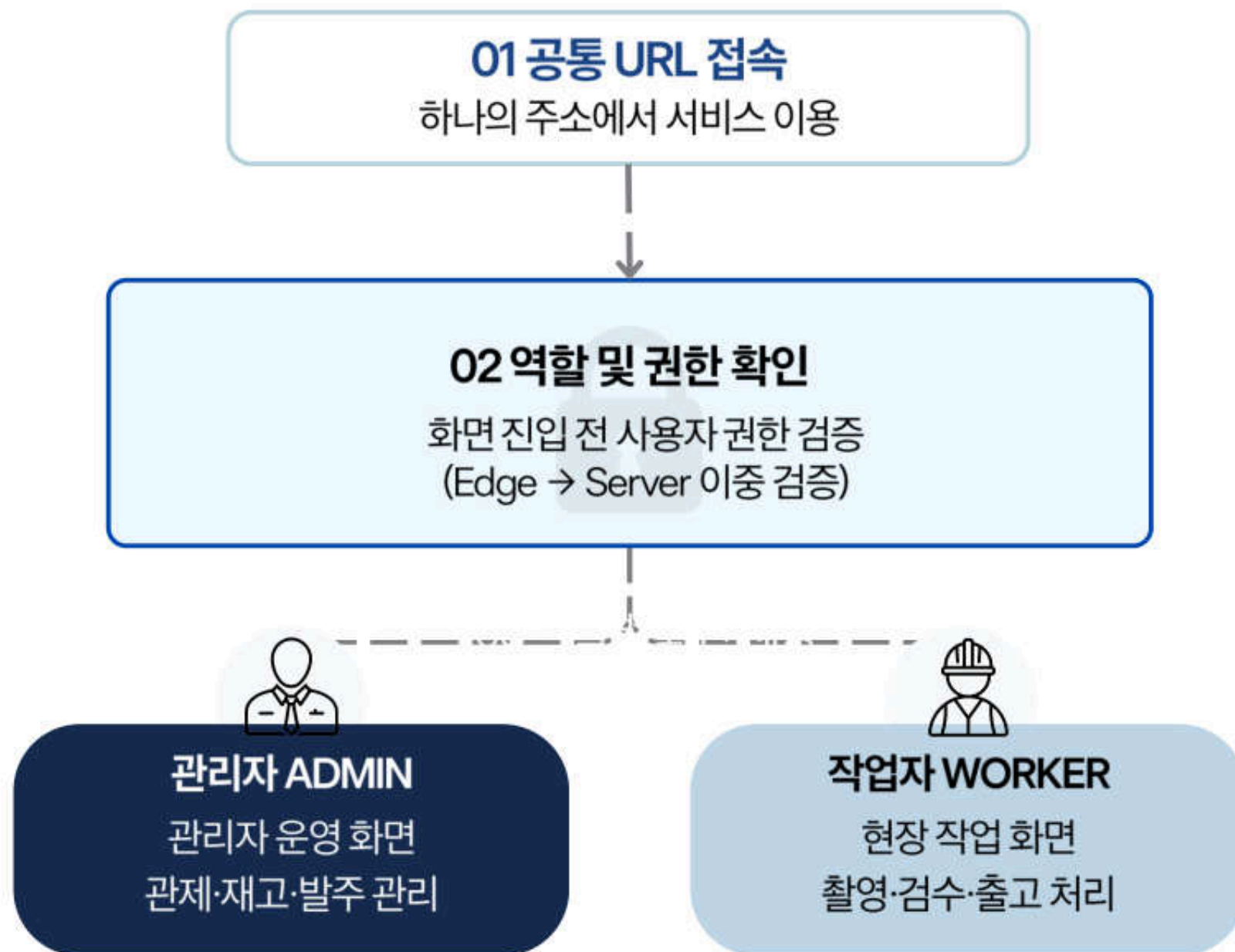


작업자는 연결 상태를 기다리지 않고 현장 업무를 계속할 수 있습니다.



하나의 URL, 역할에 맞는 화면 제공

같은 주소로 접속해도 로그인한 사용자의 역할에 따라 화면과 권한을 구분합니다.



역할별 앱을 따로 운영하지 않아 배포와 유지보수가 단순해집니다.

프론트엔드 테스트 전략

기능 단위 자동 테스트와 역할별 전체 흐름 검증을 함께 진행했습니다.

자동 테스트

기능별 자동 테스트

8개 테스트 파일

- Vitest 기반 테스트
- 컴포넌트 동작 확인
- 주요 기능과 상태 처리 검증

코드 변경 시 기능 오류를 빠르게 확인

시나리오 검증

로그인



입고



AI 검수



HITL 승인

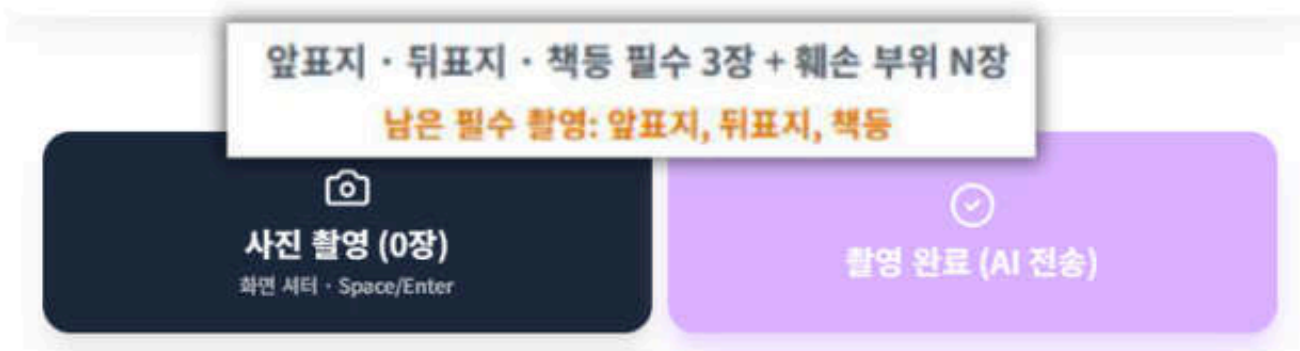


출고

기능 단위 오류와 전체 업무 흐름의 문제를 각각 확인했습니다.

현장 업무를 지원하는 주요 컴포넌트

촬영부터 판정 확인까지 반복되는 현장 작업을 공통 컴포넌트로 구현했습니다.



촬영 순서 관리

필수 촬영 순서 관리 (표지·책등·뒷면 3장 촬영)

필수 사진이 모두 촬영되도록 순서와 누락 여부를 확인합니다.

captureSequence.ts



검수 영역 수정

검수 영역 직접 수정 (드래그로 판정 영역 조정)

AI가 표시한 영역이 정확하지 않을 때 관리자가 위치를 수정합니다.

BBox 편집 캔버스



실시간 상태 공유

실시간 진행 상태 공유 (검수 결과와 알림 즉시 반영)

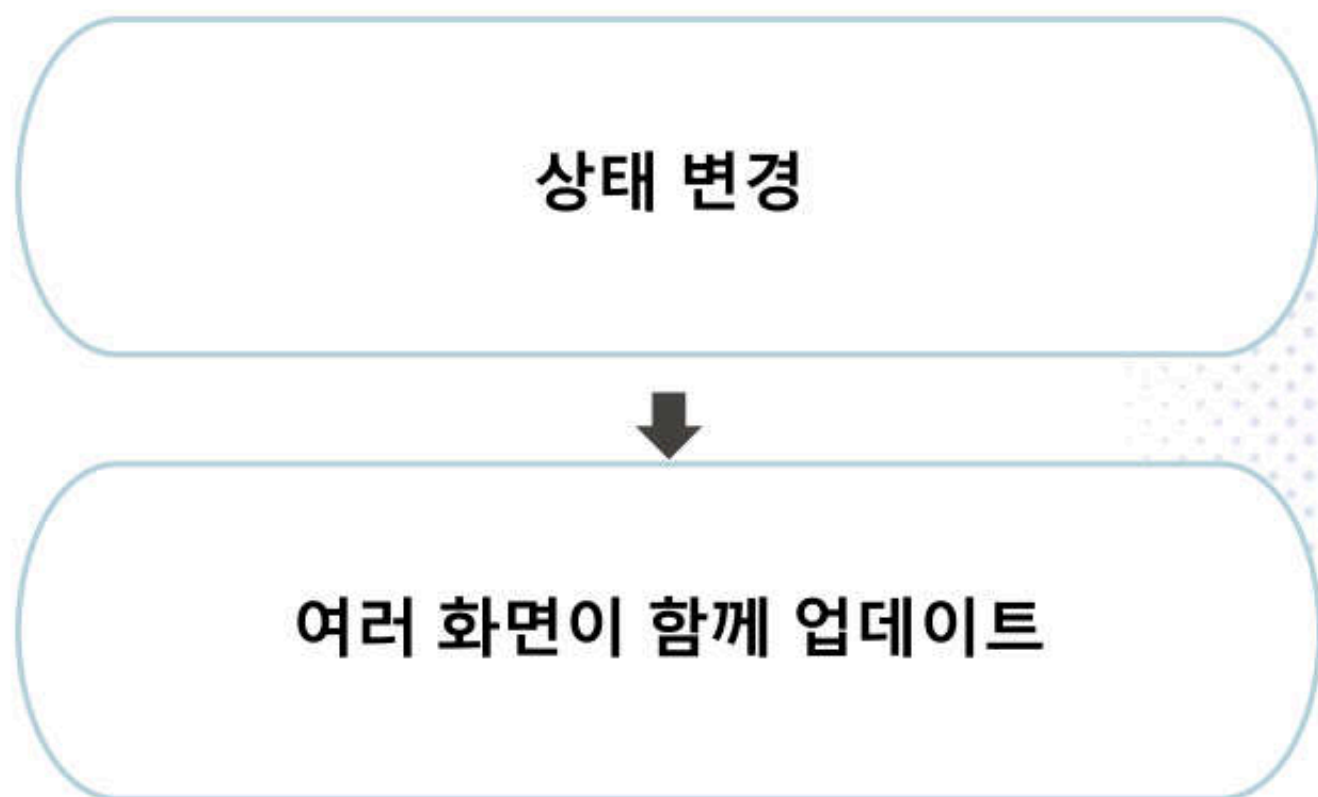
하나의 연결을 활용해 판정 진행 상황과 알림을 화면에 전달합니다.

SSE Hub

Jotai 기반 화면 상태 관리

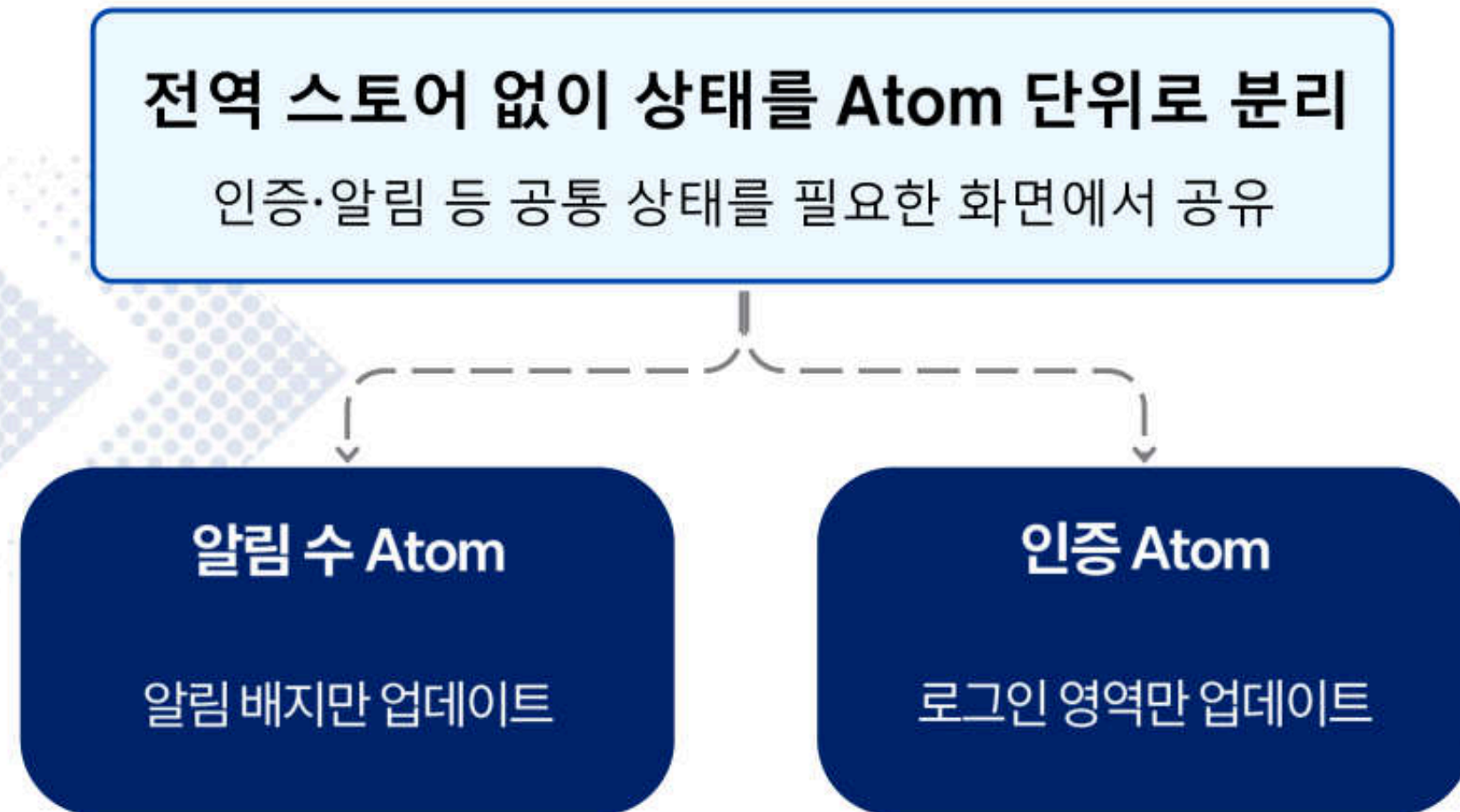
화면 상태를 작은 단위로 나누고, 필요한 컴포넌트만 업데이트합니다.

기존 방식



관계없는 컴포넌트도 다시 렌더링될 수 있음

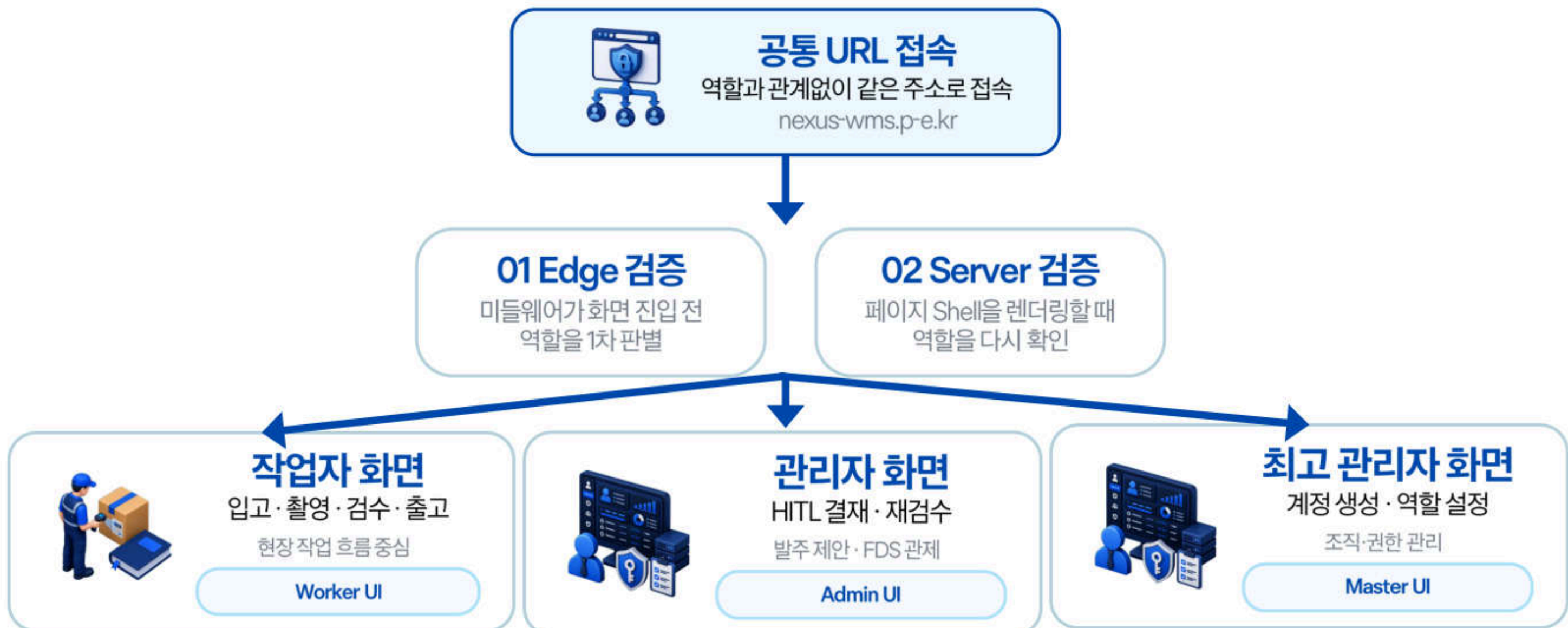
개선 방식



필요한 컴포넌트만 다시 렌더링

단일 진입점(Single Entry Point) 기반 역할별 UI/UX 라우팅 아키텍처

하나의 공통 URL로 접속하되, Edge와 Server의 이중 검증을 거쳐 역할별 화면을 제공합니다.



단일 URL을 안전하게 유지하는 장치

- RBAC 회귀 방지: Edge·Server 이중 검증 유지
- 메뉴 선택 상태: 역할 기준으로 Active 메뉴 재계산
- 기존 URL 유지: 북마크·공유 링크는 Redirect Shell로 연결
- 동시 세션 충돌 방지: 서버가 역할별 Shell을 최종 결정

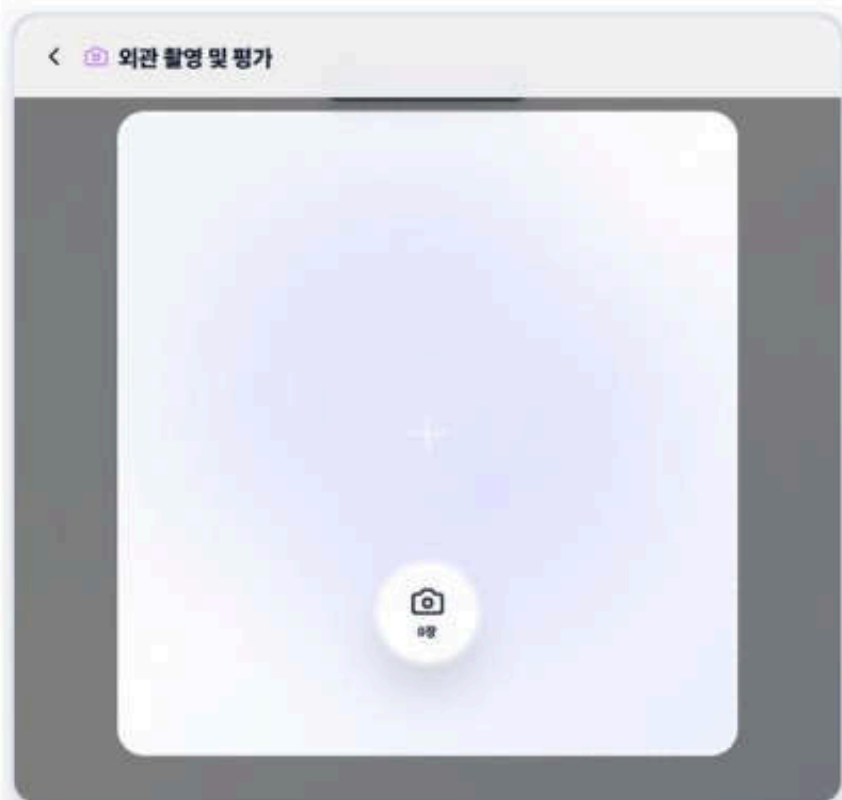
역할별 앱을 따로 운영하지 않아 배포 파이프라인은 하나로 유지하고, 인증·API·알림·상태 관리 컴포넌트도 중복하지 않습니다.

물류 창고 현장 환경을 고려한 프론트엔드 카메라 UX 최적화

장갑을 낀 손과 창고 조명 아래에서도, 한 손으로 안정적으로 촬영할 수 있게 설계했습니다.

큰 촬영 버튼

장갑을 낀 손도 누르기 쉬운 큰 터치 영역
한 손 촬영을 고려해 버튼 간격 확보
촬영 직후 미리보기로 흔들림 확인



필수 촬영 순서 고정

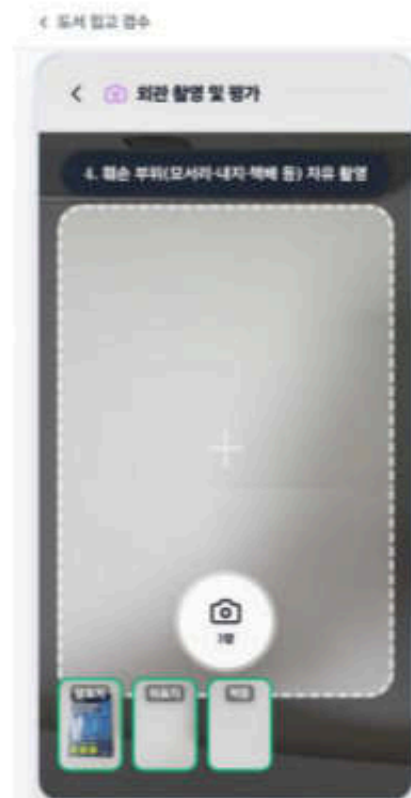
captureSequence.ts로 표지 → 책등 → 뒷면 순서 관리

필수 컷이 누락되면 다음 단계로 진행하지 않음
촬영 순서가 판정 안정성의 시작점



촬영 품질 즉시 안내

안찍은 면은 '결함 없음'이 아님
컷이 비어 있으면 자동 등급 확정 차단
재촬영이 필요하면 해당 단계부터 다시 시작



카메라 UX는 AI 판정 품질에 직접 영향을 줍니다

학습 데이터는 평면 위 도서 사진이었지만, 현장에서는 손에 들고 촬영하며 손·머리카락·가구·그림자가 함께 들어옵니다. 이로 인해 오탐이 발생해 Finetune v8에서는 모델만 보정하지 않고, 촬영 순서와 컷별 예시 실루엣으로 올바른 촬영 방식을 유도했습니다.

PWA·IndexedDB 기반 오프라인 작업 동기화

창고 통신 음영 지역에서도 작업을 저장하고, 연결이 복구되면 대기 작업을 자동으로 순차 전송합니다.

01



네트워크 단절

통신 불안정 감지
오프라인 배너 표시
Wifi OFF

02



로컬 작업 저장

IndexedDB에 사진·작업 비동기 보관
화면에 보관 중 3건 표시
IndexedDB 작업 큐

03



연결 복구 감지

오프라인 · 보관 중 3건
ONLINE

04



자동 재전송

대기 중인 작업을 순차 전송
작업별 Client ID로 중복 전송 방지
자동 동기화

오프라인 · 보관 중 3건

작업은 저장되었으며, 연결되면 자동 전송됩니다.
작업자는 연결을 기다리지 않고 다음 책으로 이동합니다.

왜 IndexedDB인가

localStorage는 용량이 작고 동기식
사진·작업 저장에는 비동기·대용량 IndexedDB 사용

중복 전송 방지

작업마다 Client ID 부여
서버가 같은 ID를 다시 받으면 무시

한계도 관리

기기를 바꾸면 대기 작업은 옮기지 못함
브라우저 저장소 삭제 시 유실 가능
장기 오프라인은 보장하지 않음

네트워크 실패를 작업 실패로 처리하지 않고, 저장·재전송 상태로 관리합니다.

알림 하나가 바뀌어도, 필요한 화면만 다시 그립니다.

인증·알림·작업 상태를 Atom 단위로 분리해, 필요한 컴포넌트만 업데이트합니다

전역 스토어 방식

알림 변경 1건

관련 없는 화면까지 함께 갱신



화면별 API 요청

상태 하나가 바뀌면 구독 컴포넌트가 함께 갱신
단일 전역 스토어는 보일러플레이트가 커짐

key 안정성

재고 목록처럼 순서가 자주 바뀌는 리스트는
index를 key로 사용하지 않음

서버 컴포넌트 우선

상호작용이 없는 화면은
클라이언트 번들에 넣지 않음

memo는 측정 후 적용

얇은 비교 비용이 더 클 수 있어
필요한 구간에만 적용

useCallback 남용 금지

의존성 배열 관리 실수가
더 큰 버그 원인이 될 수 있음

Jotai Atom 방식

알림 변경 1건

인증 Atom · 알림 Atom · 작업 Atom



공통 상태를 Atom 단위로 보관
필요한 컴포넌트만 다시 렌더링

전역 단일 스토어보다 현재 화면 규모에 맞는 가벼운 상태 관리 방식을 선택했습니다.

HTTP Client 인스턴스 통합 및 전역 인터셉터 마이그레이션

각 화면은 요청만 보내고, 인증·오류 규칙은 공통 Client가 처리합니다.



마이그레이션 순서



공통 Client가 인증 쿠키·401 처리·오류 메시지 변환을 담당해, 모든 화면이 같은 규칙으로 API를 호출합니다.

남은 작업: 일부 레거시 화면에는 직접 호출이 남아 있습니다. 기능은 정상 동작하지만, 공통 Client 규칙으로 정리 중이며 완료로 표현하지 않습니다.

공통 필수 도메인 컴포넌트화 및 코드 재사용성 극대화

회원·인증·게시판·업로드의 반복 규칙은 공통 Hook과 컴포넌트로 만들고, 각 화면은 필요한 기능만 조합합니다.

공통으로 사용하는 4가지 기능



한 번 만들고, 모든 화면에서 다시 씁니다

- 01 공통 Hook으로 로직 분리**
회원·인증·게시판·업로드 규칙을 한 곳에서 관리
- 02 공통 컴포넌트로 화면 통일**
입력·검증·업로드 UI를 같은 기준으로 재사용
- 03 페이지는 조합에만 집중**
페이지 → Hook → API Client로 분리 기능 수정 범위를 줄임

내부 WMS에 맞춘 공통 도메인 설계

직원 계정은 자유 가입이 아니라 최고관리자가 생성하고 역할을 부여합니다. 재고와 판정 이력이 남는 내부 시스템이므로, 가입 경로부터 접근 권한을 통제했습니다. 파일은 서버를 거치지 않고 S3에 직접 업로드하며, 서명 URL로 업로드 권한을 확인하고 CDN 경유 열람만 허용합니다.

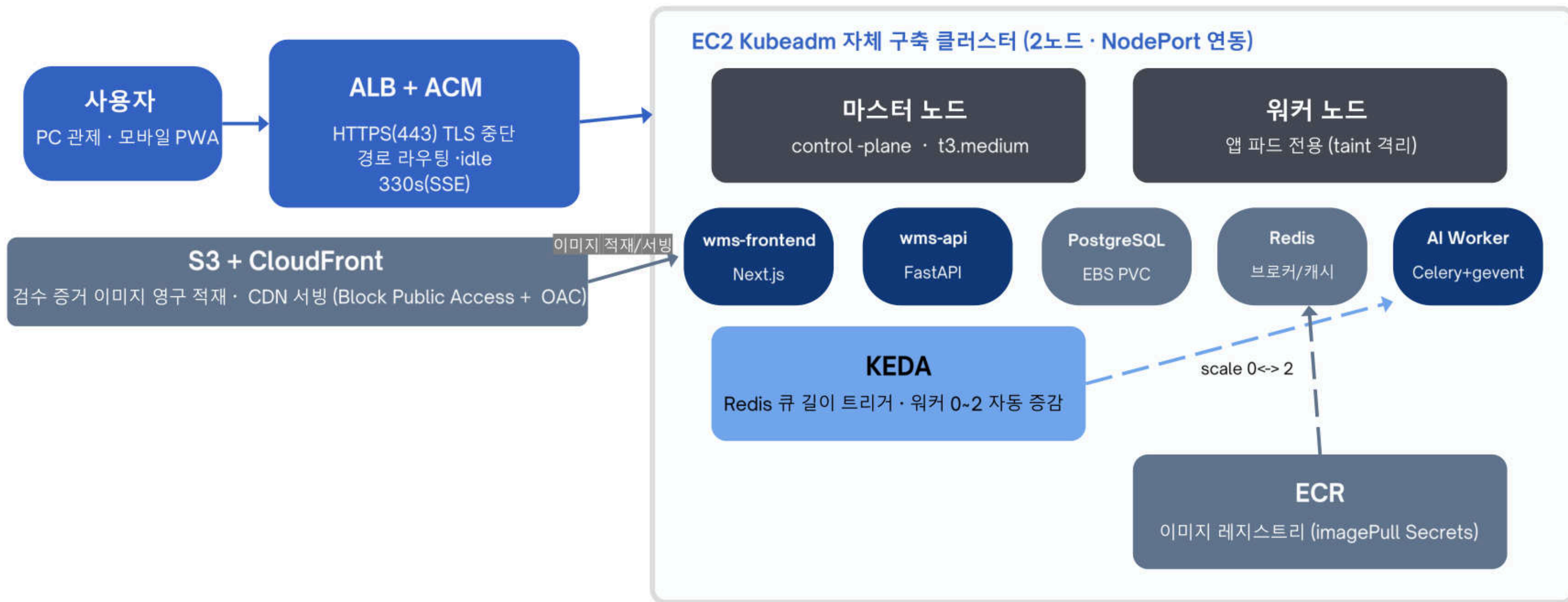
DevOps

실제 운영 환경을 구성한 클라우드 아키텍처

AWS 기반 인프라에서 HTTPS 접속, 이미지 저장, AI 워커 자동 확장을 운영합니다.

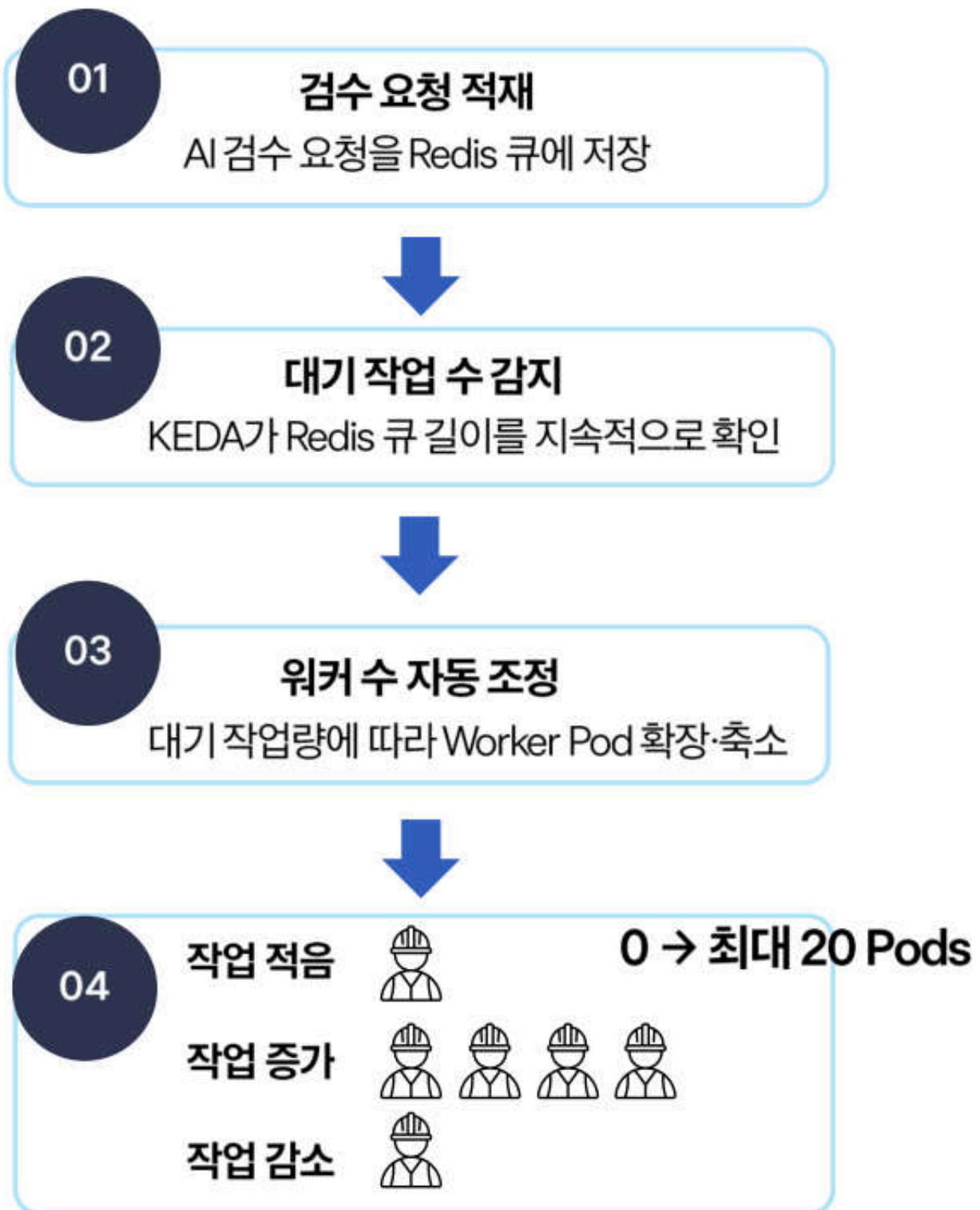
LIVE · nexus-wms.p-e.kr

2026-08-09 실배포 · HTTPS 운영 중



작업량에 따라 자동으로 확장되는 AI 워커

CPU 사용률 대신 Redis 대기열을 확인해 필요한 만큼 AI 워커를 자동으로 조정합니다.



왜 CPU 기준이 아닌가?

AI 워커는 대기 중 CPU 사용률이 낮아,
작업이 쌓여도 CPU 기반 확장이 늦어질 수 있습니다.

- CPU 기준: 실제 대기 작업량 반영 어려움
- Redis 큐 기준: 쌓인 작업을 직접 감지

요청이 몰리면 워커를 늘리고,
작업이 줄면 다시 축소해 자원을 효율적으로 사용합니다.



매일 자동 실행되는 재고 점검

CronJob이 매일 새벽 재고를 확인하고, 부족 상품의 발주 제안을 자동으로 생성합니다.



코드 변경부터 무중단 반영까지

자동 검증을 거친 뒤, 서비스 중단 없이 새 버전을 순차 반영합니다.



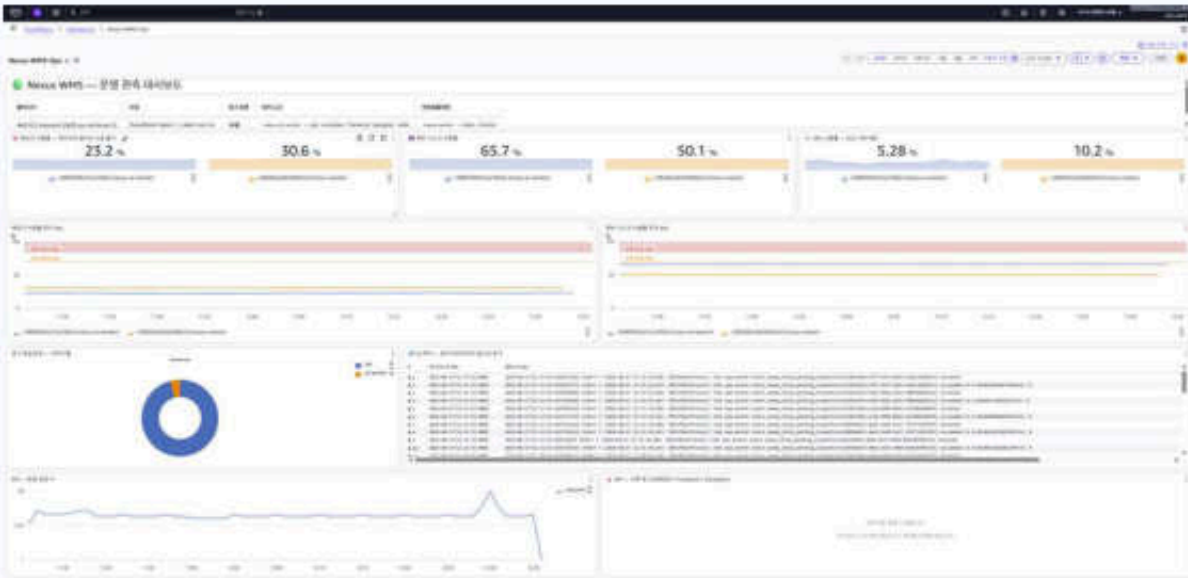
보안 인증과 이미지 검증을 거친 배포만 운영 환경에 반영합니다.

CloudWatch 기반 인프라 메트릭 수집 및 이상 징후 모니터링

서비스·노드 핵심 지표를 수집해 운영 상태를 확인하고, 임계치 규칙으로 알림 대응을 연결합니다.

서비스·노드 상태 수집

로그와 리소스 지표를 한 화면에서 확인



수집 → 집계 → 대응

01

핵심 로그·지표 수집

검수 API · AI Worker · 웹 서버
예약작업 로그 수집
워커 노드 Memory · Disk 수집

02

CloudWatch 집계

서비스 로그 수집 경로 4개
로그 보존 기간 14일
워커 노드 자원 사용률 확인

03

임계치 규칙 설계

수집 추이를 기준으로
운영 임계치 기준 마련
알림 규칙은 아직 미설정



CPU는 CloudWatch 기본 지표를 사용해 중복 수집하지 않습니다.

노드 Memory·Disk는 서버 밖에서 보이지 않아, 수집기를 노드에 직접 구성했습니다.

4개

서비스별 로그 수집 경로

14일

로그 보존 기간

19%

워커 노드 Memory 사용률

37%

워커 노드 Disk 사용률

서버 2대 규모에서는 별도 감시 도구를 추가하지 않고, CloudWatch와 노드 수집기로 필요한 지표만 확인합니다.

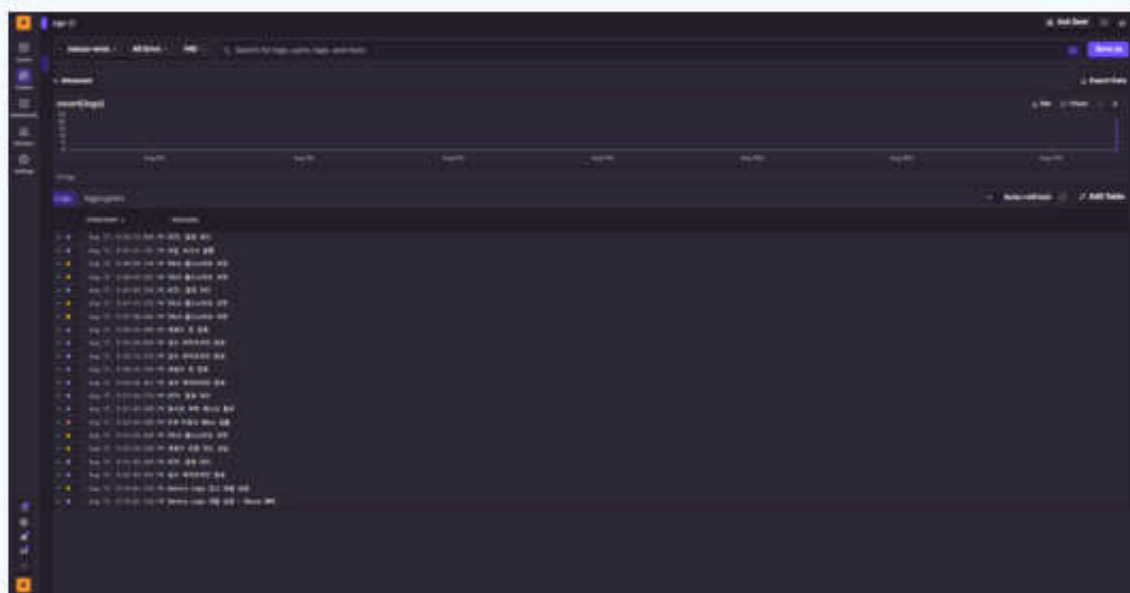
현재는 로그·자원 지표 수집과 가시화까지 구현했으며, 경보 규칙은 운영 정책 확정 후 설정할 예정입니다.

Sentry 기반 프론트·백엔드 통합 오류 추적 시스템 구축

사용자 보고 전에 오류 흔적과 Stack Trace를 확보해, 장애 복구 시간을 줄입니다.

서버 로그만으로는 놓치는 오류

- 3D 적재·결합 표시처럼 무거운 화면은 브라우저에서 실행됩니다.
- 브라우저에서 오류가 나면 서버 로그에는 기록이 남지 않을 수 있습니다.
- 사용자가 알려주기 전에는 장애를 알기 어렵습니다.



오류 추적 흐름

01

오류 감지

브라우저 런타임 오류 발생

02

Sentry 자동 수집

오류 유형 · 발생 화면 · URL ·
브라우저 정보 기록
Stack Trace 함께 확보

03

개발팀 확인·복구

사용자보고 전 원인 확인
복구 시간(MTTR) 단축

● 오류가 난 화면과 필요한 환경 정보만 기록해, 평상시 개인정보 수집은 최소화합니다.
알림 연동·코드 위치 자동 추적은 후속 운영 항목으로 분리

서버 로그로는 안 보이는 것

무거운 화면은 브라우저에서 실행
오류가 나도 서버 기록이 없을 수 있음

검증한 방법

예외 처리를 넣지 않은 화면에서 오류를 발생시켜 확인
발생 화면·URL·브라우저 정보 수집 확인

수집 범위와 원칙

오류 기록은 자체 도메인을 거쳐 전송
오류 화면만 남기고 평상시 기록은 최소화

오류를 자동 수집하고 원인을 빠르게 확인해, 사용자 경험에 영향을 주기 전 대응합니다.

GitHub Actions 기반 무중단(Rolling) CI/CD 배포 자동화 파이프라인

코드를 반영하면 검증·이미지 릴리즈·순차 교체가 자동으로 이어집니다.

01 자동화 배포 파이프라인

01

Git Push

main 브랜치 반영 / 워크플로우 자동 시작

02

GitHub Actions

Build · Test · Lint 자동 검증 / 검증 통과 시 다음 단계 진행

03

ECR 이미지 릴리즈

버전 태그와 함께 이미지 등록 / 배포 이력 추적

배포 조건 | 자동 검증을 이미지에 한해 다음단계 진행

02 Kubernetes Rolling Update

새 Pod가 Ready 상태가 된 뒤, 기존 Pod를 한 개씩 교체

서비스를 끊지 않고 버전을 바꾸는 배포 방식

기존 상태



기존 상태



서비스 반영

정상 Pod 확인 후 트래픽 전환
문제 발생 시 이전 버전 유지

OIDC 단기 인증

AWS 접근키를 저장소에 보관하지 않음
GitHub 신원 토큰으로 IAM 임시 권한 발급

이미지에 실행 환경 포함

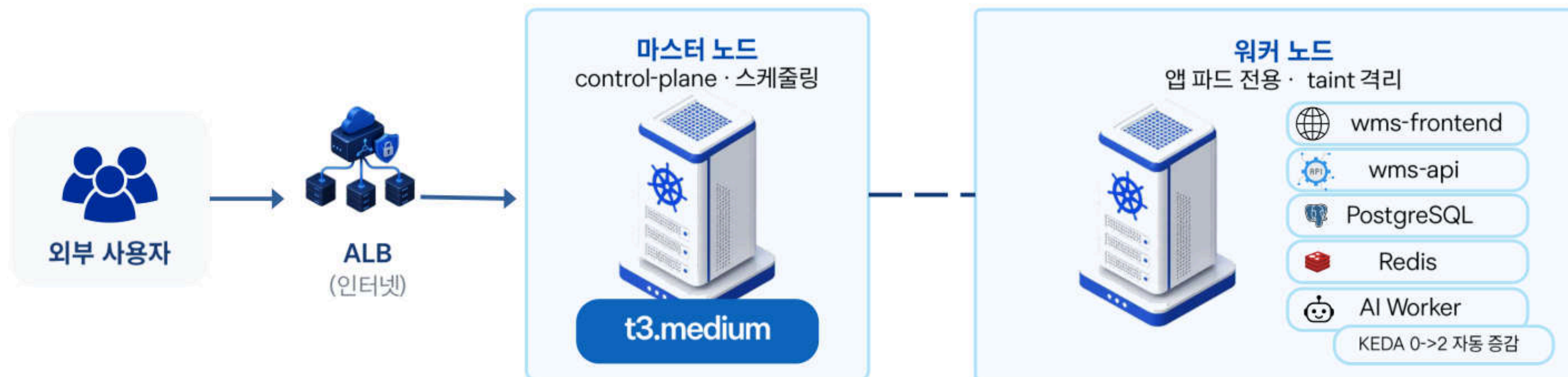
YOLO 가중치와 의존성을 이미지에 포함
기동 시 외부 다운로드 없이 실행

롤링 배포 선택

Blue-Green은 서버가 두 배 필요
2노드 환경에서는 Pod 순차 교체로 중단 최소화

Kubeadm 기반 Kubernetes 클러스터 자체 구축

관리형(EKS) 비의존 — Control-plane/Worker 직접 제어와 Taint 격리



왜 EKS 대신
Kubeadm 인가?



고정 제어 플레인 비용 절감

EKS의 고정 제어 플레인 비용이 발생하지 않습니다.



EC2 2대로 운영

마스터 1대 + 워커 1대로 클러스터를 구성합니다.



대신 직접 관리

업데이트, 백업, 모니터링 등 운영을 직접 담당합니다.



PostgreSQL · EBS PVC로 데이터 유지

EKS 기반 Persistent Volume으로 데이터를 안전하게 보존합니다.



Redis · 작업 큐 · 캐시 · Pub/Sub

작업 큐, 캐시, Pub/Sub 메시징으로 애플리케이션 성능을 향상합니다.



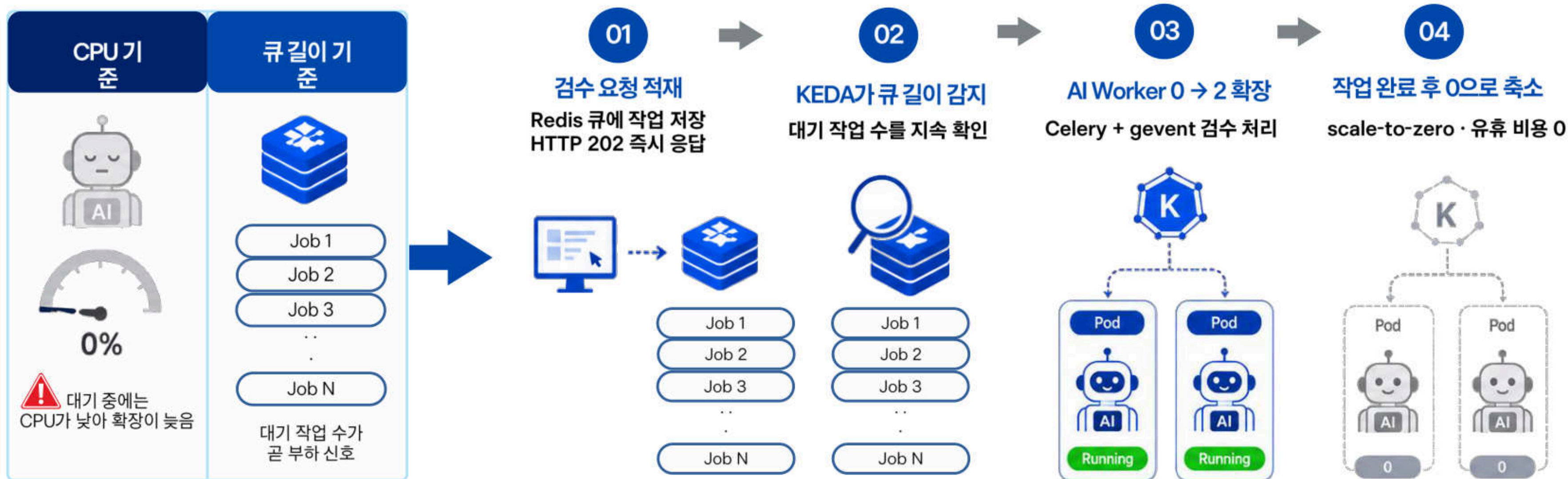
ALB + ACM · HTTPS 종료 · SSE idle 330초

ALB에서 HTTPS를 종료하고 SSE idle 타임아웃을 330초로 설정합니다.

관리 부담을 감수하는 대신, 학습 · 시연 규모에 맞는 비용 구조를 선택했습니다.

KEDA — Redis 큐 길이 기반 워커 오토스케일링

대기 워커는 CPU 신호가 낮다 — 이벤트 기반으로 큐 길이에 비례 확장



실측 확인 | 0 → 2 자동 증감 · 작업 종료 후 자동 축소 · 증설 중에도 기존 작업 정상 완료

CPU 사용량이 아니라, 실제 대기열을 기준으로 필요한 순간에만 워커를 늘립니다.

관측 경계(Observability Boundaries) 및 Alert 설계

알림 피로(Fatigue)를 줄이는 수집/제외 기준과 이벤트 기반 통보

**4개**

서비스별 로그 수집 경로

**14일**

로그 보존 기간

**19.0%**

워커 노드 메모리 사용률

**37%**

워커 노드 디스크 사용률



수집



- 서비스 로그: 검수 API · AI 워커 · 화면 서버 · 예약 작업
- 장애 구간을 서비스별로 구분
- 노드 지표: 메모리 · 디스크
- 서버 밖에서는 보이지 않는 값
→ 노드 에이전트 직접 설치



제외

- CPU: 클라우드 기본 지표로 제공 → 중복 수집 제외
- 플랫폼 자체 로그: 프로젝트 판단 근거가 없어 제외



Alert 설계

- 현재 경보(알림) 규칙은 미설정
- 임계치 · 이벤트 기준이 확정된 경우에만 통보 연동
- 알림이 온다고 말하지 않는다



Prometheus · Grafana를 쓰지 않은 이유

- 2노드 환경에서는 감시 도구가 감시 대상보다 더 많은 자원을 쓸 수 있음
- 워커 노드 메모리 19% : 여유 자원은 관측 스택보다 서비스에 우선 배분
- 14일 보존 : 2주 넘은 로그 실사용 사례가 없어 필요 시에만 확장

타임스탬프 동기화 결함 포스트모템(Postmortem)

컨테이너 UTC와 KST 혼재로 인한 기록 왜곡 → 시각 체계 전수 통일

01 증상 | 특정 시각에
로그 2.88K 건 급증

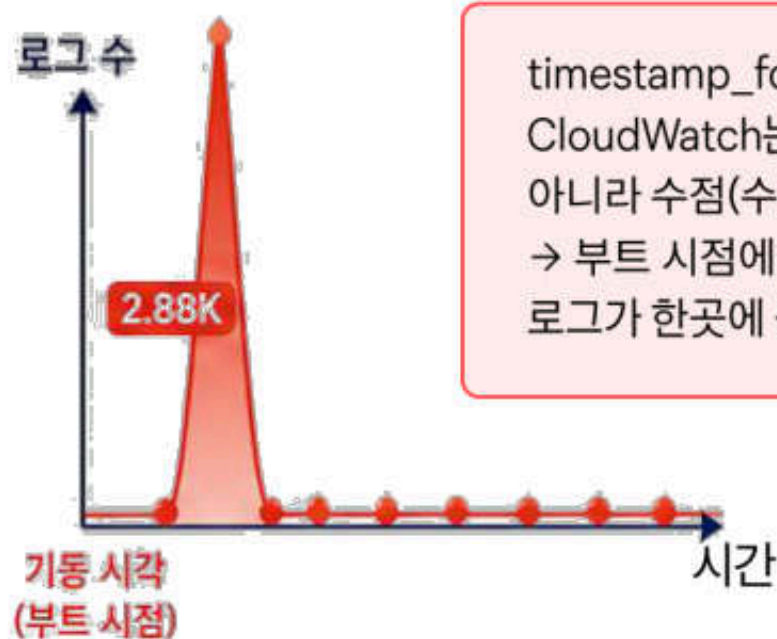
02 오해 | 장애 · 공격처럼 보였지만
실제 장애 없음

03 원인 | timestamp_format
미설정/ 수집 시각이 이벤트
시각으로 기록

04 조치 | 두 노드 시각 포맷 통일/
%Y-%m-%dT%H:%M:%S

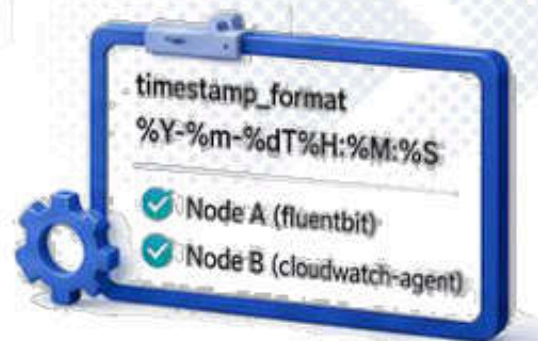


2.88K 로그 급증



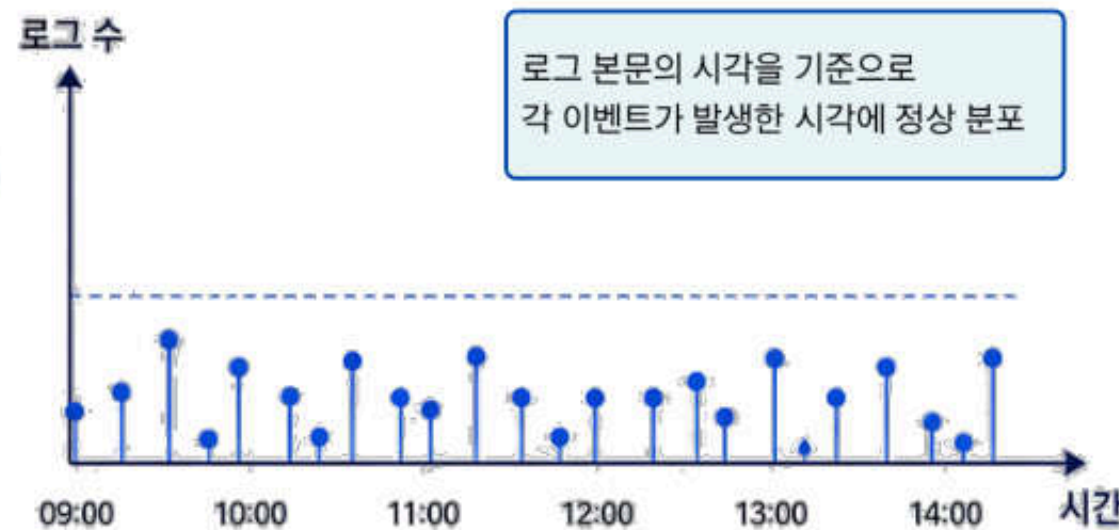
timestamp_format 미설정 시
CloudWatch는 로그 본문의 시각이
아니라 수집(수신) 시각으로 기록
→ 부트 시점에 백필(boot backfill)
로그가 한곳에 몰림

timestamp_format 지점



두 노드에 동일 포맷 명시

원래 이벤트 시각으로 복원



1) 그래프 봉우리 확인

먼저 수집 방식 · 백필 여부 점검



2) 접속 키는 서버에 두지 않음

마스터 경유 · ProxyJump로
로컬 키 사용



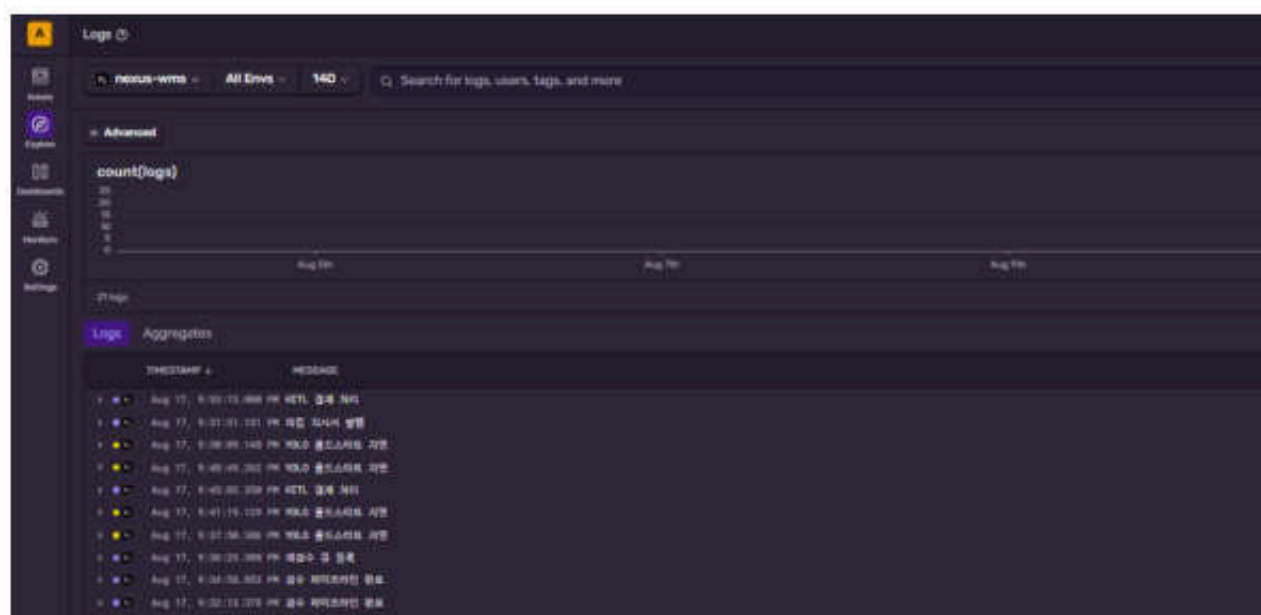
3) 설정은 문서에 전문 보존

설정 전문과 근거를 문서화

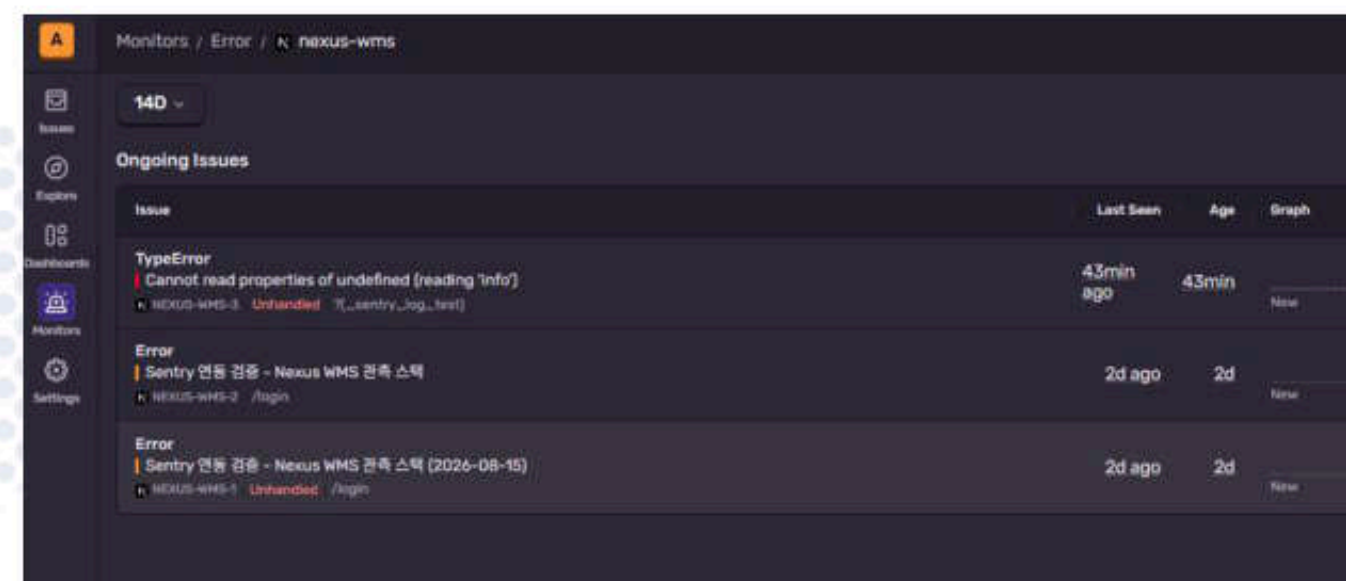
결론 | 로그 본문의 시각을 명시하자 각 줄이 원래 시각으로 복원되고 그래프가 정상화되었습니다.

Sentry 초기화 차단 3경로 — 설정과 런타임 동작 불일치

초기 상태는 '오류 없음'이 아니라 '수집 불능'일 수 있다 — 검증 이슈를 일부러 터뜨려 도달 확인



의도적 오류
발생으로 도달 확인



? 오류 없음? 수집 불능?

검증 이슈 2건 도달



서버 · 엣지 초기화

instrumentation.ts
register() 직접 import



클라이언트 초기화

instrumentation-client.ts
· Turbopack 경로



빌드 시점 주입

NEXT_PUBLIC_
DSN을 build arg 로 주입



추가 오류 경계 : global-error.tsx - 렌더 단계 오류도 수집

관측 도구는 이슈 0건이 아니라, 검증 오류가 실제로 도달하는지 확인합니다.

클라이언트 오류의 관측 사각지대와 프론트 계측

브라우저 런타임 에러는 서버 로그에 남지 않는다 — Sentry로 사각 해소

서버에서 보이는 것



API 오류 · 워커 실패 · 예약 작업
노드 메모리 · 디스크



브라우저 런타임 오류는
서버 로그에 없음

클라이언트 오류 자동 수집

Sentry로 보완하는 것



JavaScript 런타임 오류
3D 적재 뷰어 · 결합 표시 캔버스
사용자 기기 · 브라우저 정보



DSN 가드 | DSN 없으면 비활성 -
오류 없이 조용히 종료



tracesSampleRate | 운영 0.1 -
전량 수집 대신 10% 표본



조직 · 프로젝트 | azrale-vs /
nexus-wms - 소스맵 업로드 대상



둘 다 측정하지 않는 범위 : 이탈 사용자 · 실패하지 않은 느린 요청

오류 종류 · 발생화면 · URL · 브라우저 · Stack Trace를 함께 확보해 사용자 보고 전 원인을 확인합니다

CronJob 기반 재고 정합성 일일 자동 점검 배치

매일 새벽 배치가 재고·원장 정합을 대조 — 사람 개입 없는 상시 검증



01 매일 03시 실행

Kubernetes CronJob이
정시에 시작



02 재고·원장 대조

출고 현황 · 안전재고 ·
원장 데이터를 비교



03 발주 제안 생성

부족 수량과 제안 사유를
자동 산출



04 관리자 칸반 등록

검토 · 승인할 제안 카드로
등록



왜 새벽 03시인가

입출고가 적어 집계 중 변동을 줄이고,
업무 시작 전 결과를 준비합니다.



왜 CronJob 인가

앱 내부 스케줄러와 달리 파드가 늘어도
클러스터가 1회 실행을 보장합니다.



주간 인사이트도 같은 방식

Celery Beat 00:05 KST · 과거 주만
백필 · 진행 주는 갱신

정합성 확인부터 발주 제안 등록까지, 매일 자동으로 이어집니다.

보안 · 개인정보

S3 직접 접근 차단, 이미지 조회는 서명 URL로 통제

실측 결과를 기준으로 CDN 이미지 경로에 만료형 접근 권한 적용



실측 결과

S3 직링크

403

Block Public Access 정상

CloudFront

200

OAC 경유 · 158,819 bytes

DESIGN PRINCIPLES

- **Signed URL** 앱·CDN 도메인 분리로 쿠키 방식 미채택
- **응답 미들웨어** 여러 API의 이미지 URL을 한 곳에서 일괄 서명
- **원본 URL 유지** 만료형 URL은 DB에 저장하지 않고 응답 시 생성
- **키 미설정 시 no-op** 개발 환경에서는 기존 URL을 그대로 반환

구현·검증

S3 직접 접근 차단 확인 · OAC 경유 조회 · 응답·워커 서명 경로 구성

운영 전환

신뢰 키 그룹 활성화는 리허설 후 적용

6개 보안 헤더로 브라우저 공격 표면 축소

응답 단계에서 공통 적용해 전송·렌더링·권한 정책을 일괄 강제



보안 헤더 0종 → 6종

서버 종류를 노출하던 X-Powered-By 헤더 제거

ENFORCED RULES

● Strict-Transport-Security	HTTPS 접속 고정 평문 연결 전환 차단
● X-Frame-Options: DENY	외부 프레임 삽입 금지 클릭재킹 차단
● X-Content-Type-Options	콘텐츠 유형 추측 금지 업로드 파일의 스크립트 실행 방지
● Referrer-Policy	외부 전송 경로 제한 민감 경로·관리자 ID 노출 방지
● Permissions-Policy	장치 권한 제한 카메라·마이크·위치 접근 차단

단계적 적용 CSP는 차단보다 관측을 먼저 적용

Report-Only로 운영 빌드의 위반 항목을 수집 중이며, 확인된 인라인 스크립트 5건에 nonce를 적용해 차단 모드로 전환할 예정입니다.

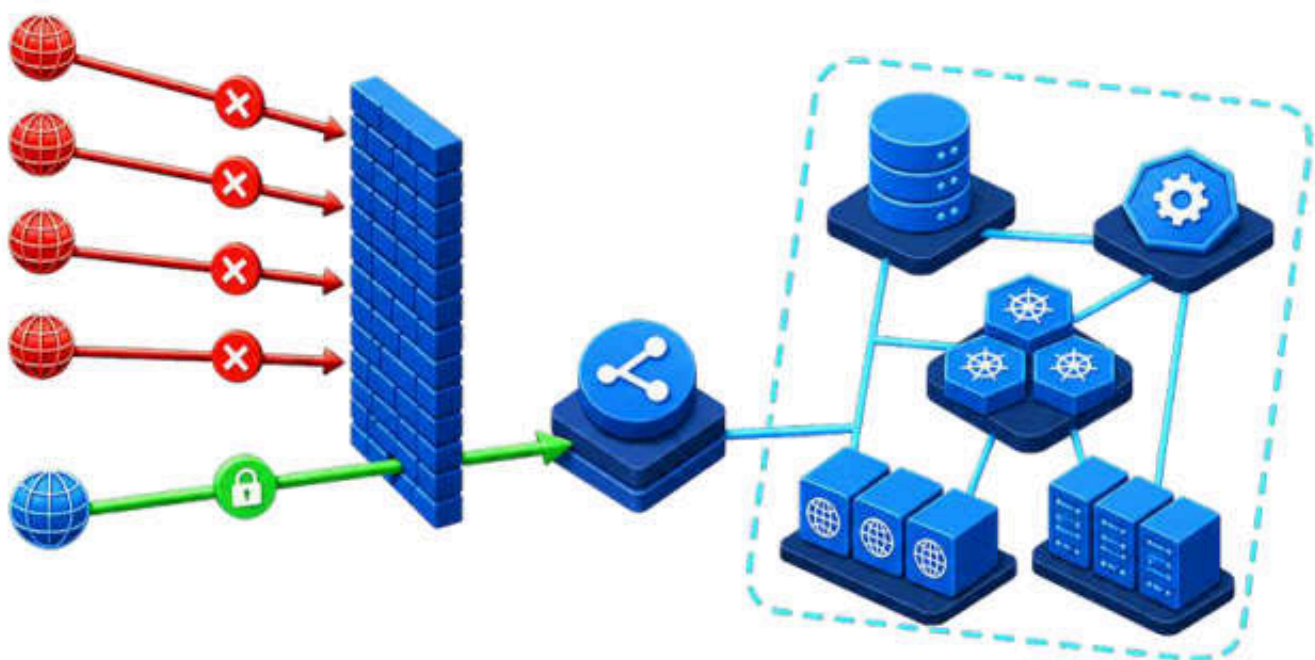
적용 지점

ALB 이후 화면 서버의 공통 응답 계층에서 헤더를 일괄 부여

엔드포인트별 누락 없이 동일 정책 유지

외부 진입 경로는 HTTPS 443 단일 포트로 제한

설정 확인이 아닌 실제 외부 접속 시험으로 내부 서비스 비공개 상태 검증



5개 경로 중 외부 허용 1개

인터넷에서 접근 가능한 경로는 ALB의 HTTPS 443만 유지

허용

443 · ALB(HTTPS) · 정상 진입 경로

차단

5432 · 8000 · 6443 · 30080/30081

VERIFIED PORTS

5432	PostgreSQL	차단	DB 인터페이스 외부 비공개
8000	WMS API · FastAPI	차단	클러스터 내부 통신 전용
6443	Kubernetes API	차단	클러스터 제어 경로 비노출
30080 / 30081	NodePort · Front/API	차단	내부 ALB 연동 전용
443	ALB · HTTPS	허용	유일한 외부 진입 경로

다중 방어 한 계층이 뚫려도 다음 계층에서 차단

네트워크 정책 → 전송 제어 → 브라우저 → 애플리케이션 → 데이터
보안 그룹만이 아닌 계층별 독립 통제로 구성

OS 방화벽 미사용

상위 계층에서 이미 차단하며,
중복 규칙으로 인한 운영 경로 불명확성을 방지

개인정보 권고 6개 항목, 적용 수준과 예외를 함께 기록

적용 여부만 표시하지 않고 보류·해당 없음의 판단 근거까지 대조

적용 3개 · 가이드 상회 1개 · 부분 적용 1개 · 해당 없음 1개



항목별 대조 결과

1. 수집·이용 동의

적용

가입 흐름에서
동의 절차 분리

2. 비밀번호 규칙

적용

KISA 권고 기준의
길이·조합 정책 반영

3. 표시 제한

적용

고지 내용과
마스킹 구현 일치

4. 해당 없음

판정

서비스 구조상
해당하지 않는 항목

5. 암호화

가이드
상회

전송 TLS·해시·
저장소 암호화

6. 접근통제

부분
적용

Edge·Server 이중 검증
일부 항목 보류

적용 완료 위험한 형식의 파일 업로드

확장자가 아닌 실제 파일 형식(Magic Byte)을 검증하고 압축 폭탄 공격까지 차단합니다. 전용 자동 테스트 34종으로 상시 회귀 확인합니다.

부분 방어 CSRF 방어 범위

SameSite 쿠키와 화면 서버 경유(BFF) 동일 출처 구조는 적용했습니다. 전면 방어 흐름은 도입하지 않았으므로 '완료'가 아닌 '부분 방어'로 기록합니다.

판정 원칙

'해당 없음'도 생략하지 않고, 서비스 구조상 적용되지 않는 이유를 근거와 함께 기록

관리자 판정 1건을 감사 로그와 학습 데이터로 연결

행동·상태 변화·등급·사유·좌표·소요 시간을 남겨 판정 복원과 모델 개선에 활용



RECORDED FIELDS

최종 조치	상태 변화	확정 등급	판정 사유	결함 좌표	판정 시간
승인·하향·반송·재호출	이전 상태 → 신규 상태	관리자 최종 품질 등급	결함 유형·ADMIN_RECALL	관리자가 수정한 위치	실제 검토 소요 시간

책임 추적 판정 복원

누가 언제 어떤 근거로 등급을 확정했는지 재구성하여 이의 제기와 사후 검증에 대응

모델 개선 정답 데이터

관리자가 수정한 결함 좌표와 최종 등급을 다음 모델의 학습·평가 데이터로 전환

같은 기록으로 통제와 개선을 동시에 충족하며, 반복 학습을 통해 HITL 개입 감소의 근거를 축적

5개 독립 계층에 순차 방어 적용

01	01 네트워크 AWS 보안 그룹	HTTPS 443만 개방 NodePort·DB·K8s API 외부 차단	서버로 직접 진입하는 경로 차단
02	02 전송 ALB · ACM	전 구간 HTTPS 암호화 HSTS로 평문 접속 차단	통신 도청과 중간자 공격 차단
03	03 브라우저 보안 응답 헤더 6종	클릭재킹·MIME 스니핑 차단 민감 경로의 외부 전송 제한	사용자 브라우저 내부 공격 차단
04	04 애플리케이션 인증 · 인가 · 요청 제한	JWT 기반 인증·권한 검증 역할별 권한 분리·요청 횟수 제한	권한 없는 기능 사용 차단
05	05 데이터 저장소 잠금 · 감사 기록	사진 저장소 직접 접근 차단 관리자 판정 이력 전건 보존	데이터 유출과 무기록 변경 차단

실측 확인	외부 포트 NodePort · K8s API · DB 차단	이미지 접근 직접 경로 403 · 허용 경로 200	브라우저 보안 헤더 6종 · 서버 정보 제거	통합 검증 보안 자동 테스트 51종 통과
-------	-------------------------------------	---------------------------------	-----------------------------	---------------------------

외부에서 열리는 경로는 HTTPS 443 하나뿐

다섯 우회 포트와 저장소 직접 접근을 실제 요청으로 검증

① 네트워크 — 열린 포트

\$ 외부에서 직접 접속 시도

30080 (프론트 NodePort) 차단
30081 (API NodePort) 차단
6443 (Kubernetes API) 차단
5432 (PostgreSQL) 차단
8000 (백엔드 직접) 차단

② 저장소 — 접근 경로

\$ 검수 사진 요청

저장소 직링크(S3) HTTP 403
CDN 경유(CloudFront) HTTP 200

→ 저장소는 직접 열 수 없고
CDN을 통해서만 열립니다

외부 요청

인터넷 사용자

→

HTTPS 443

ALB · 유일한 진입점

→

애플리케이션

인증·권한 검증

→

CDN 경유

이미지 조회

X

우회 경로 차단

NodePort · Kubernetes API · PostgreSQL · 백엔드 직접 접속 · S3 직접 접근

서버 주소나 저장소 경로를 알고 있어도 정식 진입점 외에는 열리지 않습니다

방화벽 설정값 검토가 아닌 외부 접속 시도의 HTTP·연결 응답으로 확인

역할에 따른 권한 분리

화면 · 기능	MASTER	ADMIN	WORKER	고객
관제 대시보드	○	○	×	×
AI 판정 결재 (HITL)	○	○	×	×
재고 관리	○	○	×	×
이상거래 탐지	○	○	×	×
라벨 출력	○	○	○	×
라벨 재발행	○	○	×	×
QR 품질 보증서	○	○	○	○

토큰 보호

브라우저 스크립트에서 분리

로그인 토큰을 JavaScript가 읽지 못하는 영역에 보관하여 악성 스크립트의 탈취 가능성을 제한

동일 출처

화면 서버가 API를 대신 호출

브라우저에는 같은 주소로 보이게 구성하여 다른 사이트에서 사용자 권한으로 요청하는 경로를 제한

- 엔드포인트 인가 자동 테스트 17종 통과
- 고객은 QR 품질 보증서 조회 가능, 개인정보 담지 않음

인가 전수 검사 - 조회 엔드포인트

검사 방식 - OpenAPI 스펙 기반 전수 프로빙

GET /openapi.json의 paths를 파싱해 전 엔드포인트를 열거하고, Authorization 헤더·세션 쿠키 없이 순차 호출한다.
스펙이 라우팅의 SSOT이므로 라우트를 추가하면 검사 대상에 자동 편입된다 - 검사 목록을 사람이 유지하지 않는다.

무인증 프로빙 - 401 Unauthorized

\$ 인증 컨텍스트 없이 전 경로 호출 (GET 45)

GET /inventory401

GET/orders/picking-...401

GET/po/proposals401

GET/inbound/history401

GET/notifications401

... 무인증 200 응답 0건

회귀 검증 - 정상 경로 무영향

\$ 세션 쿠키 부착 후 동일 경로 재호출

GET /inventory200

GET/orders/picking-...200

GET/po/proposals200

GET/inbound/history200

GET/notifications200

... .. 기능 회귀 0건

전수 프로빙이 검출한 인가 누락 도입 시점 인증 의존성이 없던 조회 라우트 17건을 검출해 전부 차단했다 (inventory · picking-instructions · po/proposals 계열)
차단(401)과 인증 후 정상 응답(200)을 동일 실행에서 함께 수집해 과차단 회귀를 배제했다.

정적 참조 탐색이 인가 여부를 증명하지 못하는 이유

참조 개수 ≠ 적용 범위 po 라우터는 RoleChecker 참조가 5건 잡혀 인가가 적용된 라우터로 보인다. 실제로는 5개 오퍼레이션 중 목록 조회 1건에만 Depends가 누락돼 있었고, 런타임 프로빙만이 검출했다	선언 상태 ≠ 런타임 동작 “Depends가 선언돼 있다”는 소스의 상태이고, “무인증 요청에 401이 반환됐다.” 는 런타임 관측이다. 배포본에서 그대로 재현 가능한 증거는 후자뿐이다.
--	---

조치 - 오퍼레이션 단위가 아닌 API Router 단위 적용 router = APIRouter(prefix='/inventory', dependencies=[Depends(get_current_user)])
오퍼레이션 단위 부착은 라우트 추가 시 누락이 재발한다. APIRouter의 dependencies는 하위 전 오퍼레이션에 전파되므로 기본적으로 “보호”가 된다.

읽는 법

- 검사기는 레포에 상주한다 (scripts/audit_endpoint_authz.py). 라우터 추가 시 실행을 절차로 규정했다.
- 이 장은 GET 45개 범위이다. 상태 변경 메서드(POST/PUT/PATCH/DELETE) 검사는 다음 장에서 다룬다.
- 무인증 허용 3건은 헬스 프로브 2개(/health · /db-check)와 /auth/password-policy이며, 어느 것도 도메인 데이터를 반환하지 않는다.

인가 전수 검사 - 상태 변경 메서드

HTTP 상태 코드를 사이드이펙트 프록시로 삼아 롤백 하네스 없이 검사했다

설계-HTTP 상태 코드를 사이드이펙트 프록시로 쓴다
FastAPI는 의존성(Depends)을 라우트 핸들러보다 먼저 해석한다. 따라서 401 응답은 핸들러 미실행, 즉 사이드이펙트 부재와 동치다.
트랜잭션 롤백 하네스를 구축하지 않고도 상태 변경 메서드를 안전하게 프로빙할 수 있다.

401 / 403
Depends 단계에서 차단
핸들러 미실행 - 사이드이펙트 없음

422
Pydantic 바디 검증에서 차단
인가 누락. 단 핸들러 미실행

2xx / 4xx 기타
핸들러 진입
인가 누락 + 사이드이펙트 가능

추가 안전장치 - 빈 바디({})를 전송해 필수 필드 검증에 선행 차단시키고, 실행 전후 12개 테이블의 row count를 대조해 변동 부재를 실측한다.

검사 결과 - GET 45 + 상태 변경 60 = 105 오퍼레이션 전수

항목	검사 전	조치 후
인증 요구 (401 / 403)	95	98
무인증 응답	8	5
핸들러 진입 (상태 변경)	3	2
row count 변동 (12 테이블)	-	0

**APIRouter
단위 인가 전파**
returns · admin/hitl 라우터에 dependencies를 선언해
하위 전 오퍼레이션에 인가를 전파시켰다.
users/issue만 오퍼레이션 단위로 부착했다 — 동일 라우터의
부트스트랩 엔드포인트는 무인증이어야 하기 때문이다.

**환경 게이트 실측
(APP_ENV)**
APP_ENV=prod POST /users/init-master → 403
APP_ENV=local POST /users/init-master → 400
부트스트랩-시연 복구 엔드포인트는 인증을 요구하면
존재 목적이 성립하지 않는다. 대신 _reject_in_prod()가
운영에서 호출을 차단하며, 그 발동을 실측했다.

읽는 법

- 과차단 회귀를 배제했다 — 관리자 세션으로 동일 경로가 200을 반환하는 것까지 같은 실행에서 수집했다
- row count 대조는 INSERT/DELETE만 검출한다. UPDATE는 카디널리티가 불변이라 잡히지 않으므로 무결성 증명으로 확대 해석하지 않는다
- write 모드는 로컬 스택 전용이다. 운영에 실행하면 인가가 없는 경로에서 실제 상태 변경이 발생한다

파일 업로드 보안 - 정책 서명과 격리 검사의 이중 통제

제약은 S3가 서명으로 강제하고, 검사는 격리 구역에서 실제 바이트로 수행한다

설계 - 업로드 지점과 열람 지점을 분리한다

브라우저 → quarantine/ (격리 구역) → 서버가 실제 바이트 검사 → attachments/ (정상 구역)으로 이동
검사를 통과하지 못한 파일은 정상 구역에 존재한 적이 없다. 미검증 파일이 노출되는 시간 창(window) 자체를 없앤 구조다.

1차 — S3가 서명으로 강제 (Presigned POST)

정책(policy)을 서버가 서명하므로 클라이언트가 고칠 수 없다

- content-length-range
5MB 초과 → S3가 업로드 자체를 거부
- Content-Type
선언한 타입 외 저장 불가
- starts-with \$key
quarantine/ 밖에는 쓸 수 없음

2차 — 서버가 실제 바이트로 검사

헤더·확장자는 위조된다. 파일 내용만 신뢰한다

- 매직바이트 · 휴미 페이로드
확장자 위조 · Polyglot · 실행파일(MZ/ELF)
- 컨테이너 · 능동 콘텐츠
VBA 매크로 · PDF JavaScript · 압축 내부 실행파일
- 압축비 · 픽셀 수 상한
5MB 안에 숨은 폭탄(200MB · 5천만 px) 차단

검증 결과 - 단위 95/95 · 라이브 S3 13/13 · 실제 업무 파일 695개 오탐 0

시도	막는 지점	실측 결과
5MB 초과 업로드 (개발자도구 우회 시도)	1차 · S3	EntityTooLarge 거부
확장자만 .png인 실행파일(MZ)	2차 · 서버	S3 통과 후 서버가 차단
JPEG 헤더 뒤에 ZIP을 붙인 Polyglot	2차 · 서버	차단
파일명 RTL override(U+202E) 위장	2차 · 서버	제어문자 제거 후 저장
정상 파일 6종 (jpg·png·webp·pdf·xlsx·zip)	-	전량 통과 (오탐 0)
서명 없는 직접 열람 · 만료된 열람 URL	열람 · S3	둘 다 403
PDF 내부 JavaScript · 자동 실행	2차 · 서버	차단
중첩 압축폭탄 · Zip Slip · 내부 실행파일	2차 · 서버	차단
남의 격리물을 승격 (IDOR)	2차 · 서버	403 — 소유자 결속

읽는 법

- 시그니처 기반이라 신중 악성코드는 못 잡는다 — 못 잡는 것을 전제로 격리해 피해 범위를 한정한 설계다
- Content-Type 위조는 S3가 403으로 막지 않는다. S3는 파일 파트 헤더를 무시하고 서명된 값을 저장하므로, 위조가 반영되지 않는 것으로 방어를 확인했다
- 첨부 전용 버킷을 분리해 검수 사진과 접근 정책 · 수명주기 · 사고 반경을 격리했다
- 열람도 퍼블릭이 아니다 — 만료 10분 Presigned GET을 계시물 단위로 일괄 발급한다
- 방어를 놓기 전에 먼저 돌려 봤다 — 12종이 통과했고, 막은 뒤 실제 업무 파일 695개로 오탐 0을 확인했다.

실험 · 연구

실험

결정론 엔진 검증 — 270회 반복 결과 동일성(Idempotency) 입증

동일 입력 270회 반복 실행에서 오차 0건 — 수학적 정합성 실측

270 회

반복 실행 · 전부 일치

181 건

UBCI 메트릭스 · 파라미터 조합

6/6

시드 2종 x 3회 스냅샷 일치

0 건

점수 · 등급 불일치



실험 통제 조건

- 결함 목록 · 심각도 · 좌표 고정
- 동일 입력 반복 투입
- 출력 점수 · 등급 직접 비교

검증 범위

- UBCI 점수 산출 함수 검증
- Vision 판독 · Report 서술 제외
- 점수 경로에서 LLM 미사용

재현 가능성

- 검증 스크립트 보존
- 실사 데이터로 재실행 가능
- 단일 스크린샷이 아닌 반복 증거

결과 해석

정확도 측정이 아니라, 점수 · 등급 산출에서 LLM이 분리되었음을 확인한 구조 검증

카오스 테스트 — 워커 강제 종료(Fault Injection) 유실 0% 입증

kill 주입 후 DLQ 이관·자동 재처리로 트랜잭션 무결성 검증 (4/4 완결)

01

검수 4건 시작

AI 검수 작업 동시 투입

02

워커 강제 종료

판정 진행 중 kill -9

03

복구 흐름 관찰

큐 반환·자동 재처리 추적

04

동일 조건 재실행

수정 전·후 각 2회

작업은 사라지지 않고 다른워커로 이동

강제 종료된 작업을 큐가 되돌려 보내고 정상워커가 이어서 처리



장애 복구 결과

4/4

전부 완료

0건

데이터 유실

8분

복구 대기

visibility_timeout

3600초 → 480초 · 최대 1시간 → 8분

acks_late = True

작업 완료 뒤 승인처리

reject_on_worker_lost = True

중단 작업을 즉시 큐로 반환

왜 설정부터 바꾸지 않고 장애를 먼저 재현했나

설정 파일만 보면 올바른 구성처럼 보여도 실제 복구 지연과 유실 위험은 숨어 있을 수 있습니다. 먼저 kill -9로 장애를 재현해 큐 반환과 재처리 흐름을 관찰한 뒤, 실측된 최대 1시간 지연을 8분으로 줄이는 설정을 반영했습니다. 수정 후에도 같은 실험을 두 차례씩 반복해 개선이 실제 동작하는지 확인했습니다.

라이브 성능 진단 — 원인 계층 분리와 당일(Same-day) 해소

운영 트래픽에서 간헐 지연을 실측 포착 — 가설 5개를 측정으로 소거하고 운영 설정 3건 + 자산 1건을 수정·검증

진단 — 가설을 측정으로 소거

프론트 렌더	동일 코드 로컬 대조 40ms — 기각
ALB·네트워크	파드 내부 호출도 21s — 기각
DB 쿼리	EXPLAIN ANALYZE 0.7ms — 기각
인프라(CPU·EBS)	크레딧 만땅 · gp3 — 기각
운영 설정·자산	1레플리카 · probe 1초 · 순수 파이썬 서명 · 로고 원본급 — 확정

읽기 전용 진단 도구 3종 신설 — 클러스터 실측 · 구간별 지연 분리 · 쿼리 실행계획

조치·검증 — 코드 로직 불변

로고 3종 표시 크기 리인코딩 + 정적 이미지 캐시 헤더
API 2레플리카 · probe 현실화 · 2워커 · 서명 OpenSSL 교체(정책·차단 불변)

	개선 전	개선 후
페이지당 이미지	1.66MB 매번	19KB → 캐시 0
502 오류	빈발	0건
HITL 응답	최대 26s	0.62s
API 평시	4~12s	60ms

전 항목 라이브 연속 측정으로 검증 · 무중단 배포 8회

같은 코드를 로컬 실측과 대조하면, 10분 안에 병목의 계층이 갈린다

제출 당일 운영 트래픽에서 — 실측 기반 정정 문화가 실전에서 작동한 기록

진단 → 해소 → 검증 당일 완결

실험

LLM 호출 구조와 Latency·Cost 실측 보고

노드별 토큰 계측(node_tokens) — 건당 \$0.0224~\$0.0575(결함 수 비례), 검수 중앙값 27초

추정이 아니라 실제 호출 경로에서 측정

측정기준 · 라이브 환경 · 실제촬영이미지 · 시연과 동일 경로



검수 처리 시간

27 초

중앙값

26~36초

이미지 업로드 → 최종판정

0건

카오스 테스트 2회 데이터유실

\$0.0224

1

결함 1건

결함수에 따라 증거 대조 호출이 늘어나는 실측 비용 범위

6

결함 6건

\$0.0575

단일 '건당비용' 대신 범위로 보고

결함수가 늘면 증거 대조와 재검증 호출도 증가 합니다.

그래서 이전의 '건당 약 원' 상한 추정은 폐기하고, node_tokens 실측값과 호출조건을 함께 공개합니다.

자체 역량 평가 — 기술 부채(Tech Debt) 및 한계 분석

미도입 기술과 잔존 부채를 명시 — 아키텍처에 대한 메타인지 입증

운영 가능성은 증명했고, 남은 부채는 숨기지 않았습니다

검증된 역량 3개

01 실배포가 살아 있다

데모가 아닌 실제 도메인 · HTTPS 서비스 운영 · 심사 중에도 접속 가능

02 판단에 근거가 있다

기술 선택 이유를 문서화 · 유행이 아니라 운영 환경과 제약을 기준으로 선택

03 금전 판단에서 LLM을 제외했다

가격·등급은 결정론산식 · 270회 재현검증 · 감사가능한구조

VS

잔존 기술 부채 3개

01 테스트 커버리지

확인 커버리지 38% · 모든 조합을 아직 검증하지 못함 · 가장 약한 지점으로 명시

02 모델 절대 성능

Recall 0.560 — 높은 수치가 아님 · 현재 보완 중 · 모델 자체가 좋다고 말하지 않음

03 재학습 실행 자동화

오탐 제외·학습셋 추출은 구현 · 재학습·가중치 배포 단계는 수동 · '루프완성'으로 과장 하지 않음

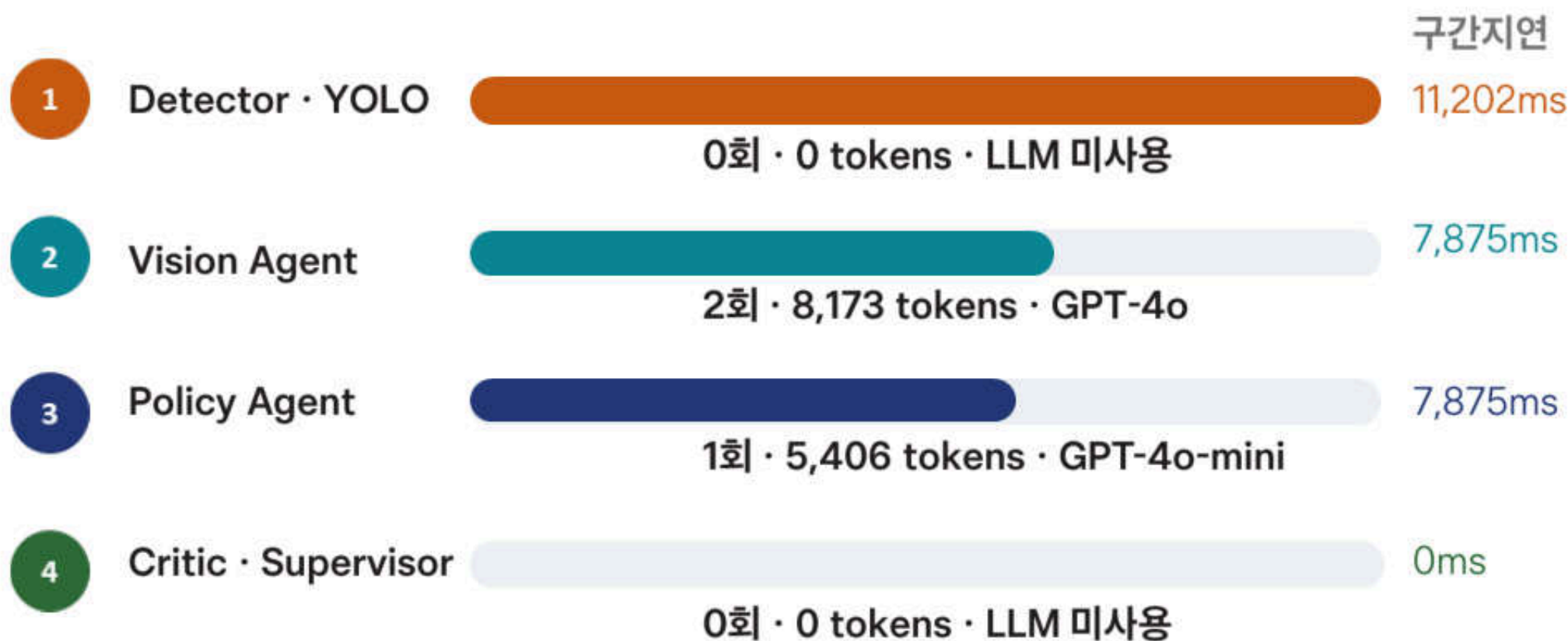
자체 평가 결론

운영·판정·비용 통제는 실측으로 입증 했습니다. 반면 테스트범위, 모델 성능, 재학습 자동화는 다음 개선과제로 남겨두고 과장없이 공개합니다.

실험

테스트 ① 파이프라인 구간 계측

LangGraph 노드별 처리지연·LLM토큰·비용측정



병목은 LLM 아닌 Detector

가장 긴 구간 11.2초 — 최적화 우선순위를 탐지 단계에 배치

실측으로 바뀐 판단

11.3초

최대 구간

3회

검수 1건당 LLM 호출

13,579

토큰 · 결함

\$0.0224

건당 비용

Vision 8,173 + Policy 5,406

노드별 토큰귀속으로 '4개 Agent 모두 LLM 사용' 가정해소

계측이 만든 세 가지 변화

1 병목 인식 수정

YOLO Detector가 최대 지연구간

2 토큰·비용 귀속

contextvar로 호출 노드까지 추적

3 최적화 근거 확보

추정 비용을 노드별 실측으로 전환

실험

테스트 ② 불필요 LLM 호출 검출 및 제거

계측으로 드러난 호출의 산출값이 실제로 쓰이는지 추적

호출 경로 검사

제거 전



제거 후



판정은 유지하고 계산 낭비만 제거

판정 동일

HITL_REQUIRED · UBCI 100 · AWAITING · 결함 1건

동일 작업 재검수로 제거전·후 결과 대조

4회 → 3회

LLM 호출 1회 제거

-17.9%

16,547 → 13,579 tokens

-2.1%

\$0.02284 → \$0.02236

0건

판정 변화

검증근거

① 소비처 정적 추적

policy.py 참조 0건 · 스키마 참조 3곳

② 프롬프트·코드 대조

감점규칙이 기존 조건문과 1:1 대응

③ 동일 Job 재검증

점수 산식은 유지 · 호출만 제거

실험

테스트 ③ 동시성 부하 시험

동일 자원 경합에서 데이터 정합성과 분산 락이 유지 되는지 검증

A. 동일 ISBN 재고경합

POST /inbound/fasttrack N건 동시 · lost update 검증 · LLM 미사용

B. return_job_id 중복실행 · LLM 1건분

서로다른 1,000건 처리 ≠ 동시성경합

처리량 시험·GPT-4o 약 \$36 / 기존 4건 근거대비 250배 규모

같은자원에 Barrier로 동시에 진입

inventory.quantity · 중복 실행을 직접 대조

한 자원에 요청 집중



조치 전 · 신규 ISBN 20건 동시

HTTP 200 6건, HTTP 500 14건

IntegrityError · ix_books_isbn 위반

조치 후 · 동일재고 1,000건 동시

HTTP 200 1,000건, 재고증가 +1,000

lost update 0건 · 37.3초

70% → 0%

A축조치 전 실패율 → 조치 후

1,000 / 1,000

A축 전건 성공

0건

lost update

14건 차단

B축 15회 중 실행 1회

문제 · check-then-insert 경합

여러 요청 이동시에 '없음'을 확인 → 모두 insert 시도 → unique 충돌이 HTTP 500으로 노출

개선 · 제약조건을 동시성 장치로

기존 unique 제약을 최종 방어선으로 사용하고, 충돌을 정상 제어 흐름으로 처리

확인 · 분산 락 정상

같은 작업 15회 동시요청 → 실행 1회 · 차단 14회 · 중복실행없음

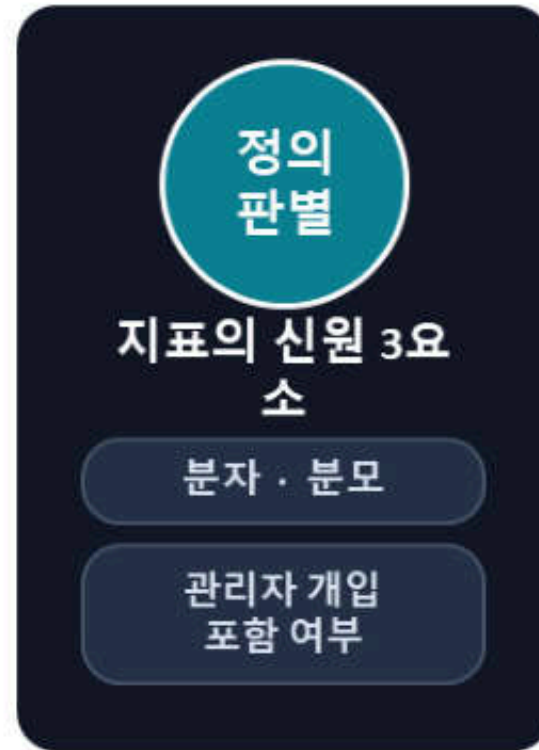
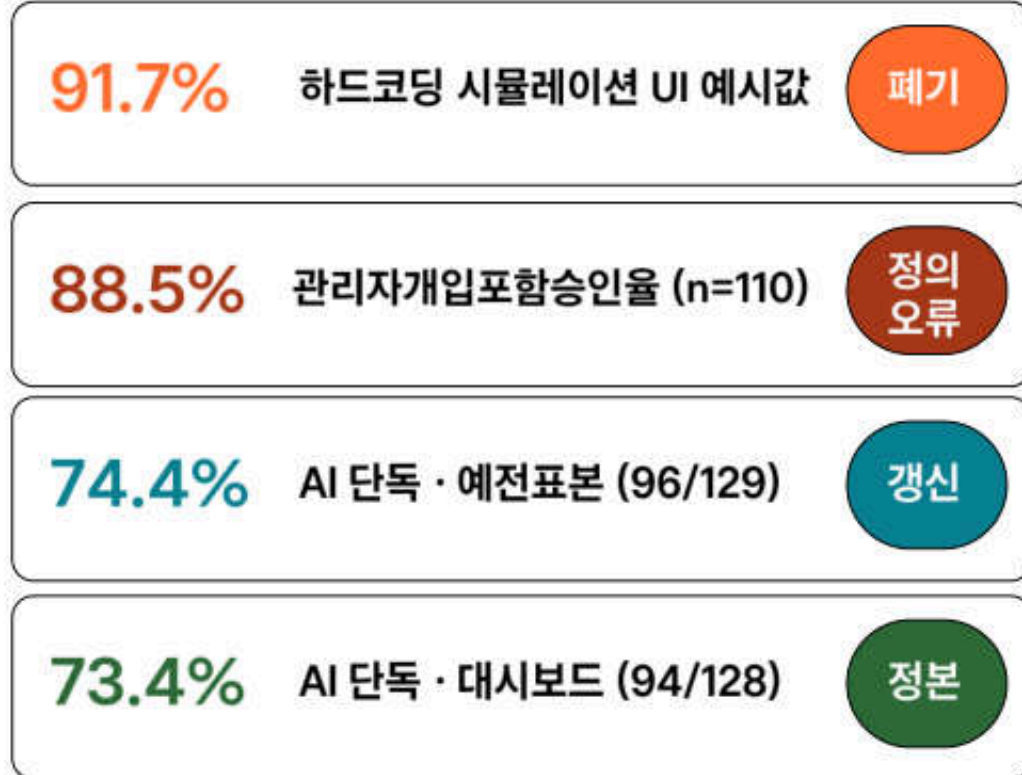
테스트 ④ 지표 정의 전수 검사

문서 전체에서 같은 지표가 같은값·같은 정의로 쓰이는지 대조

정의 동일지표명이 가리키는 값·산식·관리자 개입범위를 전 문서에서 대조
총괄·총정리 문서가 서로 다른 시점에 작성되어 정합성 확인 필요

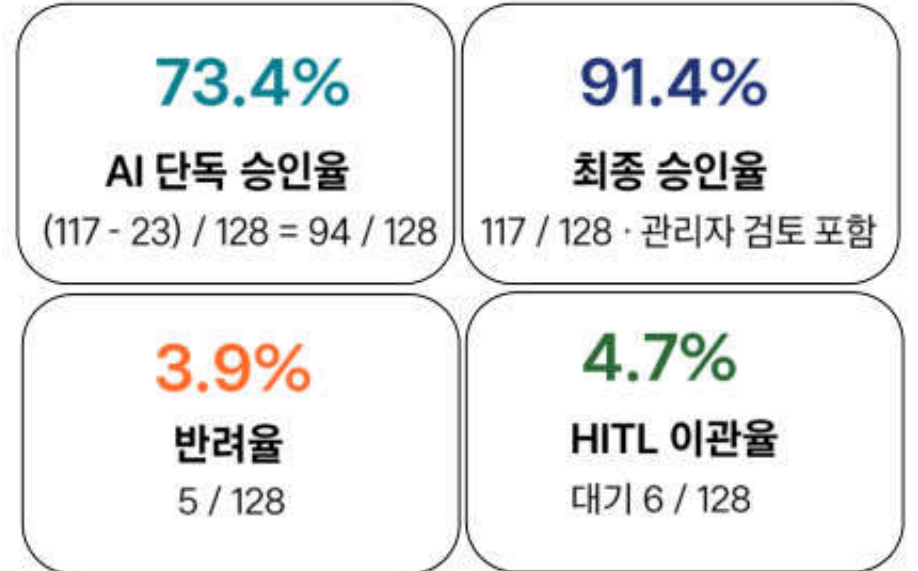
이유 화면과 문서의 수치가 다르면 심사 대조 순간 신뢰 상실

문서에서 발견된 같은이름의 네 값



대시보드표시값 = SSOT

같은 산식으로 라이브 DB 재산출 후 문서통일



검색 결과 '자동승인율'이라는 한 이름 아래 네 값이 흩어져 있었고, 88.5%와 74.4%는 서로다른대상을측정
해결: 대시보드를 정본으로 확정하고, 같은 산식으로 라이브DB를 재산출해 값과 정의를 통일

문제 · 지표명이 정의를 담지 못함

- '자동 승인율'만으로 관리자개입포함여부구분불가
- 같은 이름으로 서로다른값을측정

개선 · 분자·분모와 SSOT 명시

- AI 단독 승인율(94/128)처럼 산식 표기
- 화면값을 정본으로 삼고문서가 다르면 문서 수정

처리범위 · 이력보존

- 현행 문서 15개 값 교체
- 이력 문서 5개 본문 보존 + 시점 배너
- 과거 판단을고치면 이력이 거짓이 됨

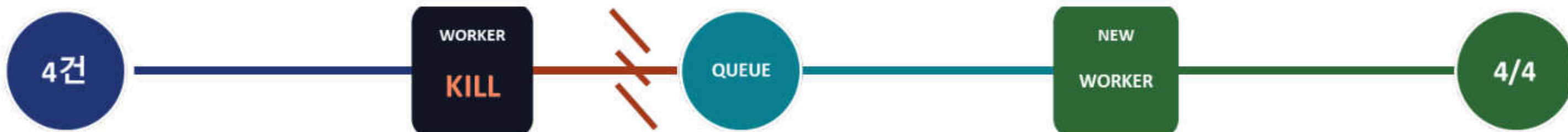
장애 주입(Fault Injection) 실험 체계 — 결함 5건 수정 이력

v1~v4 반복 실험으로 결함발견·수정 후 동일 규모 재측정 — 유실 0 도달



작업이 사라지지 않는 회복 경로

작업완료 확인 전에는 큐에서 제거하지 않고, 새 워커가 이어받는 구조



AI 검수 시작
정상 처리 중

강제 종료 x2
정상 동작이 아닌 수동 중단

작업 보존
완료 신호 전에는 큐에 유지

새 워커 인계
남은 작업을 이어서 처리

전건 재처리
완료 · 유실 0건

2회

검수 도중 워커 강제 종료

4/4건

중단됐던 작업 전부 재처리 완료

0건

데이터 유실

1

완료 후 ACK

작업 완료 신호를 받은 뒤에만 큐에서 제거하는 설계 검증

2

실패 작업의 격리

재시도 한계를 넘긴 작업은 실패 보관함으로 이동해 재처리 가능

3

프로세스 장애 독립성

한 워커가 종료돼도 대기작업과 처리 이력이 유지되는 구조 확인

보류 · 미채택 검토안

YAGNI 원칙 — 도입 기각(Rejected) 기술 및 Trade-off 분석

유행이 아니라 현장 제약(오프라인·장갑·기기사양) 기준으로 기각

판단 기준 | 현장 제약·운영 규모·비용 대비 효과

현재 구조와 규모에서 추가 도입의 실익이 낮은 기술

REJECTED

01 Nginx · Ingress 역할이 이미 나뉘어 있다

부하 분산·화면 제공·요청 제한의 역할이 기존 서버에 이미 분리

02 Prometheus · Grafana 역할이 이미 나뉘어 있다

서버 2대 환경에서는 관측 도구의 자원 부담이 관측 효과보다 큼

03 Supabase 역할이 이미 나뉘어 있다

인프라 구축 실패에 대비한 대체안으로 준비했으나 자체 구축 성공으로 미사용

04 침입 차단 서비스(WAF) 역할이 이미 나뉘어 있다

현재 트래픽 규모에서는 방어 효과보다 고정비 부담이 큼

구현 상태 또는 우선순위가 남아 있는 보안 과제

PENDING

01 CSP 차단 모드 전환

Report-Only 적용 완료 · 잔여 위반 5건에 nonce 적용 후 전환

02 사진 진열람 서명 강제

코드 배포 완료 · 시연 리허설 후 활성화 예정

03 서버 간 통신 제한

단일 구역·소규모 구성으로 우선순위를 낮춰 미적용

04 세션 유효기간 · 로그인 캡처

개인정보보호 권고 중 남아 있는 접근통제 보완 항목

남은 항목은 일정 · 운영 조건을 확인한 뒤 순차 반영

이 장을 넣은 이유

- “완벽하게 막았다”고 말하지 않기 위해서입니다.
계층별로 무엇을 막았고 무엇이 남았는지 구분해 말하는 편이 더 신뢰를 얻는다고 판단했습니다.
- 안 쓴 것에도 이유가 있다면, 그것은 빼뜨린 것이 아니라 현장 조건에 맞춰 선택한 것입니다.
필요해지는 시점에는 다시 검토하되, 현재 규모의 운영 조건에서 불필요한 복잡도는 먼저 만들지 않았습니다.

아키텍처 롤백(Rollback) 사례 — 실측 기반 철회 기록

도입 후 전제가 실측으로 깨진 설계를 되돌린 결정 근거

01 MINT 등급 지름길

제거

처음 설계

흠 없는 책은 검수 단계를 건너뛰고
바로 매입 승인하도록 구성

실측에서 깨진 전제

건너뛴 단계는 비용 절감 구간이 아니었고,
AI 실패가 '흠 없음'으로 읽히면
검증 없이 매입되는 험이 발생

결정 · 안전성 우선으로 제거

02 마모 단독 판정 게이트

도입 후 철회

처음 설계

감점 근거가 '마모' 뿐이면
사람에게 넘기도록 규칙화

실측에서 깨진 전제

근거 표본은 중복 사진이었고 실제 촬영은 20건,
다른 결함 유형까지 잡는다는 전제도
검증된 적이 없었음

결정 · 근거 재검증 전 철회

03 마모 후보 전건 채택

기능은 두되 OFF

처음 설계

탐지 모델이 찾은 마모를 검증 없이
전부 결함으로 등록

실측에서 깨진 전제

도서 1권에서 세운 결론이 3권으로 넓히자
뒤집힘 · 탐지 모델 재학습이 먼저 필요

결정 · 코드 유지, 기본값 OFF

코드가 틀린 것이 아니라, 실측으로 깨진 전제로 되돌렸습니다.

아직 못 한 것 - 숨기지 않습니다

CSP 차단 모드 전환

Report-Only 적용 · 남은 위반 5건
nonce 보완 후 전환

사진 열람 서명 강제

R코드 배포 완료 · 시연 리허설 이후 적용
예정

서버 간 통신 제한

단일 구역 · 소규모 환경 기준으로 우선순위
조정

세션 유효기간 · 로그인 캡처

개인정보 권고 중 접근 통제 잔여 항목

기록을 남기는 이유 | 완료·보류·미도입을 구분해, 같은 제안을 다시 실험하지 않아도 판단 근거가 남도록 합니다.

기술 선정 Trade-off — 검토 후 미채택 결정 근거

WebRTC · WASM 등 검토기술의 미채택 사유 명시 — 오버 엔지니어링 배제

01 Hugging Face 모델 서빙

모델 호스팅

자체 추론 유지

워커 안에서 직접 추론하는 편이
지연이 짧고 별도 비용도 들지 않습니다.

02 WebRTC

실시간 통신

사진 업로드 유지

검수는 한 장씩 찍어 올리면 되므로
스트리밍과 중계 서버가 필요 없습니다.

03 WebAssembly

브라우저 추론

서버 추론 유지

3모델 앙상블 가중치와 현장 태블릿
성능을 고려하면 서버 추론이 적합합니다.

선정 기준

● 현재 규모 · 일정 안에서 감당되는가

● 기존 구성으로 같은 목적을 달성하는가

● 도입 후 더 나빠지는 점이 없는가

개발 과정

AI 판독 오류 유형별 누적 분석 및 재반영(Feedback) 체계

실패 사례를 패턴화해 다음 라벨링·프롬프트 전략에 반영하는 사이클



01 오류 사례 누적

수치·기능·스택·판단 오류를
사례 ID와 함께 유형별 저장



02 반복 패턴 도출

서로 다른 사례에서 반복되는
공통 오독·추론 방식을 연결



03 근거 규칙 재정의

양성 기록 필드를 기준으로
라벨링 판단 가능 범위를 고정



04 다음 전략에 재반

영벨·프롬프트·검수 기준을
업데이트하고 다시 누적

누적된 실제 오류 6건

A-1 자동 승인율	94.6로 작성했으나 실제 74.4% - 로컬 시드 데이터로 산출	A-4 합성데이터 분류	AI 기능으로 썼으나 학습 데이터 확보 수단 - 기능 개수 12 → 10 정정
A-8 미배포 스택	Prometheus-Grafana를 도해에 포함 - 실제 배포 이력 없음	A-11 도넛 합계 103.9%	분류 축이 섞인 것을 놓치고 다른 문제를 지적
A-12 'AI가 스스로 넘김'	감사 로그가 없어 구분 불가인데 'AI 이관'으로 추정	A-13 'PostgreSQL에 없다'	존재하지 않는다고 단정 - 실제 YAML-ChromaDB에 존재

반복된 패턴 - 서로 다른 세 사례가 같은 실수

A-7 · A-12 · A-13은 모두
'특정 저장소에 없음'을 '존재하지 않음'으로 읽었습니다
감사 로그가 없으니 AI가 넘겼다, PostgreSQL에 없으니
그 테이블은 가상이다 - 모두 소거법 기반 추론입니다.

재라벨링 규칙

원인은 양성 기록 필드가 있을 때만 단점.
없으면 '구분할 수 없다'로 기록하고 소거법
추론 금지.

오류를 '개별 수정'으로 끝내지 않고, 패턴 → 근거 규칙 → 라벨링 · 프롬프트 업데이트로 닫힌 피드백 루프를 만듭니다.

"추정(Estimation)"과 "실측(Measurement)"의 엄격한 분리 원칙

측정한 지표만 인용하고 추정치는 추정임을 명시 — 수치 신뢰성 확보



ESTIMATION

추정치는 '사실'처럼 인용하지 않는다

- 계산·예측·소거법으로 얻은 값
- 근거와 가정을 함께 명시
- 실측값과 같은 열·문장에 혼합하지 않음



MEASUREMENT

인용 가능한 수치는 측정 근거가 있어야 한다

- 실제·호출·조회·센서·로그로 확인
- 출처와 측정 시점을 함께 기록
- 재현 가능한 값만 성과 지표로 사용

실측과 추정을 분리하기 위한 6가지 검증 규칙

01 권한 검증 규칙

- 조회 API로 권한을 테스트하지 않음
- 그 주체가 실제로 호출하는 API로 확인
- ListMetrics가 아니라 PutMetricData

02 아키텍처 도해 규칙

- 구성요소는 kubectl get으로 확인
- 존재를 증명하는 것만 그린다
- 계획 중인 것은 넣지 않는다

03 레이아웃 판단 규칙

- 워드 텍스트 목록으로 화면 배치를 추정하지 않는다
- 반드시 렌더 결과를 본다

04 산출물 판단 규칙

- '실제와 다르다'고 판단하기 전에
- 그 산출물이 스스로 밝힌 형태·출처를 먼저 읽는다

05 수치 인용 규칙

- 달성 불가능한 대조 지표를 나란히 적지 않는다
- 정부 수치를 하나로 통일

06 일괄 처리 규칙

- 기술 스택 표기를 고칠 때
- 그 스택이 적힌 모든 산출물을 나열
- 한 번에 일괄 처리

수치의 출처·측정 방식·시점을 확인할 수 없다면 '실측'이 아닙니다 - 추정값은 반드시 추정으로 표시합니다.

코드-문서 1:1 동기화 체계 (Documentation as Code)

아키텍처 변경 시 코드와 명세서를 한 묶음으로(원자적, atomic) 동시 갱신

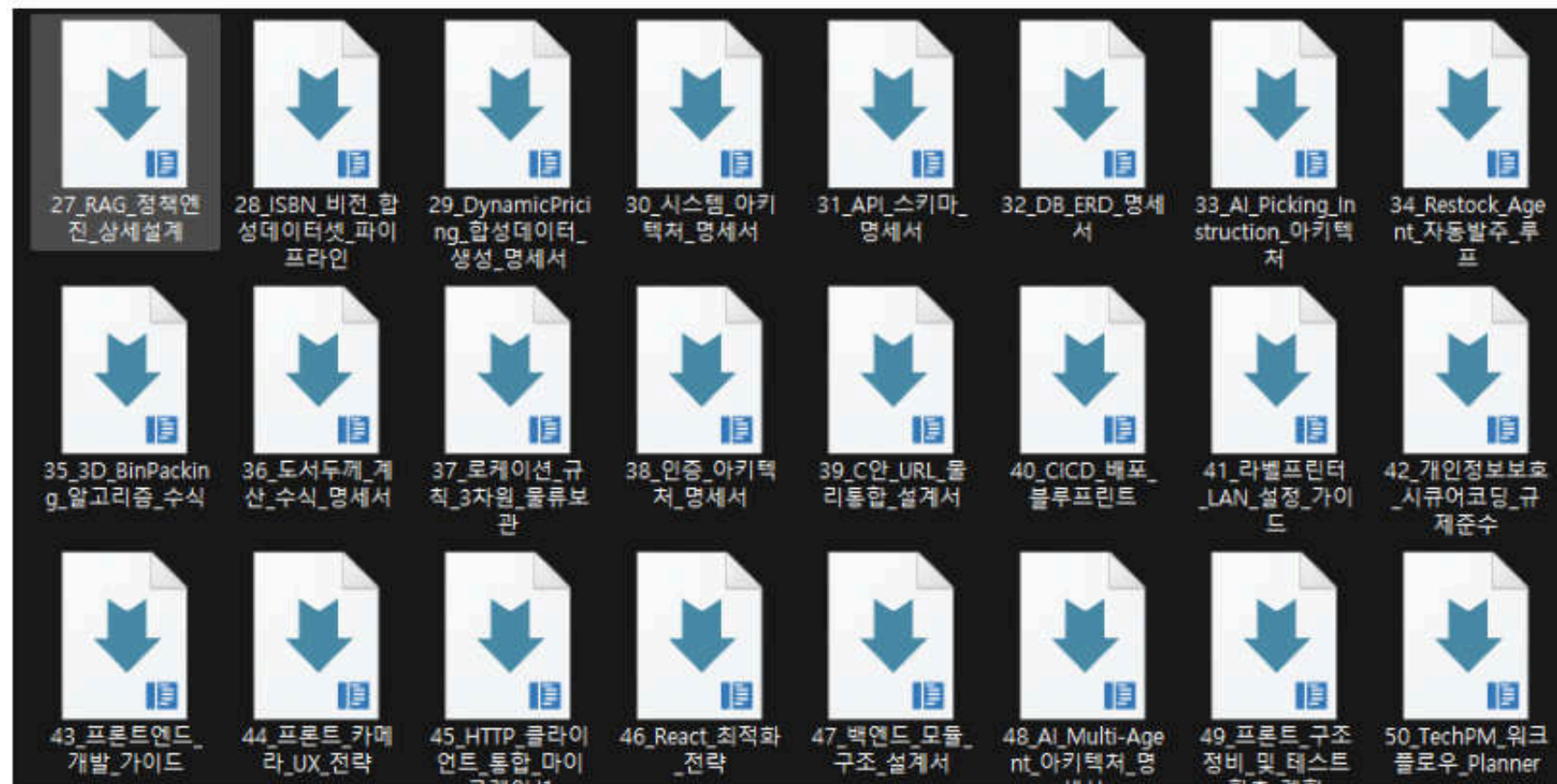
문서 레지 스트리

번호대만 보아도 문서의 역할과 책임 범위를 찾는 구조

00-09	총괄·업무논리·워크플로우 프로젝트 총괄·핵심데이터·QA	40-49	인프라·Frontend·Backend CI/CD·모듈 구조·최적화 전략
10-19	발표·영상·검증 기록 발표설명·촬영시나리오·전수검토	50-69	검증·실험·평가 기록 테스트·오류 정정·약점 평가
20-29	AI 모델·알고리즘명세 YOLO 학습이력·UBCI 산식·RAG	70-74	학습·보완·발표 자료 CloudWatch·Sentry·CloudFront·보안점검
30-39	시스템·API·DB·인증 아키텍처·ERD·인증설계	90-91	아카이브 코드변경 이력·최종 설명 기록

✓ 원자적(atomic) 동시갱신

코드변경과 명세 갱신을 같은 작업단위로 묶고,
실패·기각 가설까지 검증결과와함께보존



한 번의변경 = 코드 + 명세 + 검증기록

문서만 뒤쳐지거나 코드만 먼저 배포되는 불일치 차단

문서에 반드시 남기는 다섯 가지

01 목적·범위

무엇을 바꾸는가

02 설계 이유

기각 대안 포함

03 관련 경로

파일·Endpoint

04 검증 기록

방법·결과

05 미완·한계

남은과제

요구사항 추적 체계 — 정의부터 배포 검증까지

WMS0101001 = WMS(코어) + 01(재고/입출고) + 01(LPN 바코드 관리) + 001(일련번호)

Seq	요구사항 ID nn(주)=nn(부)+nn(소)+ mm(보안번호)	과업종류	작업범위		요구질적형	고객 요구사항 상세 내용	보유사양원천분류		중요도	수행 여부	요구사항건거	요구관리 상태	
			발무(대)	기능(소)			구현(소)	1차					2차
1	WM0101001	조교화	WMS 조커	재고/입출고 관리	LIFO 재고조 관리	인출 추적 LIFO 재고조 처리	상장상품 계획기 대신 대칭 인위 LIFO를 적용하고 입고-출고-인출을 끝까지 추적한다.	W/W보구서방	가능	양	수용	배회 검증	완료
2	WM0101002	연월유지	WMS 조커	재고/입출고 관리	코킹잔량 및 30차	UBC 기준 잔입신입(FIFO by UBC)	동일 상품 출고 시 상하 통급하고 입고 순서를 반역 그라워 출고 대상을 회전한다	W/W보구서방	가능	양	수용	배회 검증	완료
3	WM0101003	조교화	WMS 조커	재고/입출고 관리	정확한물 및 30차	3D 적체 최적화 및 속도 추는	Extreme Point + Best-path Crossing 지체 구분으로 18톤 픽스 중 최적 픽스속 배치 과로율 도출	W/W보구서방	가능	양	수용	배회 검증	완료
4	WM0101004	연월유지	WMS 조커	재고/입출고 관리	출발할 차를 분기	출발할 차를 분기 처리	UBC 기준 차량 시 속도 결정 없이 먼저 검토후 차를 분기한다	W/W보구서방	가능	양	수용	배회 검증	완료
5	WM0111001	선규	WMS 조커	재고/입출고 관리	후계타관 자동 배	후계타관 3차별 배합	통급률 큰(MMT+에 GOOD+C-NORMAL+O)로 개제조건별 하중 규칙적으로 배합한다	W/W보구서방	가능	중	수용	배회 검증	완료
6	WM0111004	선규	WMS 조커	출고 관리	회합/승용 처리	회합 지시기 선택 및	운전에서 회합 지시서를 생성하고 인출은 LIFO를 접근하는 LIFO로 처리 존 검증한다	W/W보구서방	가능	양	수용	배회 검증	완료
7	WM0111002	선규	WMS 조커	출고 관리	회합/승용 처리	회합 발송 및 모든 콘트 처리	규칙 기반 송금순위를 개선하고 작업자 정보 완료 시스템 재고를 보강 지급한다	W/W보구서방	가능	양	수용	배회 검증	완료
8	AJ0204001	선규	AI 에이전트	비전 AI 검증	LangGraph, 에이전트	LangGraph, 멀티 에이전트 다의로	Detection-Vision-Policy-Critic-Supervisor-Agent 구조로 통합으로 실행 지령 추진	W/W보구서방	가능	양	수용	배회 검증	완료
9	AJ0204002	연월유지	AI 에이전트	비전 AI 검증	LangGraph, 에이전트	SARL 시뮬레이션 및 UBC 접근 가능	결함 정보를 처리 시 연속으로 11 대칭을 저장하고 요청해 피아워처리한다	W/W보구서방	가능	양	수용	배회 검증	완료
10	AJ0208001	조교화	AI 에이전트	비전 AI 검증	영상촬영 촬영 장치	WER 3-Model 영상촬영 촬영 장치	제한을점명도나서 전달 3회촬영 신호로 자동 종료 detect 37.9%~84.1%	W/W보구서방	가능	양	수용	배회 검증	완료
11	AJ0208002	선규	AI 에이전트	비전 AI 검증	영상촬영 촬영 장치	3-track 연속 분석	학습된 연(표기 정보와) 학습된 영상정보 학습 데이터 없는 현재까지 유지되는 37.9% 연속	W/W보구서방	가능	양	수용	배회 검증	완료
12	AJ0208004	선규	AI 에이전트	비전 AI 검증	SAD 지시제어	정책 기반 제어 지원 수단	ChromeOS 170종까지 근무를 검색해 정책 수행 후배변수 부합을 확인 지원하는 개발자가 있음	W/W보구서방	가능	양	수용	배회 검증	완료
13	AJ0208005	선규	AI 에이전트	비전 AI 검증	SAD 지시제어	환경 근거 운영 방법 설계	차서 정책 범용한 환경 근거로 제작, 차서 객체인 실제 환경으로부터만 사용	W/W보구서방	보완성	양	수용	배회 검증	완료
14	AJ0204003	선규	AI 에이전트	비전 AI 검증	모집 산자 SDOF	검합 산자 SDOF 자료 포함 진행	검합 수치를 전달 당과 과탐과 후고 각 입수가 정확히 소망 변위로 리프트 보충	W/W보구서방	가능	양	수용	배회 검증	완료
15	AJ0207001	선규	AI 에이전트	비즈니스 AI	통학 거각 입력	통학 거각 입력 1000000	복합기 기술 지원실 참여수준 기준을 완성해 안정하고 1차 도입효과도 반영 할것 일련으로 서팅가치를 실제 용도로	W/W보구서방	가능	양	수용	배회 검증	완료
16	AJ0207005	선규	AI 에이전트	비즈니스 AI	배회 자동 발주	배회 자동 발주 2000000000	발달 수준 속도와 지도알고리즘으로 문제를 예측해 선제적으로 발주율을 향상	W/W보구서방	가능	중	수용	배회 검증	완료
17	AJ0207004	선규	AI 에이전트	비즈니스 AI	이슈가해 탐지	이슈가해 탐지(SOC)	규칙 기반으로 오류를 줄이고 1차 2차에 최화된게 사유를 자율 진단 관리자까지 확장	W/W보구서방	가능	중	수용	배회 검증	완료
18	AJ0208001	선규	AI 에이전트	비전 AI 검증	다중 채널 비전	다중 채널 비전서 자동 배합	검사 결과를 검사로 비전서를 양방향으로 국가 노출 검사에 적용 가능한 것	W/W보구서방	가능	양	수용	배회 검증	완료
19	RVT0206001	조교화	오픈소스언어	필리핀 통신장치	HWKTC 모바일 스텝	포니올 모바일 플랫폼 커스터마이징	특수 2개용 1기가 손쉬우며 커스터마이징 플루크스 사외 원본도 발행할 준비	W/W보구서방	가능	양	수용	배회 검증	완료
20	RVT0207001	조교화	오픈소스언어	필리핀 통신장치	PWA 오픈소스 언어	PWA 오픈소스 언어 용기화	InternetID에 활용성을 극대화 해최저하고 인터넷 사외의 직접어 접하기 용케	W/W보구서방	가능	중	수용	배회 검증	완료
21	RVT0208001	선규	오픈소스언어	필리핀 통신장치	HTML 공개 화면	HTML 공개 화면 8000 편집	관리자가 AI 환경을 검토하고 운영 자료를 직접 수정할수있다	W/W보구서방	가능	양	수용	배회 검증	완료
22	RVT0208004	선규	오픈소스언어	필리핀 통신장치	3D 제작 비전	3D 제작 비전(레이저, 카메라)	산출된 배치 좌표를 4차원으로 변환해 회전축으로 이동 가능하게 한다	W/W보구서방	사용성	양	수용	배회 검증	완료
23	RVT0210001	선규	오픈소스언어	필리핀 통신장치	차별 표현기 출력	차별 표현기(LAN 직결 출력)	2D를 생생한 3D로 표현 가능 표현기 직접 전송한다	W/W보구서방	가능	중	수용	배회 검증	완료
24	BK0408001	선규	빅데이터	빅데이터 처리/분석	Celery/Numpy 분석	Celery/Numpy 분석	AI 추론을 비동기 처리로 분석의 복잡성이 부담을 가하지 않게 한다	W/W보구서방	가능	양	수용	배회 검증	완료
25	BK0408002	조교화	빅데이터	빅데이터 처리/분석	KDDA 오픈소스개발	KDDA 오픈소스개발	대기 메시지 수백 마짜 처리 가능 성능 저하 없음	W/W보구서방	친화성	중	수용	배회 검증	완료
26	BK0802001	선규	빅데이터	빅데이터 처리/분석	KDDA 오픈소스개발	KDDA 오픈소스개발	대기 메시지 수백 마짜 처리 가능 성능 저하 없음	W/W보구서방	친화성	중	수용	배회 검증	완료

요구사항구분	내용	요구사항구분	내용	과업종류구분	내용
1차 분류	S/W요구사항	2차 분류	기능	1차 분류	현행유지
	외부I/F 요구사항		데이터		고도화
	품질 요구사항		H/W		신규
	기타 요구사항		I/F		제외
			S/W		
			커뮤니케이션		
			보안성		
			신뢰성		
			사용성		
			효율성		
			이식성		
			기타		

● 요구사항 ID 전 과정 추적

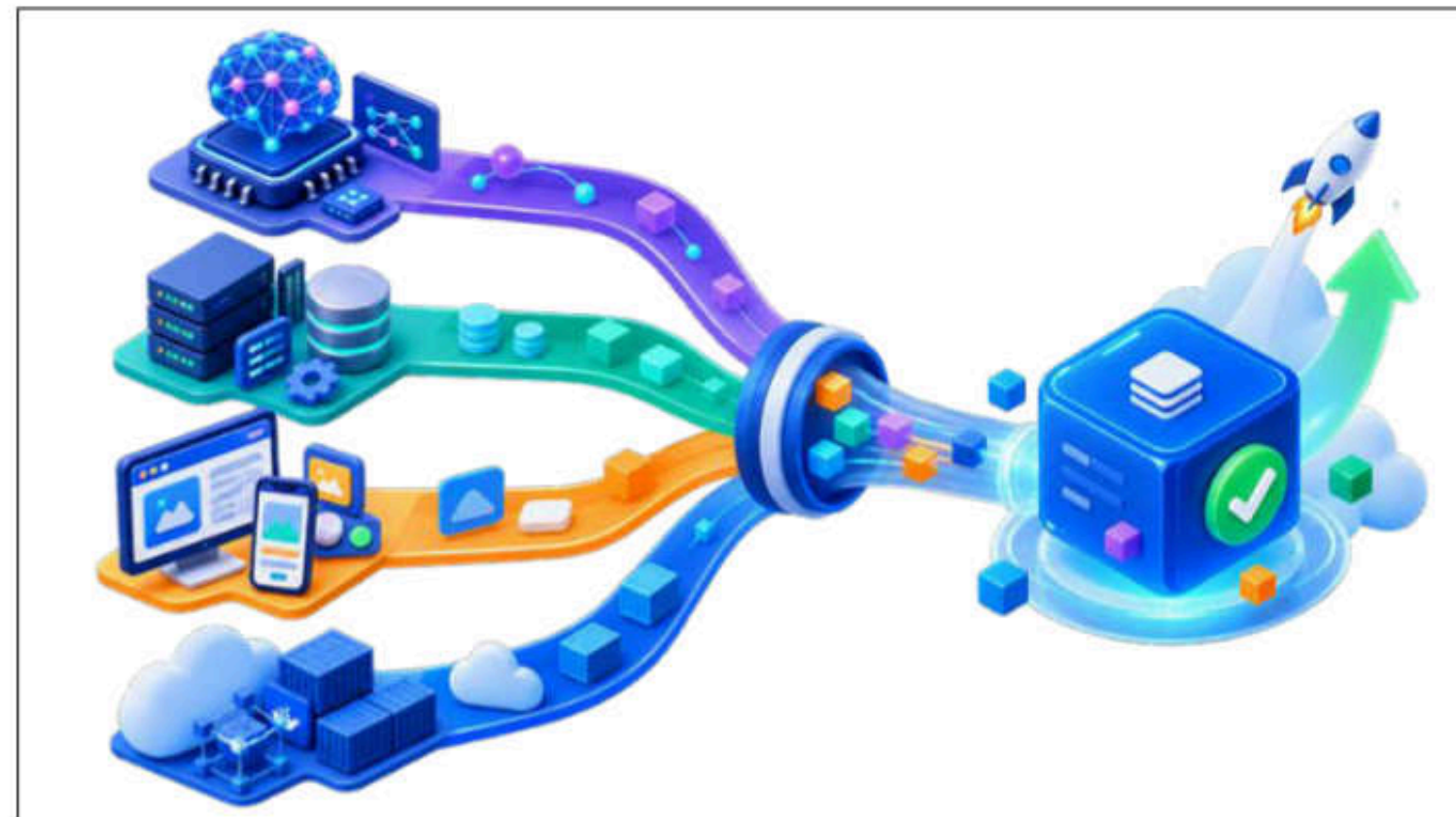
변경 이력과 검증 근거를 하나의 요구사항 ID로 1:1 보존

정의 → 구현 → 증빙의 단일 흐름

6주 타임라인 — 4-Tier 레이어 병렬 구축 과정

AI·Backend·Frontend·인프라를 모듈별 병렬로 구축한 개발 사이클

- 01 **도메인설계 · DB 스키마**
YOLO 1차 학습 · 변경 비용이 큰 계약 고정
- 02 **LangGraph파이프라인**
UBCI 산식 · 에이전트 역할 분리
- 03 **Frontend · PWA**
역할별권한 · 촬영·검수 업무 UI
- 04 **비즈니스 엔진**
3D 적재 · 동적가격 · 발주 · FDS
- 05 **AWS 실배포 · CI/CD**
관측·보안 강화 · 네 레이어 최종 합류



독립 개발 → 하나의 통합 서비스

01 FOUNDATION

먼저 만든 것

도메인·DB·모델 계약
뒤에서 바꾸면 전체가 흔들리는 기반

02 EXPERIENCE

나중으로 미룬 것

PWA·관리자 화면·복잡한 배정 규칙
산식과 API 고정 뒤 연결

03 REMAINING

끝까지 남은 것

재학습 실행·가중치 배포 자동화
관리자 승인 기반 수동 구간

개발 과정

소스 저장소 — 투 트랙 개발 구조

같은 기획을 두 트랙이 각각 구현 · 평가 기간 중 공개, 종료 후 비공개 환원

팀원 트랙

<https://github.com/jmkyong9393/wms-core-backend>

<https://github.com/jmkyong9393/wms-core-frontend>

조장 개인 트랙(제출용)

<https://github.com/jmkyong9393/wms-secret-backend>

<https://github.com/jmkyong9393/wms-secret-frontend>

선행 개발로 아키텍처와 기준 구현을 확보한 뒤, 매 주차 칸반보드로 팀원에게 과제 분배

LangGraph 파이프라인 · RAG 정책 엔진 등 동일 기능이 양 트랙에 병존하는 이유

제출본 = 개인 트랙 기준